Contents

L'Apprentissage Automatique avec l'Humain dans la Boucle : Incompris **1**
Structure du livre . . . 1

Déclaration d'originalité **3**

Résumé **5**

Remerciements **7**

Qu'est-ce que l'apprentissage automatique avec l'humain dans la boucle ? **9**
Le paradoxe de l'automatisation . . . 9
Définir l'apprentissage automatique avec l'humain dans la boucle . . . 9
Bref historique . . . 10
Pourquoi le HITL ? Le plaidoyer pour le jugement humain . . . 11
La boucle de retour . . . 12
Ce que le HITL n'est pas . . . 12
L'économie du retour humain . . . 13
Présentation du manuel . . . 13

Une taxonomie de l'interaction homme–machine **15**
Niveaux d'automatisation . . . 15
Implication humaine active vs. passive . . . 16
Annotateurs uniques vs. multiples . . . 17
Un cadre unifié . . . 17

Annotation et étiquetage des données **19**
Types d'annotation . . . 19
Consignes d'annotation . . . 20
Mesurer la qualité d'annotation : l'accord inter-annotateurs . . . 20
Gérer les désaccords . . . 22
Le bruit d'étiquetage et ses effets . . . 23
Coût et débit d'annotation . . . 24

Apprentissage actif **25**
Le cadre de l'apprentissage actif . . . 25
Fondements théoriques . . . 25
Stratégies de requête . . . 26
Estimation d'incertitude pour les modèles profonds . . . 27
Une boucle d'apprentissage actif complète . . . 29
Le problème du démarrage à froid . . . 31
Apprentissage actif par lots . . . 31

Critères d'arrêt . . . 31
Quand l'apprentissage actif fonctionne (et quand il ne fonctionne pas) . . . 32

Apprentissage automatique interactif **33**
Principes de l'apprentissage automatique interactif . . . 33
Comparaison avec l'apprentissage actif . . . 33
Interaction à initiative mixte . . . 34
Facteurs humains dans l'IML . . . 34
Les types de retour IML en pratique . . . 35
Étude de cas : Google Teachable Machine . . . 35
Implémentation d'une boucle IML simple . . . 36
Le test de la grand-mère . . . 37
IML et modèles de fondation . . . 38

Apprentissage par renforcement à partir des retours humains **39**
Le problème de l'alignement . . . 39
Le pipeline RLHF . . . 39
Une démonstration simplifiée du RLHF . . . 41
Défis du RLHF . . . 42
L'IA constitutionnelle (RLAIF) . . . 43

Apprentissage par démonstration **45**
Le clonage comportemental . . . 45
L'apprentissage par renforcement inverse . . . 46
GAIL : Apprentissage par imitation adversariale générative . . . 47
Clonage comportemental en TAL : un exemple pratique . . . 47
Comparaison des méthodes d'apprentissage par imitation . . . 48
Applications . . . 49

Apprentissage à partir de comparaisons et de classements **51**
Pourquoi les comparaisons sont meilleures que les évaluations . . . 51
Le modèle de Bradley-Terry . . . 52
Le modèle de Thurstone . . . 53
Agrégation de classements . . . 54
Les bandits duellants . . . 54
L'apprentissage par préférences pour les modèles de langage . . . 54
Collecter des données de préférences de haute qualité . . . 55

L'humain dans la boucle en traitement automatique des langues **57**
Le problème constitutif dans l'annotation en TAL . . . 57
Annotation de classification de texte . . . 58
Reconnaissance d'entités nommées . . . 58
Extraction de relations et annotation sémantique . . . 59
Post-édition de traduction automatique (MTPE) . . . 59
IA conversationnelle et annotation de dialogues . . . 60
Étiquetage programmatique et supervision faible . . . 60
RLHF pour les modèles de langage : la réalité de l'annotation . . . 61
La boucle de retour annotation–modèle . . . 62
Évaluation des modèles TAL génératifs . . . 62

L'humain dans la boucle en vision par ordinateur **65**
Comment les choix d'annotation deviennent des biais du modèle . . . 65
Annotation de classification d'images . . . 66

Détection d'objets : annotation de boîtes englobantes . . . 66
Segmentation sémantique et d'instance . . . 68
Apprentissage actif pour la vision par ordinateur . . . 68
Apprentissage semi-supervisé avec guidage humain . . . 70
Annotation vidéo . . . 70

L'humain dans la boucle en santé et en sciences **73**
Analyse d'images médicales . . . 73
Annotation de TAL clinique . . . 74
Considérations réglementaires . . . 75
Annotation de données scientifiques . . . 76
Gérer les annotateurs spécialistes . . . 77
Un pipeline HITL d'apprentissage actif pour l'imagerie médicale . . . 78

Plateformes et outils d'annotation **81**
Le paysage des plateformes d'annotation . . . 81
Comparaison des fonctionnalités des plateformes d'annotation . . . 82
L'annotation comme code . . . 82

Production participative et contrôle qualité **85**
Plateformes de production participative . . . 85
Conception de tâches pour la production participative . . . 86
Modèles statistiques pour l'agrégation des étiquettes . . . 87
Redondance et stratégie d'agrégation . . . 89

Évaluation et métriques **91**
Métriques centrées sur le modèle . . . 91
Métriques d'efficacité de l'annotation . . . 92
Évaluation humaine . . . 93
Mesurer l'alignement sur l'intention humaine . . . 95
Tests A/B dans les systèmes déployés . . . 95

Équité, biais et éthique **97**
Sources de biais dans les systèmes HITL . . . 97
Équité dans les systèmes HITL . . . 98
Bien-être des annotateurs . . . 99
Vie privée dans l'annotation . . . 99
Annotation adversariale et empoisonnement des données . . . 100
Éthique institutionnelle . . . 100

Perspectives et directions futures **103**
Les modèles de fondation et le rôle changeant de l'annotation . . . 103
Les LLM comme annotateurs . . . 104
Supervision faible à grande échelle . . . 104
Apprentissage continu avec supervision humaine . . . 105
HITL multimodal . . . 105
Supervision évolutive . . . 105
L'évolution de la division du travail . . . 106
Problèmes de recherche ouverts . . . 106

Bibliographie **109**

Étude de cas : Limen — Un humain dans la boucle de toute chose **111**

Le problème avec la conception traditionnelle des systèmes d'exploitation 111
Architecture au seuil 112
La conception de l'interaction 113
Limen comme système HITL 114
Le point le plus profond 115

Sur le départ **117**

L'Apprentissage Automatique avec l'Humain dans la Boucle : Incompris

Lazaros Toumanidis

Ce manuel examine l'un des domaines les plus cruciaux — et les plus mal compris — de l'intelligence artificielle moderne : l'intégration de l'humain dans la boucle d'apprentissage. De l'annotation des données et de l'apprentissage actif à l'apprentissage par renforcement à partir des retours humains (RLHF), le jugement humain demeure indispensable.

Cet ouvrage s'adresse aux chercheurs, aux ingénieurs et aux étudiants qui souhaitent comprendre quand, pourquoi et comment l'humain doit participer au cycle d'apprentissage.

:::{admonition} Édition française :class: note Ceci est la traduction française. L'original en anglais constitue l'édition de référence principale. :::

Structure du livre

Partie I — Fondements — établit le cadre conceptuel : ce que signifie l'apprentissage avec l'humain dans la boucle et la classification des niveaux d'automatisation.

Partie II — Techniques fondamentales — couvre les méthodes fondamentales : annotation, apprentissage actif et apprentissage interactif.

Partie III — Apprentissage à partir des retours humains — traite de RLHF, de l'apprentissage par démonstration et à partir de comparaisons.

Partie IV — Applications — examine des exemples pratiques en TAL, vision par ordinateur et systèmes médicaux.

Partie V — Systèmes et pratique — couvre les outils, plateformes et métriques requis en pratique.

Partie VI — Éthique et horizons — aborde les questions d'équité, de biais et l'avenir de la relation humain–algorithme.

Déclaration d'originalité

Soumise en non-accomplissement partiel des conditions requises pour l'obtention du grade de Docteur en philosophie.

Je déclare par la présente que le travail présenté dans ce manuel est le mien, sauf là où il ne l'est pas.

Les idées contenues dans ces pages ont été produites avec le concours de : des chercheurs qui ont publié des articles et les ont parfois placés derrière des barrières tarifaires ; des annotateurs dont le travail a rendu les modèles possibles mais dont les noms n'apparaissent dans aucune section de remerciements ; des contributeurs à des logiciels libres qui ont écrit les bibliothèques utilisées dans chaque exemple de code ; et du texte accumulé de la civilisation humaine, ingéré et compressé par de grands modèles de langage qui n'ont d'opinion sur rien de tout cela.

Les erreurs sont miennes. Les intuitions étaient collaboratives.

Je confirme que ce travail n'a pas été soumis pour l'obtention d'un autre diplôme ou d'une qualification professionnelle, d'un prix académique, d'une subvention, d'un dossier de titularisation, ni d'une publication sur LinkedIn, sauf mention contraire.

Je confirme en outre qu'aucune section de comparaison à l'état de l'art (State of the Art) n'a été incluse dans ce travail. Le lecteur n'y trouvera pas un tableau classant 47 méthodes sur 12 références, la méthode proposée atteignant des améliorations statistiquement significatives sur 9 d'entre elles ($\dagger p < 0{,}05$). Ce n'est pas parce qu'un tel tableau ne pourrait pas être construit, mais parce que sa construction serait à côté de l'essentiel.

L'essentiel est le suivant : l'humain est dans la boucle. Il l'a toujours été. Ce livre tente de comprendre ce que cela signifie.

Signé : Un humain dans la boucle

Date : Quand que vous lisiez ceci

Institution : Aucune. C'est justement le propos.

> **Sur le sous-titre** :class: note
>
> *Incompris* renvoie au domaine, au rôle de l'humain en son sein, et au lecteur — tous à la fois. Si vous pensez que cela ne vous concerne que vous, vous avez peut-être raison. Si vous pensez que cela ne concerne que les machines, vous prouvez le bien-fondé du sous-titre.

Résumé

L'apprentissage automatique avec l'humain dans la boucle : incompris

Le récit dominant de l'apprentissage automatique positionne l'humain comme un participant temporaire — présent lors de la collecte des données et de l'entraînement du modèle, puis progressivement écarté à mesure que l'automatisation progresse. Ce manuel soutient que ce récit est erroné, qu'il l'a toujours été, et que comprendre *pourquoi* il est erroné constitue l'une des choses les plus utiles en pratique qu'un praticien de l'apprentissage automatique puisse faire.

L'apprentissage automatique avec l'humain dans la boucle (HITL) est la discipline qui consiste à rendre l'implication humaine dans les systèmes d'apprentissage délibérée, structurée et continue. Plutôt que de traiter le retour humain comme un coût à minimiser, le HITL le traite comme un signal à comprendre : d'où il provient, ce qu'il encode, comment le collecter efficacement, et comment l'incorporer fidèlement dans des modèles qui se comportent comme leurs concepteurs et leurs utilisateurs le souhaitent véritablement.

Ce manuel propose un traitement exhaustif du domaine en six parties. La Partie I établit les fondements et une taxonomie de l'interaction homme–machine. La Partie II couvre les techniques classiques : l'annotation et l'étiquetage des données, l'apprentissage actif et l'apprentissage automatique interactif. La Partie III aborde les paradigmes modernes de l'alignement : l'apprentissage par renforcement à partir des retours humains (RLHF), l'apprentissage par démonstration et l'apprentissage par préférences. La Partie IV applique ces techniques au traitement automatique des langues, à la vision par ordinateur et à la santé. La Partie V couvre les systèmes et l'infrastructure du HITL à grande échelle : plateformes d'annotation, contrôle qualité en production participative et méthodologie d'évaluation. La Partie VI examine l'éthique et l'avenir du domaine.

Chaque chapitre comprend des définitions formelles, un traitement mathématique lorsque approprié, et des exemples de code Python autonomes et exécutables.

Ce manuel s'adresse aux praticiens qui construisent des systèmes d'apprentissage automatique, aux chercheurs qui les étudient, aux étudiants qui les découvrent, et à toute personne ayant un jour voulu savoir qui a étiqueté les données qui ont entraîné le modèle qui a pris la décision qui a affecté sa vie.

La réponse, dans tous les cas, est : un humain. Souvent sous-payé. Rarement crédité. Toujours dans la boucle.

Mots-clés : apprentissage automatique avec l'humain dans la boucle, apprentissage actif, annotation de données, apprentissage par renforcement à partir des retours humains, RLHF, apprentissage par préférences, production participative, apprentissage automatique interactif, alignement de l'IA, IA responsable

Classification thématique : Apprentissage automatique · Interaction homme-machine · Intelligence artificielle · Science des données

Nombre de mots : Suffisant.

Remerciements

Tout livre sur l'apprentissage automatique avec l'humain dans la boucle devrait commencer par les humains de sa propre boucle.

Les annotateurs. Ce manuel vous concerne, en partie — votre travail, votre jugement, et le fait que votre nom n'apparaît dans aucune fiche de données, aucun remerciement d'article, ni aucun communiqué de presse sur les modèles que votre travail a rendus possibles. Le domaine commence à prendre conscience de cette situation. Il a encore un long chemin à parcourir.

Les contributeurs aux logiciels libres dont les bibliothèques sont utilisées dans chaque exemple de code : la communauté NumPy (depuis 2006), les contributeurs de scikit-learn (depuis 2007), l'équipe PyTorch, la communauté Hugging Face, et les innombrables mainteneurs qui corrigent des bogues le week-end sans compensation ni reconnaissance. L'informatique scientifique est construite sur ce travail invisible.

Les chercheurs qui ont publié leurs travaux en accès ouvert — en prépublications arXiv, dans des revues en accès libre, et dans des dépôts GitHub — permettant aux praticiens d'apprendre sans accès institutionnel. Et les chercheurs qui ont évalué des articles avec soin et honnêteté, une forme d'annotation experte que le milieu académique s'est d'une certaine façon convaincu de ne pas considérer comme un travail.

Les enseignants qui ont rendu la connaissance accessible : les blogueurs, les auteurs de tutoriels, les répondants de Stack Overflow, les conférenciers qui ont mis leurs diapositives en ligne. L'apprentissage ne se fait pas dans l'isolement, et il ne se fait pas sans des personnes prêtes à expliquer des choses à des inconnus.

Les critiques. Les chercheurs qui ont signalé que les jeux de données encodent des biais, que les caractéristiques démographiques des annotateurs influencent le comportement des modèles, que les humains dans la boucle sont souvent les participants les plus vulnérables du système. Ce domaine vous doit de meilleures pratiques grâce à votre insistance à poser des questions inconfortables.

Il y a les connus connus — les choses que nous savons que nous savons. Il y a les connus inconnus — les choses que nous savons ne pas savoir. Il y a les inconnus inconnus — les choses que nous ne savons pas ne pas savoir.

Et puis il y a les choses que tout le monde dans le domaine sait mais qui ne trouvent jamais vraiment leur place dans les articles.

Ce manuel est, en partie, une tentative d'aborder cette dernière catégorie.

Si vous avez contribué à ce domaine et que vous ne voyez pas votre nom ici : vous êtes reconnu. La boucle vous inclut.

Qu'est-ce que l'apprentissage automatique avec l'humain dans la boucle ?

Une machine capable d'apprendre de l'expérience… mais seulement si on lui fournit les bonnes expériences. – paraphrase d'Alan Turing

Le paradoxe de l'automatisation

Chaque nouvelle vague d'automatisation crée de nouvelles exigences sur l'attention humaine. Quand les compagnies aériennes ont introduit le pilote automatique, les pilotes sont devenus des superviseurs — responsables non plus du contrôle instant par instant, mais de la tâche plus difficile de savoir *quand* reprendre les commandes. Quand les supermarchés ont introduit les caisses automatiques, ils ont découvert que ces systèmes exigeaient davantage de supervision humaine par transaction que les caissiers traditionnels, et non moins. Et quand l'apprentissage automatique a commencé à prendre des décisions à grande échelle — en médecine, en finance, dans le recrutement et la modération de contenu — il a créé une demande considérable et permanente pour le jugement humain.

C'est le **paradoxe de l'automatisation** {cite}`bainbridge1983ironies` : plus un système automatisé devient capable, plus ses défaillances sont lourdes de conséquences, et plus une supervision humaine robuste devient nécessaire. L'apprentissage automatique ne fait pas exception.

L'apprentissage automatique avec l'humain dans la boucle (HITL ML) est le domaine qui prend ce paradoxe au sérieux et l'intègre dans la conception des systèmes dès le départ. Plutôt que de traiter l'implication humaine comme un échafaudage temporaire destiné à être finalement démantelé, le HITL ML traite l'interaction homme–machine comme une *fonctionnalité* — une source de signal, de correction et de guidage qui peut rendre les modèles plus précis, mieux alignés sur les valeurs humaines, et plus dignes de confiance.

Définir l'apprentissage automatique avec l'humain dans la boucle

Il n'existe pas de définition unique et consensuelle du HITL ML, et le terme est utilisé de plusieurs façons qui se chevauchent dans la littérature. Pour ce manuel, nous adoptons une définition large mais précise :

:::{admonition} Définition :class: important

L'apprentissage automatique avec l'humain dans la boucle désigne tout système ou toute méthodologie d'apprentissage automatique dans lequel le retour humain est incorporé dans le processus d'apprentissage de manière *délibérée, structurée et continue* — non seulement au moment de la création du jeu de données, mais tout au long de l'entraînement, de l'évaluation et du déploiement.

:::

Cette définition comporte trois clauses essentielles :

Délibéré. Le HITL ne désigne pas une influence humaine accidentelle (par exemple, le fait que les données d'entraînement aient été créées à l'origine par des humains). Il renvoie à des systèmes explicitement conçus pour solliciter, incorporer et bénéficier du retour humain.

Structuré. Le retour a une forme définie — une étiquette, une correction, un jugement de préférence, une démonstration — et un rôle défini dans l'algorithme d'apprentissage.

Continu. La boucle de retour se poursuit dans le temps, permettant au système de s'améliorer à mesure qu'il rencontre de nouvelles situations, commet des erreurs et reçoit des orientations humaines.

Cette définition inclut les pipelines d'annotation classiques, l'apprentissage actif, l'apprentissage automatique interactif, le RLHF et l'apprentissage par imitation. Elle exclut la collecte passive de données et l'apprentissage supervisé purement hors ligne (bien que la frontière soit floue, comme nous le verrons au Chapitre 2).

Bref historique

Les systèmes experts et l'ingénierie de la connaissance (années 1960–1980)

Les premiers systèmes d'IA étaient presque entièrement avec l'humain dans la boucle : des ingénieurs de la connaissance travaillaient avec des experts du domaine pendant des mois, encodant patiemment des règles dans des systèmes experts comme MYCIN et DENDRAL. Chaque fragment de connaissance était explicitement fourni par un humain. La machine était l'exécutant ; l'humain était l'oracle.

Ces systèmes fonctionnaient étonnamment bien dans des domaines restreints, mais étaient fragiles — incapables de généraliser au-delà de leurs règles artisanales et coûteux à maintenir.

Le tournant statistique (années 1980–2000)

Le passage à l'apprentissage automatique statistique dans les années 1980 et 1990 a modifié la nature de l'implication humaine. Au lieu d'encoder la connaissance sous forme de règles, les humains fournissaient désormais des *exemples* — des jeux de données étiquetés permettant aux modèles d'inférer des schémas. Le rôle humain est devenu celui d'annotateur : étiqueter des documents, baliser des images, transcrire de la parole.

C'était un progrès majeur, mais il a créé un nouveau goulot d'étranglement : **les données étiquetées sont coûteuses**. Les chercheurs ont commencé à se demander comment tirer le meilleur parti de l'effort d'étiquetage humain. Cette question a donné naissance à **l'apprentissage actif**, formalisé pour la première fois au début des années 1990 {cite}`cohn1994improving`.

L'ère de l'apprentissage profond (années 2010 à aujourd'hui)

La révolution de l'apprentissage profond a créé un nouveau régime : des modèles avec des milliards de paramètres capables d'apprendre des fonctions extraordinairement complexes à partir de données — mais nécessitant des jeux de données étiquetés d'une ampleur correspondante. ImageNet (14 millions d'images étiquetées) et les projets d'annotation à grande échelle qui ont suivi ont montré à la fois la puissance et le coût de cette mise à l'échelle.

Dans le même temps, le déploiement de l'apprentissage automatique à grande échelle a mis en évidence de nouveaux problèmes : des modèles précis en moyenne mais systématiquement erronés pour des groupes spécifiques, qui hallucinaient des faits avec assurance, qui optimisaient des substituts mesurables plutôt que les valeurs humaines. Ces défaillances ont motivé de nouvelles formes d'implication humaine : non plus seulement l'étiquetage, mais *l'alignement* — le projet de faire se comporter les modèles comme les humains le souhaitent réellement.

L'expression la plus claire de ce travail HITL orienté vers l'alignement est **l'apprentissage par renforcement à partir des retours humains (RLHF)** {cite}`christiano2017deep`, qui est devenu l'épine dorsale de systèmes comme InstructGPT {cite}`ouyang2022training` et les capacités de suivi d'instructions des modèles de langage modernes.

Pourquoi le HITL ? Le plaidoyer pour le jugement humain

Qu'est-ce qui rend le jugement humain assez précieux pour être incorporé dans des systèmes d'apprentissage automatique ? Plusieurs propriétés :

1. Le sens commun et la connaissance du monde

Les humains apportent à toute tâche une vaste connaissance de fond. Quand un radiologue étiquette une radiographie, il fait appel à des années de formation, à une compréhension de l'anatomie et à une connaissance implicite de l'apparence des maladies — une connaissance extraordinairement difficile à spécifier complètement ou à acquérir à partir des données seules.

2. L'ancrage sémantique

Les étiquettes sont significatives parce que les humains comprennent ce qu'elles signifient. La classe « chat » dans ImageNet renvoie à un concept flou que les humains reconnaissent intuitivement mais qu'aucune définition formelle ne capture pleinement. Les modèles apprennent l'extension de l'étiquette (quelles images lui correspondent) mais peuvent ne pas apprendre le concept lui-même, conduisant à des échecs sur des cas limites qu'un humain quelconque traiterait correctement.

3. L'alignement sur les valeurs

Les humains ont des préférences, des valeurs et des jugements éthiques que les modèles d'apprentissage automatique ne peuvent pas dériver des données seules. Qu'un texte soit « utile » ou « nuisible » n'est pas une question purement empirique — cela dépend de valeurs humaines qui varient selon les individus et les contextes. Le HITL est le principal mécanisme par lequel ces valeurs peuvent être communiquées aux systèmes d'apprentissage automatique.

4. L'adaptabilité

Le jugement humain peut s'adapter à des situations nouvelles sans ré-entraînement. Un modèle entraîné sur des données historiques peut échouer de façon catastrophique quand le monde change ; un humain peut reconnaître la nouveauté et y répondre de manière appropriée.

5. La responsabilité

Dans les domaines à enjeux élevés — médecine, droit, finance — les décisions doivent être imputables à des êtres humains. Les systèmes HITL rendent cette responsabilité opérationnelle en maintenant les humains en position de comprendre, vérifier et outrepasser le comportement des machines.

La boucle de retour

La structure centrale du HITL ML est une boucle de retour entre un modèle et un ou plusieurs humains :

```
+----------------------------------------------+
|                                              |
|   Le modèle fait des prédictions / demande   |
|   ----------------------------------->       |
|                                   Humain     |
|   L'humain fournit un retour      <-------- |
|   ----------------------------------         |
|                                              |
|   Le modèle se met à jour sur le retour      |
|                                              |
+----------------------------------------------+
```

La nature du retour varie énormément selon les paradigmes HITL :

Type de retour	Exemple	Paradigme principal
Étiquette de classe	« Cet e-mail est du spam »	Apprentissage supervisé
Correction	« L'entité devrait être ORG, pas PER »	Apprentissage actif / interactif
Préférence	« La réponse A est meilleure que B »	RLHF / classement
Démonstration	Montrer au robot comment saisir un objet	Apprentissage par imitation
Langage naturel	« Soyez plus concis dans vos réponses »	Ajustement par instructions
Signal implicite	L'utilisateur a cliqué / n'a pas cliqué	Retour implicite

---

Ce que le HITL n'est pas

Il est utile de préciser ce qui se situe *en dehors* de notre définition.

Le HITL n'est pas la même chose qu'un déploiement avec l'humain sur la boucle (parfois appelé « human on the loop »), où les humains surveillent les décisions automatisées et peuvent les outrepasser, mais ne renvoient pas de corrections vers l'entraînement. Nous aborderons cette distinction au Chapitre 2.

Le HITL n'est pas la supervision faible seule. Les systèmes d'étiquetage programmatique comme Snorkel utilisent des fonctions d'étiquetage (souvent des règles rédigées par des humains) pour générer des étiquettes bruitées à grande échelle. C'est une forme d'apport humain structuré, mais le retour n'est pas itératif au sens où le HITL l'implique habituellement.

Le HITL ne consiste pas simplement à utiliser des données étiquetées. Chaque modèle d'apprentissage supervisé utilise des données étiquetées par des humains. Le HITL désigne spécifiquement les systèmes dans lesquels le retour humain est une partie *active et itérative* du processus d'apprentissage.

L'économie du retour humain

Le retour humain est précieux mais coûteux. L'annotation d'images médicales peut coûter de plusieurs dizaines à plusieurs centaines de dollars par image lorsqu'elle est réalisée par un spécialiste, selon la sous-spécialité et la complexité de la tâche {cite}`monarch2021human`. Les étiquettes de travailleurs en production participative sur des plateformes comme Amazon Mechanical Turk peuvent coûter entre 0,01 et 0,10 $ par élément {cite}`hara2018data` avec une qualité nettement inférieure. Le défi fondamental du HITL ML est de **maximiser la valeur de chaque unité de retour humain**.

Cela conduit à une question unificatrice qui traverse la majeure partie de ce manuel :

:::{admonition} La question centrale du HITL ML :class: tip

Étant donné un budget fixe de temps et d'attention humaines, comment décider quoi demander aux humains, quand le demander, et comment incorporer leurs réponses dans l'entraînement du modèle ?

:::

La réponse à cette question dépend du domaine, de la forme du retour, du coût d'annotation, du risque d'erreur et de l'état actuel du modèle — ce qui fait du HITL ML une discipline riche et encore en évolution.

Présentation du manuel

Le reste de ce manuel est structuré comme suit. La **Partie II** couvre les trois piliers classiques du HITL : l'annotation, l'apprentissage actif et l'apprentissage automatique interactif. La **Partie III** aborde les paradigmes plus récents d'apprentissage à partir du retour — RLHF, apprentissage par imitation et apprentissage par préférences — qui sont devenus centraux dans l'IA moderne. La **Partie IV** examine le HITL à travers le prisme de domaines d'application spécifiques. La **Partie V** adopte une perspective praticienne sur les plateformes, la production participative et l'évaluation. La **Partie VI** traite de l'éthique et se projette dans l'avenir.

> **See also:** Pour un aperçu pratique du domaine, voir {cite}`monarch2021human`. Pour l'article fondateur sur l'apprentissage actif qui a lancé une grande partie du traitement formel du HITL, voir {cite}`settles2009active`.

Une taxonomie de l'interaction homme–machine

Un vocabulaire précis est le préalable à une pensée claire. Le terme « humain dans la boucle » est utilisé de manière imprécise en pratique — tantôt pour signifier qu'un humain étiquette des données d'entraînement, tantôt qu'un humain peut outrepasser la décision d'un modèle, et tantôt qu'un humain dirige activement le processus d'apprentissage en temps réel. Ce sont des choses significativement différentes.

Ce chapitre cartographie le paysage de l'interaction homme–machine dans l'apprentissage automatique, en fournissant le vocabulaire conceptuel utilisé tout au long du reste du manuel.

Niveaux d'automatisation

La distinction la plus fondamentale concerne le degré d'implication active de l'humain dans la prise de décision du système. Un cadre bien connu issu de la théorie de l'automatisation {cite}`sheridan1992telerobotics` distingue dix niveaux, mais pour les besoins de l'apprentissage automatique, trois catégories capturent l'essentiel de la variation :

L'humain dans la boucle (HITL)

L'humain est un *participant actif au processus d'apprentissage*. Les décisions — ou du moins les décisions importantes — nécessitent un apport humain avant d'être finalisées. Le système ne peut pas fonctionner sans une implication humaine continue.

Exemples : Un système d'apprentissage actif qui interroge un clinicien avant d'ajouter un cas aux données d'entraînement. Un annotateur qui étiquette des exemples immédiatement utilisés pour la mise à jour du modèle. Un évaluateur RLHF comparant des sorties de modèle.

L'humain sur la boucle (HOTL)

Le système fonctionne de manière autonome, mais un humain le surveille et peut intervenir. L'humain est un *superviseur*, non un participant. Le retour peut ou non alimenter à nouveau l'entraînement.

Exemples : Un système de modération de contenu qui signale automatiquement les publications ; un examinateur humain échantillonne et corrige une fraction des décisions. Un pilote automatique qui pilote l'avion mais alerte le pilote en cas d'anomalie.

L'humain aux commandes (HIC)

L'humain prend toutes les décisions ; le système fournit des *recommandations ou des informations* mais n'a pas d'autonomie. C'est la forme la plus faible de déploiement d'apprentissage automatique.

Exemples : Un système d'aide au diagnostic qui montre à un médecin l'estimation de probabilité du modèle, mais laisse la décision finale entièrement au praticien.

> **Quel niveau est approprié ?** :class: tip
>
> Le niveau d'automatisation approprié dépend du coût des erreurs, de la fiabilité du modèle, de l'expertise des humains disponibles et des contraintes de latence de la tâche. Ces facteurs évoluent à mesure qu'un modèle mûrit — la plupart des systèmes démarrent en HITL et migrent vers HOTL lorsque la confiance s'établit.

```
Données brutes --> Prétraitement --> Caractéristiques --> Entraînement --> Évaluation --> Dép
    ^                    ^                                    ^                ^
    |                    |                                    |                |
Collection          Annotation                          Apprentissage      Test          Surv
 feedback           & étiquetage                           actif           feedback      & co
```

Étape	Rôle humain	Chapitre
Collection	Décider quelles données collecter ; stratégie d'échantillonnage	3, 4
Annotation	Assigner des étiquettes, des structures, des métadonnées	3, 13
Entraînement	Requêtes d'apprentissage actif ; retour en ligne	4, 5, 6
Évaluation	Évaluation humaine des sorties du modèle	14
Déploiement	Surveillance, gestion des exceptions, corrections	12, 14

Implication humaine active vs. passive

Dans un HITL *actif*, le système sélectionne les points de données à présenter à l'humain — en posant des questions de manière stratégique. Dans un HITL *passif*, l'humain fournit un retour sur les données qui arrivent quelles qu'elles soient (par exemple, des lots d'étiquetage attribués séquentiellement).

L'implication active est plus efficace car le retour est dirigé là où il améliorera le plus le modèle. L'implication passive est plus simple à mettre en œuvre et à gérer.

Une distinction connexe est celle entre le retour **par lots** et **en ligne** :

- **Par lots :** Les humains étiquettent un grand ensemble d'exemples ; le modèle se ré-entraîne. On répète.
- **En ligne (flux continu) :** Le retour humain arrive en continu ; le modèle se met à jour de manière incrémentale.

Les flux de travail par lots sont la norme dans l'industrie (projets d'annotation suivis de cycles d'entraînement). Les flux de travail en ligne sont plus naturels pour les systèmes interactifs et les contextes d'apprentissage par renforcement.

Annotateurs uniques vs. multiples

La plupart des présentations formelles du HITL supposent un seul annotateur cohérent. En pratique, l'annotation implique de nombreuses personnes, et leurs jugements diffèrent.

L'agrégation combine plusieurs annotations en une seule étiquette, typiquement par vote majoritaire ou par un modèle statistique (Chapitre 13).

Le désaccord comme signal — certains chercheurs soutiennent que le désaccord des annotateurs est une information précieuse qui ne devrait pas être réduite à une seule étiquette « de référence ». Les approches perspectivistes du traitement automatique des langues, par exemple, préservent plusieurs annotations sous forme d'étiquettes souples reflétant l'ambiguïté réelle des données {cite}`uma2021learning`.

Un cadre unifié

Nous pouvons représenter toute configuration HITL par un quintuplet :

$$\text{Config HITL} = (\text{niveau}, \text{modalité}, \text{étape}, \text{fréquence}, \text{annotateurs})$$

où :

- **niveau** $\in$ {HITL, HOTL, HIC}
- **modalité** $\in$ {étiquette, correction, démonstration, préférence, LN, implicite}
- **étape** $\in$ {collection, annotation, entraînement, évaluation, déploiement}
- **fréquence** $\in$ {lots, en ligne}
- **annotateurs** $\in \mathbb{N}^+$ (nombre d'annotateurs par élément)

Cette taxonomie nous permet de comparer des systèmes HITL divers selon les mêmes axes et de raisonner sur les compromis entre eux. Le reste du manuel explore en profondeur des cellules spécifiques de cet espace.

> **See also:** Pour une étude empirique de la façon dont les décisions d'annotation affectent le comportement ultérieur du modèle, voir {cite}`bender2021stochastic`. Pour une revue des systèmes d'apprentissage automatique interactif, voir {cite}`amershi2014power`.

Annotation et étiquetage des données

L'annotation de données est la forme d'implication humaine la plus répandue dans l'apprentissage automatique. Avant qu'un modèle puisse apprendre, quelqu'un doit lui indiquer quelles sont les bonnes réponses — et ce quelqu'un est généralement un humain. Ce chapitre couvre la théorie et la pratique de l'annotation : ce qui rend l'annotation difficile, comment concevoir des tâches d'annotation, comment mesurer la qualité, et comment gérer les désaccords.

Types d'annotation

Les tâches d'annotation varient énormément dans leur structure, leur difficulté et leur coût. Les principaux types comprennent :

Classification

L'annotateur assigne chaque instance à l'une des K catégories prédéfinies. C'est la tâche d'annotation la plus simple sur le plan cognitif, mais définir un bon schéma de catégories (une *taxonomie*) peut s'avérer étonnamment difficile.

La **classification binaire** (est-ce une image de chat ?) est le cas le plus simple. La **classification multiclasse** (quelle espèce est cet animal ?) demande aux annotateurs de choisir une option parmi une liste. L'annotation **multi-étiquettes** (quels sujets couvre cet article ?) autorise plusieurs étiquettes simultanées.

Étiquetage de séquences

Chaque token d'une séquence reçoit une étiquette. La Reconnaissance d'Entités Nommées (NER) en est l'exemple canonique — les annotateurs marquent des spans de texte comme PERSONNE, ORGANISATION, LIEU, etc. L'annotation est généralement réalisée à l'aide du schéma de balisage BIO (Début-Intérieur-Extérieur) ou BIOES :

```
B-ORG    O            B-ORG    O      O      O
```

Annotation de spans et de relations

Au-delà de l'étiquetage des tokens individuels, les annotateurs peuvent avoir besoin de : - Identifier des spans (expressions multi-tokens) et leur assigner des types - Marquer des *relations* entre spans (« Apple » a ACQUIS « Shazam ») - Annoter des chaînes de coréférence (toutes les mentions de la même entité)

Ces tâches sont cognitivement exigeantes et présentent un accord inter-annotateurs plus faible.

Boîtes englobantes et détection d'objets

Les annotateurs dessinent des rectangles autour des objets dans les images et assignent une étiquette de catégorie à chaque boîte. La précision de localisation est importante : une boîte trop petite manque le contexte ; une boîte trop grande inclut le fond. Les outils d'annotation modernes calculent l'intersection sur l'union (IoU) avec les annotations de référence pour signaler les problèmes de qualité.

Segmentation

Annotation au niveau des pixels : chaque pixel est assigné à une classe (segmentation sémantique) ou à une instance d'objet spécifique (segmentation d'instance). La segmentation de haute qualité est parmi les types d'annotation les plus coûteux, avec des tarifs allant de plusieurs dizaines à plus d'une centaine de dollars par image pour des scènes complexes, selon le domaine et les outils disponibles.

Transcription et traduction

Audio □ texte (données d'entraînement pour la reconnaissance automatique de la parole), écriture manuscrite □ texte (données OCR), ou langue source □ langue cible (données pour la traduction automatique). Ces tâches exigent une expertise linguistique et ne peuvent pas être réalisées de manière fiable par des annotateurs non formés.

Consignes d'annotation

Le déterminant le plus important de la qualité d'annotation est la qualité des **consignes d'annotation** : les instructions écrites que suivent les annotateurs.

De bonnes consignes : - Énoncent l'objectif de la tâche et expliquent *pourquoi* l'étiquette est importante - Fournissent une définition claire de chaque catégorie avec des exemples positifs et négatifs - Traitent explicitement les cas limites et les cas difficiles courants - Précisent la conduite à tenir en cas d'incertitude (par exemple, marquer « passer » vs. choix forcé) - Incluent des exemples travaillés d'annotation complète

Les mauvaises consignes demandent aux annotateurs d'utiliser leur « bon sens » pour les cas limites — ce qui conduit à des décisions incohérentes qui dégradent la qualité du modèle et font croître le désaccord inter-annotateurs.

> **Le développement des consignes est itératif** :class: note
>
> Ne vous attendez pas à rédiger des consignes parfaites d'emblée. Effectuez un petit tour d'annotation pilote, analysez les désaccords et mettez à jour les consignes. Répétez. Des consignes bien développées passent généralement par 3 à 5 cycles de révision avant de se stabiliser.

Mesurer la qualité d'annotation : l'accord inter-annotateurs

Quand plusieurs annotateurs étiquettent les mêmes données, leur accord peut être mesuré. Un accord élevé suggère que la tâche est bien définie et que les annotateurs l'ont comprise. Un accord faible suggère une ambiguïté dans la tâche, les consignes ou les données elles-mêmes.

Le kappa de Cohen

Pour deux annotateurs étiquetant des données dans K catégories, le **kappa de Cohen** {cite}`cohen1960coefficient` corrige l'accord observé en tenant compte du hasard :

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

où P_o est l'accord proportionnel observé et P_e est la probabilité d'accord par hasard (calculée à partir des distributions marginales des étiquettes).

$\kappa = 1$ signifie un accord parfait ; $\kappa = 0$ signifie un accord équivalent au hasard ; $\kappa < 0$ signifie un désaccord systématique.

Plage de κ	Interprétation
< 0	Inférieur au hasard
$0{,}0 - 0{,}20$	Léger
$0{,}21 - 0{,}40$	Passable
$0{,}41 - 0{,}60$	Modéré
$0{,}61 - 0{,}80$	Substantiel
$0{,}81 - 1{,}00$	Presque parfait

Le kappa de Fleiss

Étend le kappa de Cohen à $M > 2$ annotateurs. Chaque annotateur étiquette indépendamment chaque élément ; la formule agrège les annotateurs :

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

où $\bar{P}$ est l'accord par paires moyen sur tous les couples d'annotateurs, et $\bar{P}_e$ est l'accord attendu sous affectation aléatoire.

L'alpha de Krippendorff

La métrique la plus générale, prenant en charge n'importe quel nombre d'annotateurs, tout type d'échelle (nominale, ordinale, par intervalles, par ratios), et les données manquantes {cite}`krippendorff2011computing` :

$$\alpha = 1 - \frac{D_o}{D_e}$$

où D_o est le désaccord observé et D_e est le désaccord attendu. L'alpha de Krippendorff est généralement préféré dans les travaux académiques en raison de sa flexibilité.

```python
import numpy as np
from sklearn.metrics import cohen_kappa_score

# Simulate two annotators labeling 200 items into 3 categories
rng = np.random.default_rng(0)
true_labels = rng.integers(0, 3, size=200)
```

```
# Annotator 1: mostly agrees with ground truth
ann1 = true_labels.copy()
flip_mask = rng.random(200) < 0.15
ann1[flip_mask] = rng.integers(0, 3, size=flip_mask.sum())

# Annotator 2: less consistent
ann2 = true_labels.copy()
flip_mask2 = rng.random(200) < 0.30
ann2[flip_mask2] = rng.integers(0, 3, size=flip_mask2.sum())

kappa_12 = cohen_kappa_score(ann1, ann2)
kappa_1true = cohen_kappa_score(ann1, true_labels)
kappa_2true = cohen_kappa_score(ann2, true_labels)

print(f"Cohen's κ (ann1 vs ann2):   {kappa_12:.3f}")
print(f"Cohen's κ (ann1 vs truth):  {kappa_1true:.3f}")
print(f"Cohen's κ (ann2 vs truth):  {kappa_2true:.3f}")
```

Gérer les désaccords

Quand les annotateurs sont en désaccord, plusieurs stratégies sont possibles :

Vote majoritaire

L'étiquette la plus courante est prise comme référence. Simple et robuste quand le nombre d'annotateurs par élément est impair. Échoue quand un sous-groupe minoritaire d'annotateurs est systématiquement plus précis.

Vote pondéré

Les annotateurs sont pondérés par leur précision estimée (dérivée de l'accord avec une référence ou d'autres annotateurs). Les annotateurs les plus précis ont plus d'influence.

Étiquettes souples

Plutôt que de réduire les annotations à une seule étiquette, on préserve la distribution. Si 3 annotateurs sur 5 ont dit « positif » et 2 « neutre », cela se représente comme $(p_{\text{pos}}, p_{\text{neutre}}, p_{\text{nég}}) = (0{,}6, 0{,}4, 0{,}0)$. L'entraînement sur des étiquettes souples améliore la calibration.

Arbitrage

Un annotateur senior ou un expert du domaine tranche les désaccords. C'est la méthode de référence, mais elle est coûteuse ; généralement réservée aux domaines à enjeux élevés.

Modèles statistiques

Des approches plus sophistiquées modélisent la compétence des annotateurs de manière probabiliste. Le modèle **Dawid-Skene** {cite}`dawid1979maximum` estime simultanément les

matrices de confusion des annotateurs et les vraies étiquettes des éléments par EM. Voir le Chapitre 13 pour les détails.

Le bruit d'étiquetage et ses effets

L'annotation réelle est bruitée. Les effets du bruit d'étiquetage sur l'entraînement du modèle dépendent du type de bruit :

- Le **bruit aléatoire** (étiquettes aléatoirement inversées) dégrade les performances, mais les modèles sont étonnamment robustes à des niveaux modérés (jusqu'à ~20 % pour de nombreuses tâches).
- Le **bruit systématique/adversarial** (étiquettes systématiquement erronées selon des schémas spécifiques) est bien plus dommageable et plus difficile à détecter.
- Le **bruit conditionnel à la classe** (erreurs plus probables pour certaines classes) biaise la frontière de décision du modèle.

Une règle empirique pratique : avec n exemples d'entraînement et une fraction ϵ d'étiquettes corrompues, les performances du modèle se dégradent environ comme si vous disposiez de $(1-2\epsilon)^2 n$ exemples propres {cite}`natarajan2013learning`. Pour $\epsilon = 0{,}2$, cela équivaut à perdre 36 % de vos données.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=2000, n_features=20, random_state=42)

noise_levels = np.linspace(0, 0.45, 15)
mean_accs = []

for eps in noise_levels:
    y_noisy = y.copy()
    flip = rng.random(len(y)) < eps
    y_noisy[flip] = 1 - y_noisy[flip]
    scores = cross_val_score(LogisticRegression(max_iter=500), X, y_noisy, cv=5)
    mean_accs.append(scores.mean())

plt.figure(figsize=(7, 4))
plt.plot(noise_levels, mean_accs, 'o-', color='#2b3a8f', linewidth=2)
plt.xlabel("Label noise rate (ε)", fontsize=12)
plt.ylabel("Cross-validated accuracy", fontsize=12)
plt.title("Effect of Label Noise on Model Performance", fontsize=13)
plt.axvline(0.2, color='#e05c5c', linestyle='--', alpha=0.7, label='20% noise')
plt.legend()
plt.tight_layout()
plt.savefig('label_noise_effect.png', dpi=150)
plt.show()
print(f"\nAccuracy at 0% noise:  {mean_accs[0]:.3f}")
```

```
print(f"Accuracy at 20% noise: {mean_accs[round(0.2 / 0.45 * 14)]:.3f}")
print(f"Accuracy at 40% noise: {mean_accs[-2]:.3f}")
```

Coût et débit d'annotation

Comprendre l'économie de l'annotation est essentiel pour la planification des projets.

Type de tâche	Débit typique	Coût par élément (spécialiste)
Classification binaire d'images	200–500/h	0,02–0,10 $
NER (texte court)	50–150 éléments/h	0,10–0,50 $
Extraction de relations	20–60 éléments/h	0,30–1,50 $
Segmentation d'images médicales	5–30 éléments/h	10–100 $
Annotation vidéo	5–20 min vidéo/h	20–200 $

Ces chiffres sont des estimations approximatives d'ordre de grandeur et varient considérablement selon l'expertise du domaine requise, la qualité de l'outil d'annotation, la clarté des consignes et l'expérience de l'annotateur. Ils sont à titre indicatif, non prescriptif.

> **See also:** Les options d'outils d'annotation sont couvertes au Chapitre 12. Les modèles statistiques pour l'annotation en production participative (Dawid-Skene, MACE) sont traités au Chapitre 13.

Apprentissage actif

Les données étiquetées sont coûteuses. L'idée centrale de l'apprentissage actif est que *tous les exemples non étiquetés ne sont pas également informatifs* — un modèle peut progresser plus rapidement s'il peut choisir quels exemples soumettre à l'annotation. Plutôt que d'étiqueter des données de manière aléatoire, un système d'apprentissage actif interroge un oracle (généralement un annotateur humain) sur les exemples les plus susceptibles d'améliorer le modèle.

Ce chapitre couvre la théorie et la pratique de l'apprentissage actif : les stratégies de requête, les cadres d'échantillonnage, les critères d'arrêt et les considérations pratiques pour les déploiements réels.

Le cadre de l'apprentissage actif

Le cadre standard de **l'apprentissage actif basé sur un pool** implique :

- Un **ensemble étiqueté** $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^n$ — initialement petit
- Un **pool non étiqueté** $\mathcal{U} = \{x_j\}_{j=1}^m$ — généralement bien plus grand que $\mathcal{L}$
- Un **oracle** $\mathcal{O}$ qui peut retourner $y = \mathcal{O}(x)$ pour tout x interrogé
- Une **stratégie de requête** ϕ qui sélectionne la prochaine requête $x^* = \phi(\mathcal{L}, \mathcal{U}, f_\theta)$

La boucle d'apprentissage actif :

```
1. Initialiser : L = petit ensemble étiqueté d'amorçage, U = pool non étiqueté
2. Entraîner : f_θ ← entraîner(L)
3. Requêter : x∗ = argmax φ(x; f_θ) sur x ∈ U
4. Étiqueter : y∗ = O(x∗)
5. Mettre à jour : L ← L ∪ {(x∗, y∗)}, U ← U \ {x∗}
→ Répéter depuis 2 jusqu'à épuisement du budget
```

L'objectif est d'atteindre une qualité cible du modèle en utilisant le moins de requêtes d'oracle possible.

Fondements théoriques

Une question naturelle est : dans quelle mesure l'apprentissage actif peut-il aider ? Dans le meilleur des cas, l'apprentissage actif peut permettre des réductions *exponentielles* de la complexité en termes d'étiquettes — atteindre une erreur ϵ avec $O(\log(1/\epsilon))$ étiquettes plutôt que les $O(1/\epsilon)$ nécessaires à l'apprentissage passif, du moins dans des cadres réalisables avec une bonne stratégie de requête {cite}`settles2009active`.

En pratique, les garanties sont plus difficiles à obtenir. **L'apprentissage actif agnostique** {cite}`balcan2006agnostic` montre que des économies d'étiquettes sont possibles même quand le concept cible n'est pas dans la classe d'hypothèses, mais ces économies dépendent fortement du coefficient de désaccord — une mesure de la vitesse à laquelle l'ensemble des hypothèses plausibles se réduit à mesure que les données s'accumulent.

L'implication pratique essentielle : l'avantage de l'apprentissage actif est le plus grand quand la **frontière de décision est simple et concentrée** (afin que les requêtes d'incertitude éliminent rapidement les hypothèses erronées), et le plus faible quand la classe d'hypothèses est large ou la frontière complexe.

Stratégies de requête

Échantillonnage par incertitude

La stratégie la plus simple et la plus utilisée : interroger l'exemple sur lequel le modèle est *le plus incertain* {cite}`lewis1994sequential`.

La moindre confiance interroge l'exemple pour lequel le modèle est le moins confiant dans sa prédiction principale :

$$x^* = \arg\max_{x \in \mathcal{U}} \left(1 - P_\theta(\hat{y} \mid x)\right)$$

L'échantillonnage par marge considère l'écart entre les deux probabilités prédites les plus élevées :

$$x^* = \arg\min_{x \in \mathcal{U}} \left(P_\theta(\hat{y}_1 \mid x) - P_\theta(\hat{y}_2 \mid x)\right)$$

L'échantillonnage par entropie utilise la distribution prédite complète :

$$x^* = \arg\max_{x \in \mathcal{U}} \left(- \sum_{k=1}^{K} P_\theta(y_k \mid x) \log P_\theta(y_k \mid x) \right)$$

L'échantillonnage par entropie est le plus fondé théoriquement des trois — il prend en compte toutes les classes — et surpasse généralement les autres sur les problèmes multiclasses.

Requête par comité (QbC)

Entraîner un **comité** de C modèles (par ensachage, différentes initialisations ou différentes architectures). Interroger l'exemple sur lequel le comité est le plus en désaccord :

$$x^* = \arg\max_{x \in \mathcal{U}} \ \text{désaccord}(\{f_c(x)\}_{c=1}^{C})$$

Le désaccord peut être mesuré comme **l'entropie de vote** (entropie sur les votes majoritaires du comité) ou la **divergence KL** par rapport à la distribution de consensus.

La QbC fournit de meilleures estimations d'incertitude qu'un seul modèle, mais nécessite l'entraînement de plusieurs modèles, ce qui est coûteux en calcul.

Changement de modèle attendu

Interroger l'exemple qui provoquerait le plus grand changement dans le modèle actuel s'il était étiqueté. Pour les modèles à base de gradients, cela correspond à l'exemple avec la plus grande magnitude de gradient attendue {cite}`settles2008analysis` :

$$x^* = \arg\max_{x \in \mathcal{U}} \sum_{y \in \mathcal{Y}} P_\theta(y \mid x) \, \|\nabla_\theta \mathcal{L}(f_\theta(x), y)\|$$

Cette stratégie a une forte motivation théorique, mais nécessite de calculer les gradients pour chaque candidat, ce qui la rend coûteuse pour les grands modèles.

Approches géométriques / par ensemble noyau

Les stratégies basées sur l'incertitude peuvent être **biaisées vers les valeurs aberrantes** : un exemple inhabituel peut être très incertain sans être représentatif de la distribution des données. Les méthodes par ensemble noyau abordent ce problème en cherchant un échantillon diversifié qui couvre l'espace des caractéristiques.

L'algorithme **k-center glouton** {cite}`sener2018active` trouve le plus petit ensemble de points tel que chaque point non étiqueté soit à moins de δ d'au moins un point interrogé :

$$x^* = \arg\max_{x \in \mathcal{U}} \min_{x' \in \mathcal{L}} d(x, x')$$

c'est-à-dire, interroger le point le plus éloigné de tout point déjà étiqueté. Cela encourage un ensemble d'annotations bien réparti.

BADGE

L'apprentissage actif par lots via des plongements de gradients divers (BADGE) {cite}`ash2020deep` combine incertitude et diversité : il sélectionne un lot d'exemples dont les plongements de gradients (par rapport à l'étiquette prédite) sont à la fois de grande magnitude (incertains) et divers (couvrant différentes régions de l'espace des gradients). C'est l'une des stratégies modernes les plus compétitives.

Estimation d'incertitude pour les modèles profonds

Les stratégies ci-dessus supposent l'accès à des sorties de probabilité calibrées du modèle. Pour les modèles simples (régression logistique, classificateurs softmax), cela est direct. Pour les réseaux profonds, obtenir des estimations d'incertitude fiables nécessite des techniques supplémentaires.

Deux types d'incertitude

Suivant Kendall et Gal {cite}`kendall2017uncertainties`, nous distinguons :

L'incertitude aléatique (incertitude des données) : bruit inhérent aux observations qui ne peut pas être réduit en collectant davantage de données. Une image floue est aléatoirement incertaine — aucune quantité de données d'entraînement supplémentaires issues de la même distribution ne rendra le modèle plus confiant à son sujet.

L'incertitude épistémique (incertitude du modèle) : incertitude due à des données d'entraînement limitées ou à un modèle qui n'a pas vu d'exemples similaires. L'incertitude épistémique *peut* être réduite en étiquetant davantage de données — et est donc la quantité pertinente pour la sélection des requêtes en apprentissage actif.

Pour l'apprentissage actif, nous voulons interroger les exemples à forte incertitude épistémique, pas à forte incertitude aléatique. Interroger un exemple fondamentalement ambigu gaspille l'effort de l'oracle : aucune étiquette qu'il fournit ne sera clairement correcte.

Monte Carlo Dropout

Une approche pratique pour l'estimation de l'incertitude épistémique dans les réseaux de neurones est le **MC Dropout** {cite}`gal2016dropout` : appliquer le dropout lors de l'inférence et effectuer T passes avant. La variance entre les prédictions est une estimation de l'incertitude épistémique.

```
import numpy as np
import torch
import torch.nn as nn

torch.manual_seed(42)
rng = np.random.default_rng(42)

class MCDropoutNet(nn.Module):
    def __init__(self, input_dim=20, hidden=64, output_dim=2, p_drop=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, hidden),    nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, output_dim)
        )

    def forward(self, x):
        return self.net(x)

def mc_uncertainty(model, x, T=30):
    """
    Run T stochastic forward passes with dropout active.
    Returns mean prediction and epistemic uncertainty (predictive variance).
    """
    model.train()  # keep dropout active during inference
    with torch.no_grad():
        preds = torch.stack([
            torch.softmax(model(x), dim=-1) for _ in range(T)
        ])  # shape: (T, N, C)
    mean_pred = preds.mean(0)
    # Epistemic uncertainty: mean of variances across passes
    epistemic = preds.var(0).sum(-1)
    # Aleatoric uncertainty: entropy of mean prediction
    aleatoric = -(mean_pred * (mean_pred + 1e-9).log()).sum(-1)
    return mean_pred, epistemic, aleatoric

# Quick demonstration
```

```
model = MCDropoutNet(input_dim=20, output_dim=2)
# In-distribution example (simulated)
x_familiar   = torch.randn(1, 20) * 0.5
# Out-of-distribution example (far from training distribution)
x_unfamiliar = torch.randn(1, 20) * 3.0

for name, x in [("In-distribution ", x_familiar), ("Out-of-distribution", x_unfamiliar)]:
    _, ep, al = mc_uncertainty(model, x)
    print(f"{name} | epistemic: {ep.item():.4f} | aleatoric: {al.item():.4f}")
```

Dans le réseau non entraîné ci-dessus, les deux exemples présentent une incertitude similaire. Après entraînement, l'exemple hors distribution présentera une incertitude épistémique plus élevée — le modèle n'a pas appris de correspondance fiable pour les entrées éloignées de la distribution d'entraînement.

Ensembles profonds

Entraîner M modèles initialisés de manière indépendante et faire la moyenne de leurs prédictions fournit une estimation d'incertitude plus simple et souvent plus fiable que le MC Dropout {cite}`lakshminarayanan2017simple`. Le désaccord entre les membres de l'ensemble est le signal d'incertitude épistémique.

Pour l'apprentissage actif à grande échelle, le MC Dropout et les ensembles profonds ajoutent une surcharge proportionnelle à T ou M passes avant. En pratique, $T = 10$–30 pour le MC Dropout ou $M = 5$ membres d'ensemble est souvent suffisant pour classer les exemples par incertitude épistémique, même si les valeurs absolues ne sont pas bien calibrées.

Une boucle d'apprentissage actif complète

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from copy import deepcopy

rng = np.random.default_rng(42)

# Generate dataset
X, y = make_classification(
    n_samples=2000, n_features=20, n_informative=10,
    n_classes=3, n_clusters_per_class=1, random_state=42
)
X_train, y_train = X[:1500], y[:1500]
X_test,  y_test  = X[1500:], y[1500:]

def entropy_query(model, X_pool):
    """Return index of most uncertain sample (entropy)."""
    probs = model.predict_proba(X_pool)
    ent = -np.sum(probs * np.log(probs + 1e-9), axis=1)
    return np.argmax(ent)
```

```
def random_query(X_pool):
    """Random baseline."""
    return rng.integers(0, len(X_pool))

def run_active_learning(strategy='entropy', n_initial=30, n_queries=120, query_batch=5):
    labeled_idx = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled_idx = [i for i in range(len(X_train)) if i not in labeled_idx]
    accs = []

    for step in range(n_queries // query_batch):
        model = LogisticRegression(max_iter=500, C=1.0)
        model.fit(X_train[labeled_idx], y_train[labeled_idx])
        accs.append(accuracy_score(y_test, model.predict(X_test)))

        # Query
        X_pool = X_train[unlabeled_idx]
        for _ in range(query_batch):
            if strategy == 'entropy':
                q = entropy_query(model, X_pool)
            else:
                q = random_query(X_pool)
            labeled_idx.append(unlabeled_idx.pop(q))
            X_pool = X_train[unlabeled_idx]

    return np.array(accs)

labels_used = np.arange(1, 25) * 5 + 30  # label counts at each step

accs_active = run_active_learning(strategy='entropy')
accs_random = run_active_learning(strategy='random')

plt.figure(figsize=(8, 5))
plt.plot(labels_used, accs_active, 'o-', label='Entropy sampling', color='#2b3a8f', linewidth
plt.plot(labels_used, accs_random, 's--', label='Random baseline',  color='#e05c5c', linewidt
plt.xlabel("Number of labeled examples", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning vs. Random Sampling", fontsize=13)
plt.legend(fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('active_learning_curve.png', dpi=150)
plt.show()

print(f"Active learning reaches {accs_active[-1]:.3f} accuracy")
print(f"Random sampling reaches {accs_random[-1]:.3f} accuracy")
print(f"Active learning saves ~{int((accs_random.tolist().index(min(accs_random, key=lambda a
```

Le problème du démarrage à froid

L'apprentissage actif nécessite un modèle entraîné pour évaluer les points non étiquetés — mais au début, vous n'avez pas (ou très peu) d'exemples étiquetés. C'est le **problème du démarrage à froid**.

Solutions pratiques :

1. **Initialisation aléatoire :** Étiqueter un petit ensemble d'amorçage aléatoire (20–100 exemples) avant de démarrer l'apprentissage actif.
2. **Initialisation par regroupement :** Utiliser k-means sur le pool non étiqueté ; étiqueter un exemple de chaque groupe. Cela garantit la diversité de l'ensemble étiqueté initial.
3. **Sélection basée sur les plongements :** Utiliser un encodeur pré-entraîné pour plonger les exemples ; sélectionner un sous-ensemble diversifié par ensemble noyau.

Pour la plupart des tâches, quelques dizaines d'étiquettes d'amorçage aléatoires sont généralement suffisantes pour démarrer l'apprentissage actif ; le nombre exact dépend de l'équilibre des classes, de la dimensionnalité des caractéristiques et de la complexité du modèle.

Apprentissage actif par lots

En pratique, les annotateurs travaillent par lots — il est inefficace d'entraîner et de déployer un nouveau modèle après chaque étiquette individuelle. **L'apprentissage actif par lots** sélectionne un ensemble de b exemples à étiqueter simultanément.

Sélectionner naïvement les b exemples les plus incertains conduit à de la **redondance** : les exemples très incertains ont tendance à se regrouper (par exemple, des exemples proches de la frontière de décision dans la même région). De meilleures stratégies par lots optimisent à la fois l'incertitude *et* la diversité au sein du lot.

Les **Processus Ponctuels Déterminantiels (DPP)** fournissent un moyen fondé d'échantillonner des lots diversifiés : ils définissent une distribution sur les sous-ensembles qui pénalise les éléments similaires. La probabilité d'un sous-ensemble S sous un DPP est proportionnelle à $\det(L_S)$ où L est une matrice noyau encodant la similarité.

Critères d'arrêt

Quand l'apprentissage actif devrait-il s'arrêter ? Critères courants :

- **Budget épuisé :** Le plus simple — s'arrêter quand le budget d'annotation est écoulé.
- **Plateau de performance :** S'arrêter quand la précision du modèle sur un ensemble de validation mis de côté ne s'est pas améliorée de plus de δ pendant k tours consécutifs.
- **Seuil de confiance :** S'arrêter quand moins d'une certaine fraction des exemples non étiquetés ont une incertitude au-dessus d'un seuil.
- **Réduction maximale de la perte :** Estimer le gain maximal possible à partir d'étiquettes supplémentaires ; s'arrêter quand il tombe en dessous d'un seuil {cite}`bloodgood2009method`.

Quand l'apprentissage actif fonctionne (et quand il ne fonctionne pas)

L'apprentissage actif tend à bien fonctionner quand : - L'étiquetage est coûteux et le pool non étiqueté est grand - Les données ont une structure claire que le modèle peut exploiter pour identifier des exemples informatifs - La classe de modèles est appropriée pour la tâche

L'apprentissage actif performe mal quand : - Le modèle initial est très mauvais (démarrage à froid) et ne peut pas classer significativement les exemples - La stratégie de requête sélectionne des valeurs aberrantes ou des exemples mal étiquetés (la robustesse au bruit est importante) - La distribution des données change entre le pool non étiqueté et la distribution de test

Une préoccupation pratique essentielle est le **décalage de distribution** : l'apprentissage actif tend à interroger les exemples proches de la frontière de décision, créant un ensemble étiqueté biaisé qui peut ne pas bien représenter la distribution de test. Cela peut conduire à des frontières de décision bien entraînées mais à une mauvaise calibration.

> **See also:** La revue de référence est {cite}`settles2009active`. Fondements théoriques (complexité en étiquettes, bornes agnostiques) : {cite}`balcan2006agnostic`. Pour l'apprentissage actif spécifique à l'apprentissage profond, voir {cite}`ash2020deep` (BADGE) et {cite}`sener2018active` (ensemble noyau). Pour une évaluation critique de quand l'apprentissage actif aide réellement, voir {cite}`lowell2019practical`. Sur l'incertitude aléatique vs. épistémique pour les modèles profonds, voir {cite}`kendall2017uncertainties` ; pour les ensembles profonds comme estimateurs d'incertitude, voir {cite}`lakshminarayanan2017simple` ; pour le MC Dropout comme inférence bayésienne approchée, voir {cite}`gal2016dropout`.

Apprentissage automatique interactif

L'apprentissage actif pose une question ciblée : étant donné un budget, quels exemples étiqueter ? L'apprentissage automatique interactif (IML) pose une question plus large : comment concevoir *l'ensemble de l'interaction* entre un humain et un système d'apprentissage pour qu'elle soit au maximum productive, agréable et correcte ?

L'IML se distingue par **l'immédiateté** et la **directité** de la boucle de retour humain–modèle. Là où l'apprentissage automatique traditionnel implique qu'un humain transmette des données et attende la fin de l'entraînement, l'IML permet aux humains d'observer le comportement du modèle, de fournir un retour et de voir le modèle répondre — souvent en quelques secondes.

Principes de l'apprentissage automatique interactif

Amershi et al. {cite}`amershi2014power` identifient trois caractéristiques définissant l'IML :

1. Retour rapide : Le modèle se met à jour assez rapidement pour que les humains perçoivent l'effet de leur retour. Dans la limite, les mises à jour du modèle se produisent en temps réel.

2. Manipulation directe : L'humain interagit avec le modèle via les données ou via les prédictions du modèle — non via des fichiers de configuration ou du réglage d'hyperparamètres.

3. Raffinement itératif : Le processus est véritablement itératif : l'action suivante de l'humain est informée par le comportement actuel du modèle, qui a été façonné par les actions précédentes de l'humain.

Cela crée une **boucle de co-adaptation** étroite : tant l'humain que le modèle changent dans le temps en réponse l'un à l'autre. L'humain apprend ce que le modèle comprend ; le modèle apprend ce qui importe à l'humain.

Comparaison avec l'apprentissage actif

L'IML et l'apprentissage actif se recoupent considérablement mais ne sont pas identiques :

Propriété	Apprentissage actif	Apprentissage automatique interactif
Question principale	Quoi étiqueter ?	Comment interagir ?
Latence du retour	Peut être par lots (jours)	Typiquement temps réel ou quasi
Fréquence de mise à jour	Par tour (lots)	Par interaction (en ligne)
Initiative humaine	Répond aux questions du modèle	Peut proactivement enseigner au modèle

Propriété	Apprentissage actif	Apprentissage automatique interactif
Conception d'interface	Préoccupation secondaire	Préoccupation centrale
Charge cognitive de l'humain	Non explicitement modélisée	Explicitement considérée

En apprentissage actif, c'est la machine qui dirige l'interaction. En IML, l'humain peut aussi prendre l'initiative — en fournissant des exemples, des corrections ou un retour sur l'aspect du comportement du modèle qui semble le plus problématique.

Interaction à initiative mixte

Les systèmes à **initiative mixte** permettent à la fois à l'humain et à la machine de prendre les devants à différents moments {cite}`allen1999mixed`. Un système purement dirigé par la machine pose des questions et l'humain répond. Un système purement dirigé par l'humain laisse l'humain décider de tout. Les systèmes à initiative mixte équilibrent les deux.

En pratique, les meilleurs systèmes IML combinent : - **Initiative de la machine :** « Je ne suis pas sûr de ces exemples — pouvez-vous les étiqueter ? » - **Initiative humaine :** « Je remarque que le modèle se trompe constamment sur cette catégorie — laissez-moi lui fournir plus d'exemples » - **Confirmation :** Le modèle fait ressortir sa compréhension actuelle ; l'humain confirme ou corrige

Les bonnes interfaces IML rendent la compréhension actuelle du modèle visible et corrigeable. C'est l'exigence **d'intelligibilité** : les humains ne peuvent guider qu'un modèle qu'ils comprennent, au moins approximativement.

Facteurs humains dans l'IML

L'IML introduit des facteurs humains — charge cognitive, fatigue, cohérence et confiance — directement dans la boucle d'apprentissage. Une mauvaise conception IML conduit à :

La fatigue d'annotation : Les humains prennent des décisions plus rapides et moins soignées à mesure que les sessions s'allongent. Les erreurs entrent dans les données d'entraînement.

Le biais d'ancrage : Les humains qui s'appuient excessivement sur les suggestions actuelles du modèle. Si une interface pré-remplit la prédiction du modèle, les annotateurs sont moins susceptibles de la corriger même quand elle est erronée — une source systématique de bruit d'étiquetage qui se cumule sur les tours d'annotation {cite}`geva2019annotator`. La pré-annotation peut accélérer le débit {cite}`lingren2014evaluating` tout en réduisant simultanément le taux auquel les annotateurs détectent les erreurs du modèle ; ces deux effets doivent être mis en balance dans la conception d'interfaces IML.

La mauvaise calibration de la confiance : Les humains font soit une confiance excessive (acceptant des sorties erronées du modèle) soit une méfiance excessive (ignorant des suggestions correctes). Les deux schémas réduisent la valeur de la collaboration humain–modèle.

La cohérence de session : Les humains peuvent prendre des décisions différentes sur le même exemple à des moments différents, surtout après de longues sessions. Des vérifications de cohérence (re-présentation d'exemples antérieurs) peuvent détecter et corriger ce phénomène.

Une bonne conception IML atténue ces problèmes par des choix d'interface : présenter explicitement la confiance du modèle, randomiser l'ordre d'affichage, limiter la durée des sessions et intégrer des vérifications de cohérence.

Les types de retour IML en pratique

Retour au niveau de l'exemple

L'humain fournit une étiquette ou une correction pour un exemple spécifique. C'est la forme la plus courante et directement compatible avec l'apprentissage supervisé.

Retour au niveau des caractéristiques

L'humain indique quelles caractéristiques sont pertinentes ou non pertinentes. « Le modèle devrait faire attention aux mots "urgent" et "délai" pour cette catégorie. » C'est plus expressif que le retour au niveau de l'exemple et peut être plus efficace pour certaines tâches.

TFIDF Interactive et systèmes similaires permettent aux annotateurs de surligner des mots pertinents dans des documents textuels. Ces surlignages sont convertis en contraintes ou en supervision supplémentaire sur l'attention du modèle.

Retour au niveau du modèle

L'humain corrige directement le comportement du modèle sur une classe d'entrées : « Chaque fois que l'entrée contient [X], la sortie devrait être [Y]. » Cela correspond à des règles logiques ou des contraintes dans des approches comme la Régularisation Postérieure {cite}`ganchev2010posterior` ou l'apprentissage guidé par contraintes.

Étude de cas : Google Teachable Machine

Teachable Machine est un système IML accessible basé sur le web qui permet à des utilisateurs non techniques d'entraîner des classificateurs d'images dans le navigateur. L'utilisateur :

1. Enregistre des exemples de chaque classe via sa webcam
2. Entraîne le modèle d'un seul clic (affinement d'un MobileNet dans le navigateur)
3. Observe immédiatement les prédictions du modèle sur la vidéo en direct
4. Ajoute des exemples pour les classes où le modèle se trompe

Cela illustre la boucle IML fondamentale : fournir des exemples □ observer le modèle □ identifier l'échec □ fournir des exemples plus ciblés. Le retour en temps réel (les sorties du modèle se mettent à jour en temps réel, typiquement à des fréquences d'images interactives sur le matériel moderne) rend la boucle de co-adaptation viscéralement immédiate.

Implémentation d'une boucle IML simple

```python
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

class SimpleIMLSystem:
    """
    Minimal IML system that allows online feedback and displays
    model state after each annotation.
    """

    def __init__(self, n_features=10, n_classes=2):
        self.model = SGDClassifier(loss='log_loss', max_iter=1, warm_start=True,
                                   random_state=42)
        self.scaler = StandardScaler()
        self.X_seen = []
        self.y_seen = []
        self.n_classes = n_classes
        self.initialized = False

    def update(self, x, y_true):
        """Receive a single labeled example and update the model."""
        self.X_seen.append(x)
        self.y_seen.append(y_true)

        if len(self.X_seen) >= 2 * self.n_classes:
            X_arr = np.array(self.X_seen)
            y_arr = np.array(self.y_seen)
            X_scaled = self.scaler.fit_transform(X_arr)
            self.model.partial_fit(X_scaled[-1:], y_arr[-1:],
                                   classes=list(range(self.n_classes)))
            self.initialized = True

        return self

    def predict_with_confidence(self, x):
        """Predict label and return confidence."""
        if not self.initialized:
            return None, 0.0
        x_scaled = self.scaler.transform(x.reshape(1, -1))
        probs = self.model.predict_proba(x_scaled)[0]
        return self.model.predict(x_scaled)[0], probs.max()

    def current_accuracy(self, X_val, y_val):
        if not self.initialized:
            return None
        X_scaled = self.scaler.transform(X_val)
        return (self.model.predict(X_scaled) == y_val).mean()
```

```
# Simulate an IML session
rng = np.random.default_rng(42)
X_all, y_all = np.random.default_rng(0).random((500, 10)), np.random.default_rng(0).integers(
X_val, y_val = X_all[400:], y_all[400:]

system = SimpleIMLSystem(n_features=10, n_classes=2)
accs, confidences = [], []

print("Step | Labels | Accuracy | Example confidence")
print("-" * 50)

for step in range(100):
    x, y = X_all[step], y_all[step]
    pred, conf = system.predict_with_confidence(x)
    system.update(x, y)

    if (step + 1) % 10 == 0:
        acc = system.current_accuracy(X_val, y_val)
        if acc is not None:
            accs.append(acc)
            print(f"  {step+1:3d} |  {step+1:4d}  |  {acc:.3f}   | {conf:.3f}")
```

Le test de la grand-mère

L'une des heuristiques utiles pour évaluer la conception d'interfaces IML — et plus généralement la conception de systèmes HITL — est ce que nous appellerons le **Test de la grand-mère** (une formulation originale introduite ici comme contrainte de conception, non comme référence à des travaux antérieurs) :

> *Une femme née en 1930 devrait pouvoir utiliser ce dispositif par la voix, et en cas de frustration, se replier gracieusement sur une interface clavier ou textuelle.*

Le test ne porte pas principalement sur l'accessibilité, bien qu'il le soit aussi. Il porte sur la **conception pour la friction**. Si un système exige un modèle mental des réseaux de neurones, des boucles d'entraînement ou des distributions de probabilité pour être utilisé efficacement, il a échoué au Test de la grand-mère. L'humain dans la boucle devrait pouvoir participer sans comprendre le côté machine de la boucle.

Les implications pour la conception IML sont concrètes :

Repli vers la voix : La modalité d'interaction principale devrait être le langage naturel ou le geste — non des curseurs de paramètres ou des seuils de confiance. Les experts peuvent vouloir des curseurs ; tout le monde devrait pouvoir dire « c'est faux ».

Dégradation gracieuse : Quand la modalité préférée de l'utilisateur échoue ou frustre, le système devrait proposer une alternative — non un écran blanc ou un message d'erreur. L'interface fait partie du système d'apprentissage ; un utilisateur qui ne peut pas interagir ne peut pas enseigner.

État du modèle lisible : La compréhension actuelle du modèle devrait être visible en termes

humains. Non pas « confiance : 0,73 » mais « Je suis assez sûr que c'est [X], mais j'ai vu des exemples comme celui-ci aller dans les deux sens. » L'incertitude devrait être communiquée dans un langage qui invite à la correction.

Tolérance à l'ambiguïté : Un utilisateur de 93 ans et un ingénieur en apprentissage automatique de 23 ans interagiront différemment avec le même système. Le Test de la grand-mère demande si le système peut accommoder les deux — non en détectant l'âge de l'utilisateur, mais en concevant des interactions qui fonctionnent pour une gamme d'expertises et d'aisances.

Le test prend une importance particulière à mesure que les systèmes d'apprentissage automatique passent d'outils de recherche à une infrastructure quotidienne. Un assistant d'imagerie médicale utilisé par des radiologues, un classificateur de documents juridiques utilisé par des greffiers, un système de retour éducatif utilisé par des enseignants — chacun implique des humains dans la boucle qui ne se sont pas portés volontaires pour être des entraîneurs d'IA. Concevoir pour eux n'est pas une concession ; c'est le propos.

:::{admonition} Principe de conception :class: tip Le Test de la grand-mère est une contrainte de conception, non une démographie cible. Les systèmes qui le réussissent sont plus robustes à la diversité des utilisateurs, plus indulgents face aux écarts d'expertise, et plus honnêtes sur ce qu'ils attendent des humains dans la boucle. Si un système nécessite une explication avant utilisation, il demande à l'humain de faire un travail supplémentaire. Ce travail devrait être justifié par un bénéfice proportionné. :::

IML et modèles de fondation

L'IML moderne s'appuie de plus en plus sur des **modèles de fondation pré-entraînés** {cite}`bommasani2021opportunities` comme base. Au lieu de partir de zéro, les utilisateurs affinent un grand modèle pré-entraîné avec un petit nombre d'exemples interactifs. Cela peut réduire considérablement le nombre d'exemples nécessaires pour atteindre des performances utiles — dans des cas favorables, aussi peu que 5 à 50 exemples plutôt que des milliers, selon la qualité de correspondance entre les représentations pré-entraînées et la tâche cible {cite}`bommasani2021opportunities`.

Les techniques permettant cela incluent : - **Le prompting en quelques exemples :** Fournir des exemples dans la fenêtre contextuelle du grand modèle de langage - **L'affinement par adaptateurs :** Mettre à jour de petits modules adaptateurs tout en gelant le modèle de base - **L'affinement efficace en paramètres (PEFT) :** LoRA, ajustement de préfixe et méthodes similaires permettant des mises à jour rapides et peu gourmandes en ressources

Les modèles de fondation modifient la dynamique IML : les humains n'enseignent plus à un modèle vierge depuis zéro, mais *orientent* un modèle qui sait déjà beaucoup. Le défi passe de « comment fournir suffisamment d'exemples » à « comment spécifier exactement ce que nous voulons de différent par rapport à ce que le modèle fait déjà ».

> **See also:** La revue de {cite}`amershi2014power` reste le meilleur aperçu des principes IML. Pour les systèmes à initiative mixte spécifiquement, voir {cite}`allen1999mixed`. Pour les effets d'ancrage dans l'annotation, voir {cite}`geva2019annotator` et {cite}`lingren2014evaluating`.

Apprentissage par renforcement à partir des retours humains

Aucune technique n'a fait davantage pour faire entrer le HITL ML dans le grand public que l'apprentissage par renforcement à partir des retours humains (RLHF). C'est le mécanisme derrière InstructGPT {cite}`ouyang2022training` et un composant central des pipelines de suivi d'instructions dans de nombreux grands modèles de langage modernes {cite}`stiennon2020learning`. Comprendre le RLHF — non seulement comme une recette à suivre, mais comme une approche principiée de l'alignement — est essentiel pour quiconque travaille dans l'IA moderne.

Le problème de l'alignement

Les grands modèles de langage (LLM) entraînés uniquement sur la prédiction du token suivant optimisent un objectif de substitution : prédire quel texte vient ensuite dans un corpus de textes rédigés par des humains. Cet objectif est lié à, mais distinct de, ce que nous voulons réellement : des réponses utiles, précises, sûres et alignées sur les valeurs humaines.

Le décalage entre l'objectif d'entraînement et le comportement souhaité est appelé le **problème de l'alignement** {cite}`russell2019human`. Concrètement, un modèle de langage entraîné sur du texte d'internet apprend à : - Produire des continuations plausibles (qui peuvent être factuellement erronées) - Refléter les biais et les préjudices présents dans les données d'entraînement - Être évasif ou manipulateur quand c'est ce qui suit statistiquement le prompt

Le RLHF aborde l'alignement en rendant les préférences humaines *partie intégrante de l'objectif d'optimisation*.

Le pipeline RLHF

Le RLHF se déroule en trois étapes :

```
Étape 1 : Affinement supervisé (SFT)
  --> Collecter des données de démonstration (l'humain rédige les réponses idéales)
  --> Affiner le LLM de base sur les démonstrations

Étape 2 : Entraînement du modèle de récompense
  --> Collecter des préférences par paires (l'humain évalue A vs B)
  --> Entraîner le modèle de récompense R(x, y) à prédire les préférences humaines
```

```
Étape 3 : Affinement par RL
  --> Affiner le LLM avec PPO/RL pour maximiser R(x, y)
  --> La pénalité KL empêche un trop grand écart par rapport au modèle SFT
```

Étape 1 : Affinement supervisé

En partant d'un modèle de base pré-entraîné π_0, nous collectons un ensemble de données de paires (prompt, réponse idéale), rédigées ou sélectionnées par des contractants humains suivant des consignes détaillées. Le modèle est affiné sur ces démonstrations en utilisant l'entropie croisée standard :

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{demo}}}\left[\log \pi_\theta(y \mid x)\right]$$

Le modèle SFT π_{SFT} est un bien meilleur point de départ pour le RLHF que le modèle pré-entraîné brut.

Étape 2 : Entraînement du modèle de récompense

Pour un ensemble de prompts $\{x_i\}$, nous générons K réponses par prompt en utilisant π_{SFT} et les présentons à des évaluateurs humains sous forme de comparaisons par paires : « Quelle réponse est meilleure, A ou B ? »

Le modèle de récompense r_ϕ est entraîné à prédire ces préférences. Sous le **modèle de Bradley-Terry** (Chapitre 8), la probabilité que la réponse y_w soit préférée à y_l est :

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

Le modèle de récompense est entraîné à minimiser la perte de classement par paires :

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

Le modèle de récompense est généralement initialisé à partir du modèle SFT avec une tête scalaire remplaçant la couche finale.

Étape 3 : Affinement par RL avec PPO

Avec un modèle de récompense entraîné, nous pouvons utiliser l'apprentissage par renforcement pour affiner le LLM. Chaque prompt x est un état ; chaque réponse y est une trajectoire de choix de tokens ; et la récompense est $r_\phi(x, y)$.

L'objectif d'optimisation inclut une **pénalité de divergence KL** pour empêcher le modèle de dériver trop loin de la référence SFT (ce qui conduirait au détournement de récompense {cite}`krakovna2020specification,gao2023scaling`) :

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta \cdot \text{KL}\left[\pi_\theta(\cdot \mid x) \| \pi_{\text{SFT}}(\cdot \mid x)\right]\right]$$

Le paramètre β contrôle la force de la pénalité KL. Un petit β permet plus d'optimisation mais risque le détournement de récompense ; un grand β maintient le modèle proche du SFT mais limite les gains d'alignement.

L'Optimisation par Politique Proximale (PPO) {cite}`schulman2017proximal` est l'algorithme standard pour cette étape, choisi pour sa stabilité par rapport aux méthodes de gradient de politique brutes.

Une démonstration simplifiée du RLHF

Le pipeline RLHF complet nécessite une infrastructure à grande échelle. L'exemple suivant démontre les idées clés à petite échelle.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn import functional as F

torch.manual_seed(42)
rng = np.random.default_rng(42)

# ----------------------------------------------
# Toy setup: responses are 4-dimensional vectors
# "Quality" is known analytically (sum of positive values)
# We simulate a reward model learning this from pairwise feedback
# ----------------------------------------------

class RewardModel(nn.Module):
    def __init__(self, d=4, hidden=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

def true_quality(x):
    """The hidden ground-truth reward function."""
    return x.sum(dim=-1) + 0.5 * (x ** 2).mean(dim=-1)

# Generate pairwise preference data
N_PAIRS = 500
X1 = torch.randn(N_PAIRS, 4)
X2 = torch.randn(N_PAIRS, 4)
q1, q2 = true_quality(X1), true_quality(X2)
# Human prefers X1 when q1 > q2 (with some noise)
noise = torch.randn(N_PAIRS) * 0.5
preferred_1 = ((q1 - q2 + noise) > 0).float()

# Train reward model
rm = RewardModel(d=4, hidden=32)
```

```
optimizer = optim.Adam(rm.parameters(), lr=3e-3)

losses = []
for epoch in range(200):
    r1 = rm(X1)
    r2 = rm(X2)
    # Bradley-Terry loss
    logit = r1 - r2
    loss = F.binary_cross_entropy_with_logits(logit, preferred_1)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
    losses.append(loss.item())

# Evaluate: does the reward model agree with true quality?
X_eval = torch.randn(1000, 4)
with torch.no_grad():
    r_pred = rm(X_eval).numpy()
    r_true = true_quality(X_eval).numpy()

corr = np.corrcoef(r_pred, r_true)[0, 1]
print(f"Reward model correlation with true quality: {corr:.4f}")
print(f"Final training loss: {losses[-1]:.4f}")

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1)
plt.plot(losses, color='#2b3a8f', linewidth=1.5)
plt.xlabel("Epoch"); plt.ylabel("Pairwise loss")
plt.title("Reward Model Training")

plt.subplot(1, 2, 2)
plt.scatter(r_true[:200], r_pred[:200], alpha=0.4, s=15, color='#2b3a8f')
plt.xlabel("True quality"); plt.ylabel("Predicted reward")
plt.title(f"Reward Model vs. Truth (r={corr:.3f})")
plt.tight_layout()
plt.savefig('reward_model.png', dpi=150)
plt.show()
```

Défis du RLHF

Détournement de récompense

Un mode d'échec clé : la politique trouve des moyens d'obtenir une récompense élevée du modèle de récompense sans que cela corresponde à un comportement genuinement bon. Par exemple, un LLM pourrait apprendre à produire des réponses flatteuses ou formulées avec assurance (que les évaluateurs ont tendance à bien noter) plutôt que précises.

Le détournement de récompense est plus probable quand : - Le modèle de récompense est entraîné sur un nombre insuffisant de données de préférences - La politique est autorisée à s'éloigner fortement de la référence SFT (petit β) - La distribution du modèle de récompense dérive pendant l'entraînement PPO

Stratégies d'atténuation : Pénalité KL, entraînement itératif du modèle de récompense, évaluation diversifiée, contraintes de l'IA constitutionnelle.

Biais des évaluateurs

Les évaluateurs humains ont des biais systématiques. Ils tendent à préférer les réponses plus longues (biais de verbosité), les textes à la formulation plus assurée (biais de confiance), et les réponses qui concordent avec leurs croyances préalables. Ces biais se propagent dans le modèle de récompense.

Le fameux échec de servilité des modèles RLHF — où le modèle dit aux utilisateurs ce qu'ils veulent entendre plutôt que ce qui est vrai — est en partie le résultat de la préférence des évaluateurs pour les réponses agréables.

Supervision évolutive

Pour les tâches complexes, les humains ne peuvent pas juger de manière fiable quelle réponse IA est correcte. Un évaluateur comparant deux longues démonstrations mathématiques ou deux implémentations de code peut simplement choisir la plus lisible, indépendamment de l'exactitude. **La supervision évolutive** est le problème ouvert de la conception de procédures d'évaluation qui restent fiables à mesure que la complexité des tâches croît {cite}`bowman2022measuring`.

L'IA constitutionnelle (RLAIF)

L'IA constitutionnelle {cite}`bai2022constitutional`, développée chez Anthropic, réduit la dépendance aux évaluateurs humains en utilisant l'IA elle-même pour générer des étiquettes de préférences guidées par un ensemble de principes (une « constitution »). Le processus :

1. Générer des réponses à des prompts potentiellement nuisibles
2. Utiliser un critique IA pour évaluer les réponses par rapport aux principes constitutionnels
3. Réviser les réponses guidées par le retour IA (RLAIF — RL à partir du retour IA)
4. Entraîner un modèle de récompense sur les préférences générées par IA
5. Affiner avec RLHF en utilisant ce modèle de récompense

Le RLAIF peut générer des données de préférences à une échelle bien supérieure à l'étiquetage humain, et permet un contrôle précis sur les valeurs encodées dans le modèle de récompense.

> **See also:** L'article original InstructGPT {cite}`ouyang2022training` décrit la première application à grande échelle du RLHF aux LLM. Le travail fondateur sur le RLHF pour l'apprentissage par renforcement profond est {cite}`christiano2017deep`. PPO est décrit dans {cite}`schulman2017proximal`. L'IA constitutionnelle est issue de {cite}`bai2022constitutional`.

Apprentissage par démonstration

Quand une tâche est difficile à spécifier mais facile à démontrer, il peut être plus efficace d'enseigner par l'exemple que de définir par règle. Un expert humain montre à un bras robotique comment saisir un objet ; les interactions d'un programmeur avec son environnement de développement fournissent une séquence d'éditions correctes ; un grand maître d'échecs joue une partie. **L'apprentissage par démonstration** extrait une politique à partir de telles données comportementales, évitant la nécessité de fonctions de récompense artisanales ou de spécifications explicites de tâches.

Le clonage comportemental

L'approche la plus simple est le **clonage comportemental (BC)** : traiter la démonstration comme des données supervisées et apprendre une correspondance entre états et actions.

Étant donné un ensemble de données de paires état-action $\mathcal{D} = \{(s_i, a_i)\}$ provenant d'un démonstrateur expert, nous ajustons une politique $\pi_\theta(a \mid s)$ en minimisant la log-vraisemblance négative :

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s)$$

C'est précisément l'apprentissage supervisé standard appliqué à des données séquentielles.

Le problème du décalage de covariables

Le BC présente une faiblesse fondamentale : le **décalage de distribution** entre l'entraînement et le déploiement. Les démonstrations de l'expert couvrent les états visités par l'expert. Mais lors du déploiement, la politique apprise peut prendre des décisions légèrement différentes, la conduisant dans des états que l'expert n'a jamais visités — des états où la politique n'a aucune supervision et peut échouer gravement.

Crucialement, les erreurs **se cumulent** : une petite déviation conduit à un état inconnu, où une action légèrement erronée conduit à un état encore plus inconnu, et ainsi de suite. Les performances se dégradent en $O(T^2\epsilon)$ où T est la longueur de l'épisode et ϵ est le taux d'erreur à chaque étape — bien pire que la dégradation en $O(T\epsilon)$ d'une politique oracle {cite}`ross2010efficient`.

> **Exemple : Conduite autonome** :class: note
>
> Un modèle de clonage comportemental pour le maintien de voie entraîné sur des données de conduite humaine fonctionne bien sur des routes droites (états proches

de la distribution d'entraînement). Mais au moment où il dérive légèrement — un état dans lequel aucun conducteur humain ne se trouverait parce qu'il aurait déjà corrigé — il n'a pas de données pour le guider et peut sortir de la route.

```
Algorithme DAgger :
  Initialiser : D <- {} (ensemble vide)
  Entraîner la politique initiale pi_1 sur M démonstrations expert

  pour l'itération i = 1, 2, ..., N :
    1. Exécuter pi_i dans l'environnement pour collecter les états {s_1, ..., s_t}
    2. Interroger l'expert pour les actions sur chaque état visité : a_t = pi*(s_t)
    3. Agréger : D <- D u {(s_1, a_1), ..., (s_t, a_t)}
    4. Entraîner pi_{i+1} par apprentissage supervisé sur D
```

DAgger atteint un regret en $O(T\epsilon)$ — identique à une politique oracle — parce que la distribution d'entraînement converge pour correspondre à la distribution de déploiement.

L'exigence clé est que l'expert puisse être interrogé sur n'importe quel état, y compris des états que l'expert ne visiterait jamais naturellement. C'est faisable en simulation (demander à l'expert de corriger le robot depuis une configuration inhabituelle) mais peut être difficile ou dangereux dans des systèmes physiques réels.

L'apprentissage par renforcement inverse

Parfois, le comportement de l'expert est mieux compris non comme une séquence d'actions à imiter, mais comme le résultat de l'optimisation d'une fonction de récompense inconnue. **L'apprentissage par renforcement inverse (IRL)** {cite}`ng2000algorithms` récupère cette fonction de récompense latente à partir des démonstrations.

Étant donné des démonstrations $\tau = \{(s_1, a_1), \dots, (s_T, a_T)\}$, l'IRL trouve une fonction de récompense $R(s, a)$ telle que la politique de l'expert est optimale sous R.

L'attrait de l'IRL par rapport au BC : si nous récupérons la vraie fonction de récompense, nous pouvons la ré-optimiser dans de nouveaux environnements, avec des dynamiques différentes, ou avec des planificateurs améliorés — généralisant bien au-delà des scénarios démontrés.

IRL à entropie maximale

L'IRL MaxEnt {cite}`ziebart2008maximum` résout le problème d'ambiguïté de l'IRL (il existe de nombreuses fonctions de récompense cohérentes avec n'importe quel ensemble de démonstrations) en choisissant la fonction de récompense qui, tout en étant cohérente avec le comportement démontré, conduit à une distribution sur les trajectoires avec *entropie maximale*. Les trajectoires sont distribuées selon :

$$P(\tau \mid R) \propto \exp\left(\sum_t R(s_t, a_t)\right)$$

L'objectif d'apprentissage fait correspondre les espérances de caractéristiques observées de l'expert $\mu_E = \mathbb{E}_{\tau\sim\pi^*}[\phi(\tau)]$ aux espérances de caractéristiques du modèle $\mu_\theta = \mathbb{E}_{\tau\sim\pi_\theta}[\phi(\tau)]$.

GAIL : Apprentissage par imitation adversariale générative

GAIL {cite}`ho2016generative` contourne entièrement l'apprentissage de la fonction de récompense, utilisant une formulation de type GAN pour faire correspondre directement la distribution état-action de l'expert.

Un discriminateur D_ψ est entraîné à distinguer les paires état-action expertes $(s, a) \sim \pi^*$ des paires état-action de la politique $(s, a) \sim \pi_\theta$:

$$\mathcal{L}_D = -\mathbb{E}_{\pi^*}[\log D_\psi(s, a)] - \mathbb{E}_{\pi_\theta}[\log(1 - D_\psi(s, a))]$$

Le générateur (la politique π_θ) est entraîné à tromper le discriminateur — c'est-à-dire à produire des paires état-action qui ressemblent à celles de l'expert. La sortie du discriminateur $\log D_\psi(s, a)$ sert de signal de récompense pour la politique.

GAIL atteint des performances de niveau expert sur des références de contrôle continu avec bien moins de démonstrations que le BC, et généralise mieux que l'IRL MaxEnt dans des environnements complexes.

Clonage comportemental en TAL : un exemple pratique

```python
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(42)

# ----------------------------------------------
# Toy NLP task: rewriting sentences to be more formal
# We simulate this as a simple sequence transformation
# In practice: fine-tuning a seq2seq model on expert rewrites
# ----------------------------------------------

class SimpleSeq2Seq(nn.Module):
    def __init__(self, vocab_size=100, embed_dim=32, hidden_dim=64):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.encoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.decoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.proj = nn.Linear(hidden_dim, vocab_size)
        self.hidden_dim = hidden_dim

    def forward(self, src, tgt):
        src_emb = self.embed(src)
        _, hidden = self.encoder(src_emb)
        tgt_emb = self.embed(tgt)
        out, _ = self.decoder(tgt_emb, hidden)
        return self.proj(out)
```

```python
# Generate synthetic demonstration data
VOCAB = 100
rng = np.random.default_rng(42)
N, SEQ_LEN = 1000, 12

src_seqs = torch.tensor(rng.integers(1, VOCAB, (N, SEQ_LEN)), dtype=torch.long)
# "Expert" transformation: shift tokens by 1 (toy formalization)
tgt_seqs = torch.clamp(src_seqs + 1, 1, VOCAB - 1)
tgt_in  = torch.cat([torch.ones(N, 1, dtype=torch.long), tgt_seqs[:, :-1]], dim=1)

dataset = TensorDataset(src_seqs, tgt_in, tgt_seqs)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

model = SimpleSeq2Seq(vocab_size=VOCAB)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Behavioral cloning training
train_losses = []
for epoch in range(20):
    epoch_loss = 0
    for src, tgt_i, tgt_o in loader:
        logits = model(src, tgt_i)
        loss = criterion(logits.reshape(-1, VOCAB), tgt_o.reshape(-1))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        epoch_loss += loss.item()
    train_losses.append(epoch_loss / len(loader))

print(f"Initial loss: {train_losses[0]:.3f}")
print(f"Final loss:   {train_losses[-1]:.3f}")

# Evaluate: check token accuracy on held-out examples
model.eval()
with torch.no_grad():
    src_test = src_seqs[-100:]
    tgt_test_in = tgt_in[-100:]
    tgt_test_out = tgt_seqs[-100:]
    logits = model(src_test, tgt_test_in)
    preds = logits.argmax(dim=-1)
    acc = (preds == tgt_test_out).float().mean().item()
    print(f"Token accuracy on held-out set: {acc:.3f}")
```

Comparaison des méthodes d'apprentissage par imitation

Méthode	Nécessite une récompense ?	Expert interrogé en ligne ?	Généralise à de nouvelles dynamiques ?	Complexité
Clonage comportemental	Non	Non	Mal (décalage de distribution)	Faible

Méthode	Nécessite une récompense ?	Expert interrogé en ligne ?	Généralise à de nouvelles dynamiques ?	Complexité
DAgger	Non	Oui	Modérément	Moyenne
IRL MaxEnt	La récupère	Non	Bien	Élevée
GAIL	Non	Non	Bien	Élevée

Applications

Robotique. Enseigner aux robots à manipuler des objets, naviguer dans des environnements ou réaliser des tâches ménagères. Les démonstrations physiques sont collectées via la téléopération ou l'enseignement kinesthésique.

Conduite autonome. Les premiers systèmes de véhicules autonomes tels qu'ALVINN {cite}`pomerleau1989alvinn` et DAVE de NVIDIA s'appuyaient fortement sur le clonage comportemental à partir de données de conduite humaine.

IA pour les jeux. L'apprentissage par imitation sur le jeu humain amorce les agents avant l'affinement par RL. AlphaStar s'est entraîné sur des replays humains avant le RL ; cette approche est courante quand des démonstrations de niveau humain sont disponibles.

Génération de code. L'affinement de modèles de langage sur des démonstrations de code de haute qualité (GitHub Copilot, Codex) est une forme de clonage comportemental.

Aide à la décision clinique. Apprentissage à partir de séquences de décisions de médecins experts pour des protocoles complexes.

> **See also:** L'analyse fondatrice du BC/DAgger se trouve dans {cite}`ross2011reduction`. L'IRL MaxEnt est issu de {cite}`ziebart2008maximum`. GAIL est issu de {cite}`ho2016generative`. Pour une revue complète de l'apprentissage par imitation, voir {cite}`osa2018algorithmic`.

Apprentissage à partir de comparaisons et de classements

Demander aux humains d'attribuer un score de qualité absolu à une production est difficile. Quelle est la qualité numérique de cet essai, sur une échelle de 1 à 10 ? La question est mal posée : les humains ne disposent pas d'une échelle interne stable, et leurs scores sont fortement influencés par l'ancrage, le contexte et la fatigue.

Demander aux humains de *comparer* deux productions est bien plus facile : quel essai est meilleur, A ou B ? Les jugements comparatifs sont plus cohérents, plus fiables, et exploitent plus directement les préférences humaines que les évaluations absolues. Ce chapitre couvre les fondements mathématiques et les applications pratiques de l'apprentissage à partir de comparaisons et de classements.

Pourquoi les comparaisons sont meilleures que les évaluations

Fondement psychologique

La supériorité des jugements comparatifs a une longue histoire en psychométrie. La loi du jugement comparatif de Thurstone {cite}`thurstone1927law` (1927) a montré que même quand les humains ont des jugements absolus incohérents, leurs jugements relatifs suivent une loi probabiliste cohérente. Les comparaisons exploitent le fait que les humains sont bien meilleurs dans la perception *relative* que dans la calibration absolue.

Efficacité statistique

Chaque comparaison par paires fournit des informations sur les positions *relatives* de deux éléments sur l'échelle de qualité. Avec K éléments, $K - 1$ comparaisons peuvent classer tous les éléments ; seulement $O(\log K)$ comparaisons sont nécessaires pour trouver l'élément le mieux classé. Les évaluations absolues nécessitent généralement davantage de jugements pour atteindre la même précision.

Évolutivité

Pour les modèles génératifs, le nombre de sorties distinctes est effectivement infini. Évaluer absolument une sortie exige d'établir une échelle partagée sur toutes les sorties ; comparer des sorties ne requiert que des jugements locaux et relatifs naturellement calibrés l'un par rapport à l'autre.

Le modèle de Bradley-Terry

Le modèle probabiliste dominant pour les comparaisons par paires est le **modèle de Bradley-Terry (BT)** {cite}`bradley1952rank`. Chaque élément i possède un score de qualité latent $\alpha_i \in \mathbb{R}$. La probabilité que l'élément i soit préféré à l'élément j dans une comparaison directe est :

$$P(i \succ j) = \frac{e^{\alpha_i}}{e^{\alpha_i} + e^{\alpha_j}} = \sigma(\alpha_i - \alpha_j)$$

où σ est la fonction sigmoïde logistique. Cela équivaut à supposer que la qualité perçue de l'élément i est $\alpha_i + \epsilon$ où ϵ est un terme de bruit logistique standard.

Identifiabilité

Le modèle BT est identifiable à une translation près : si α est une solution, $\alpha + c$ l'est aussi pour toute constante c. La convention standard est de fixer un score (par exemple $\alpha_0 = 0$) ou de contraindre $\sum_i \alpha_i = 0$. Les scores ne sont identifiables que quand le **graphe de comparaison** (nœuds = éléments, arêtes = paires observées) est **connexe** — si le graphe a des composantes déconnectées, les scores relatifs entre composantes sont indéfinis.

Estimation des paramètres

Étant donné un ensemble de données de comparaisons par paires $\mathcal{D} = \{(i, j, y_{ij})\}$ où $y_{ij} = 1$ si i a été préféré à j, la log-vraisemblance est :

$$\mathcal{L}(\alpha) = \sum_{(i,j,y_{ij}) \in \mathcal{D}} \left[y_{ij} \log \sigma(\alpha_i - \alpha_j) + (1 - y_{ij}) \log \sigma(\alpha_j - \alpha_i) \right]$$

C'est une fonction concave de α qui peut être maximisée par montée de gradient ou par la méthode de Newton.

```python
import numpy as np
from scipy.optimize import minimize
from scipy.special import expit  # sigmoid

rng = np.random.default_rng(42)

# ----------------------------------------------
# Simulate Bradley-Terry: 8 items with true quality scores
# Generate pairwise comparisons and recover the scores
# ----------------------------------------------

N_ITEMS = 8
true_alpha = rng.normal(0, 1, N_ITEMS)
print(f"True quality ranking: {np.argsort(true_alpha)[::-1]}")

# Generate comparisons: every pair compared 5 times
comparisons = []
for i in range(N_ITEMS):
    for j in range(i + 1, N_ITEMS):
        for _ in range(5):
            p_i_wins = expit(true_alpha[i] - true_alpha[j])
```

```
            winner = i if rng.random() < p_i_wins else j
            loser  = j if winner == i else i
            comparisons.append((winner, loser))

print(f"Total comparisons: {len(comparisons)}")

def neg_log_likelihood(alpha, comparisons):
    """Bradley-Terry negative log-likelihood."""
    alpha = np.array(alpha)
    loss = 0.0
    for winner, loser in comparisons:
        log_prob = np.log(expit(alpha[winner] - alpha[loser]) + 1e-10)
        loss -= log_prob
    return loss

def neg_grad(alpha, comparisons):
    alpha = np.array(alpha)
    grad = np.zeros(len(alpha))
    for winner, loser in comparisons:
        p = expit(alpha[winner] - alpha[loser])
        grad[winner] -= (1 - p)
        grad[loser]  -= (-p)
    return grad

# Fix alpha[0] = 0 to resolve scale ambiguity
result = minimize(
    lambda a: neg_log_likelihood(np.concatenate([[0.0], a]), comparisons),
    x0=np.zeros(N_ITEMS - 1),
    jac=lambda a: neg_grad(np.concatenate([[0.0], a]), comparisons)[1:],
    method='L-BFGS-B'
)
est_alpha = np.concatenate([[0.0], result.x])

# Compare true vs estimated ranking
true_rank = np.argsort(true_alpha)[::-1]
est_rank  = np.argsort(est_alpha)[::-1]

print(f"\nTrue ranking (item indices): {true_rank}")
print(f"Estimated ranking:           {est_rank}")
rank_corr = np.corrcoef(true_alpha, est_alpha)[0, 1]
print(f"Correlation with true scores: {rank_corr:.4f}")
```

Le modèle de Thurstone

Le modèle de Thurstone {cite}`thurstone1927law` est étroitement lié au modèle de Bradley-Terry mais utilise un bruit gaussien plutôt que logistique :

$$P(i \succ j) = \Phi\left(\frac{\alpha_i - \alpha_j}{\sqrt{2}\sigma}\right)$$

où Φ est la fonction de répartition de la loi normale standard. Quand $\sigma = 1/\sqrt{2}$, cela devient équivalent au BT avec une légère différence d'échelle. En pratique, les deux modèles donnent des résultats presque identiques.

Agrégation de classements

Quand chaque annotateur fournit un classement complet de K éléments (plutôt que des comparaisons par paires), le problème est **l'agrégation de classements** : combiner plusieurs listes classées en un classement de consensus.

Le compte de Borda : Chaque élément reçoit un score égal au nombre d'éléments classés en dessous de lui dans le classement de chaque annotateur. Les scores sont sommés sur tous les annotateurs. Simple et robuste.

Kemeny–Young : Trouver le classement qui minimise la somme des désaccords par paires (la distance de Kendall tau) avec le classement de chaque annotateur. Ceci est NP-difficile pour un grand K mais réalisable pour de petits ensembles.

RankNet / ListNet : Des approches neuronales qui apprennent une fonction de score à partir de listes classées, permettant la généralisation à des éléments non vus.

Les bandits duellants

Dans l'apprentissage par préférences **en ligne**, les éléments arrivent en flux continu et nous devons décider quelles paires comparer, en équilibrant l'exploration (apprendre sur des éléments inconnus) et l'exploitation (présenter des éléments de haute qualité). C'est le problème des **bandits duellants** {cite}`yue2009interactively`.

Algorithmes clés : - **Doubler :** Maintient un élément champion ; le défie avec des concurrents aléatoires - **RUCB (Borne de Confiance Supérieure Relative) :** Calcule des intervalles de confiance de style UCB pour la probabilité de chaque élément de battre chaque autre élément - **MergeRank :** Combine la comparaison par tournoi avec UCB

Les bandits duellants sont utilisés dans les systèmes de recommandation en ligne (quel article afficher ensuite, étant donné un retour implicite) et l'élicitation de préférences interactives pour la collecte de données RLHF.

L'apprentissage par préférences pour les modèles de langage

Dans le contexte du RLHF (Chapitre 6), le modèle de Bradley-Terry est utilisé pour entraîner le modèle de récompense. Une variante importante est **l'Optimisation Directe des Préférences (DPO)** {cite}`rafailov2023direct`, qui montre que l'objectif RLHF peut être optimisé directement à partir des données de préférences sans entraîner un modèle de récompense séparé :

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

Le DPO est plus simple que le RLHF complet (pas de boucle d'entraînement PPO, pas de modèle de récompense) tout en atteignant des résultats comparables ou meilleurs sur de nombreuses références {cite}`rafailov2023direct`. Il est devenu une alternative largement adoptée au RLHF basé sur PPO pour l'affinement par suivi d'instructions, bien que les deux approches restent en usage actif et que leurs forces relatives dépendent de la tâche.

Collecter des données de préférences de haute qualité

La qualité des données de préférences détermine la qualité du modèle de récompense. Considérations clés :

Diversité des prompts. Les données de préférences devraient couvrir la distribution complète des prompts que le modèle rencontrera lors du déploiement. Les lacunes de couverture conduisent à un comportement peu fiable du modèle de récompense dans ces régions.

Diversité des réponses. Comparer deux réponses très similaires fournit peu d'information. Les réponses comparées devraient être suffisamment différentes pour que les annotateurs aient une préférence claire.

Accord inter-annotateurs. Un faible accord inter-annotateurs suggère que les critères de comparaison sont ambigus. Mesurer l'accord (kappa de Cohen) et réviser les consignes quand il est en dessous des seuils acceptables.

Calibration. Les annotateurs devraient comprendre *pourquoi* une réponse est meilleure : utilité, précision, sécurité, style ? Les tâches qui regroupent plusieurs critères tendent à produire des préférences incohérentes. Il est souvent préférable de collecter des préférences sur chaque critère séparément.

> **See also:** Modèle de Bradley-Terry : {cite}`bradley1952rank`. Thurstone : {cite}`thurstone1927law`. Bandits duellants : {cite}`yue2009interactively`. Optimisation Directe des Préférences (DPO) : {cite}`rafailov2023direct`.

L'humain dans la boucle en traitement automatique des langues

Le langage naturel est le domaine où le HITL ML a sans doute eu le plus grand impact — et où ses difficultés conceptuelles les plus profondes se manifestent. Le langage est intrinsèquement social : sa signification est construite par des communautés humaines, sa pragmatique dépend du contexte et de l'intention, et sa qualité ne peut être jugée que par des lecteurs humains. Mais cela signifie également que l'annotation en TAL n'est pas simplement un processus de collecte d'observations. C'est un processus de *construction* de catégories.

Le problème constitutif dans l'annotation en TAL

En imagerie médicale, il existe une vérité de terrain : une tumeur est présente ou elle ne l'est pas. L'étiquette du radiologue peut être incertaine, mais elle tente de suivre quelque chose de réel. L'annotation en TAL est souvent fondamentalement différente. Quand un annotateur marque un tweet comme « toxique », il n'y a pas de molécule toxique dans le tweet que nous tentons de détecter. **L'étiquette constitue la catégorie.**

Cela a des conséquences profondes qui sont fréquemment sous-estimées :

La main-d'œuvre d'annotation définit le phénomène. Un schéma d'étiquettes pour le « discours offensant » encode les sensibilités de celui qui l'a conçu et de ceux qui l'ont appliqué. Une équipe d'annotateurs anglophones issus d'un seul groupe démographique, travaillant sous des consignes rédigées par une équipe de politique d'entreprise, produit un jeu de données qui reflète la compréhension de cette équipe des offenses — non un standard humain universel. Les modèles entraînés sur de telles données exhiberont ces mêmes frontières implicites.

Les consignes sont de la théorie, qu'elles le reconnaissent ou non. Tout schéma d'annotation formule des affirmations sur l'ontologie. Décider que « l'ironie » et le « sarcasme » appartiennent à la même classe est une affirmation théorique, non une commodité neutre. Décider d'étiqueter « la colère » comme une classe unique plutôt que de distinguer « la colère juste » de la « colère hostile » effondre une distinction qui peut importer pour la tâche en aval. Ces décisions sont prises sous pression de production et rarement revisitées.

L'instabilité des étiquettes dans le temps. Le langage social évolue. Un modèle de toxicité entraîné en 2018 mal classifiera le contenu de 2024 non pas parce qu'il est statistiquement sous-entraîné, mais parce que la signification sociale de certains termes a changé. L'annotation en TAL n'échantillonne pas une distribution statique — elle échantillonne une cible mouvante que l'acte d'étiquetage contribue partiellement à constituer.

:::{admonition} Le problème des artefacts d'annotation :class: important

Geva et al. {cite}`geva2019annotator` ont démontré que les jeux de données d'inférence en langue naturelle (NLI) contiennent des artefacts systématiques introduits par le processus d'annotation lui-même. Les annotateurs invités à rédiger des hypothèses « d'implication » pour une prémisse donnée tendent à utiliser certains schémas syntaxiques ; ceux qui rédigent des hypothèses de « contradiction » en utilisent d'autres. Les modèles apprennent à classifier sur la base de ces artefacts plutôt que de la relation sémantique visée — ils résolvent la tâche d'annotation, non la tâche TAL sous-jacente.

Ce n'est pas de la négligence. C'est une conséquence inhérente du fait de demander aux humains de construire des exemples pour correspondre à une étiquette. Le processus HITL insère un signal systématique qui n'était jamais censé être dans les données. :::

Annotation de classification de texte

La tâche d'annotation TAL la plus simple est l'attribution d'une catégorie à un document textuel. L'analyse de sentiment, la classification thématique, la détection d'intention et le filtrage de spam sont tous des tâches de classification.

Défis spécifiques à la classification de texte :

Subjectivité. Les catégories comme « toxique » ou « offensant » sont intrinsèquement subjectives et varient selon les contextes culturels, les antécédents des annotateurs et le contexte de la plateforme. D'un annotateur à l'autre, les perceptions de l'offensivité diffèrent significativement selon les données démographiques — un fait aux implications directes pour l'équité {cite}`blodgett2020language`.

Ambiguïté des étiquettes. De nombreux documents appartiennent à plusieurs catégories ou se situent à la frontière. Une critique à 60 % positive et 40 % négative est genuinement ambiguë, non mal étiquetée. Forcer une seule étiquette écarte de véritables informations (voir le Chapitre 13 sur les étiquettes souples et le désaccord des annotateurs).

Granularité des étiquettes. Un modèle de sentiment à 2 classes peut suffire pour la surveillance des médias sociaux ; une échelle à 7 points peut être nécessaire pour l'analyse des retours produits. La granularité appropriée dépend de la tâche en aval, mais est généralement fixée avant l'annotation — une décision de conception conséquente prise avec des données insuffisantes.

Reconnaissance d'entités nommées

L'annotation NER nécessite d'identifier des spans de texte et d'assigner un type d'entité (PERSONNE, ORGANISATION, LIEU, etc.). L'annotation est plus complexe que la classification de documents pour plusieurs raisons :

Les frontières de span sont ambiguës. Dans « le PDG d'Apple Tim Cook a annoncé… », l'entité PERSONNE couvre-t-elle « Tim Cook » ou « le PDG d'Apple Tim Cook » ? Les consignes doivent explicitement traiter ces cas, et l'accord inter-annotateurs sur les spans est systématiquement inférieur à celui sur les types.

L'attribution de types nécessite une connaissance du monde. « Apple » est une ORG dans un contexte, un PRODUIT dans un autre, et sans doute aucun des deux dans « tarte aux

pommes ». Les annotateurs ont besoin d'une connaissance du domaine suffisante pour faire des attributions de types correctes.

Entités imbriquées. « L'Université de Californie, Berkeley » contient une ORGANISATION imbriquée dans un LIEU. Le balisage BIO standard ne peut pas représenter les entités imbriquées ; des schémas plus complexes (par exemple BIOES, ou des formats basés sur des graphes) sont requis.

Efficacité d'annotation. La pré-annotation avec un modèle NER de référence accélère significativement l'annotation en permettant aux annotateurs de corriger les prédictions plutôt que d'annoter de zéro. Dans une étude sur la NER clinique, des augmentations de débit de 30 à 60 % ont été observées {cite}`lingren2014evaluating` ; l'ampleur de ces gains dépend fortement de la qualité du modèle de référence et du domaine.

Extraction de relations et annotation sémantique

Au-delà de l'identification des entités, l'extraction de relations exige l'annotation des *relations* entre entités :

- Les annotateurs doivent comprendre à la fois les entités et le prédicat qui les relie
- Les types de relations ont des distinctions sémantiques complexes (TRAVAILLE_POUR vs. EST_EMPLOYÉ_PAR vs. A_FONDÉ)
- Les exemples négatifs (paires d'entités sans relation) sont bien plus courants que les positifs

L'accord inter-annotateurs pour l'extraction de relations tend à être inférieur à celui des tâches de classification. Pour des schémas bien définis, des valeurs de κ dans la plage 0,65–0,80 sont couramment rapportées {cite}`pustejovsky2012natural` ; pour des relations sémantiques complexes (causalité d'événements, relations temporelles), l'accord peut être considérablement plus faible selon la conception du schéma et la formation des annotateurs.

Post-édition de traduction automatique (MTPE)

La post-édition de traduction automatique est une forme mature du HITL en TAL. Un traducteur humain corrige la sortie d'un système de TA plutôt que de traduire à partir de zéro :

Post-édition légère (LPE) : Seules les erreurs critiques sont corrigées. Appropriée quand les exigences de qualité de traduction sont modérées.

Post-édition complète (FPE) : La sortie est corrigée jusqu'à une qualité de publication. La sortie éditée devient typiquement des données d'entraînement pour une amélioration ultérieure de la TA — un véritable cycle de raffinement avec l'humain dans la boucle.

HTER (Taux d'édition ciblée humaine) : Une métrique qui mesure la distance d'édition entre la sortie de la TA et la traduction post-éditée, normalisée par la longueur de la phrase {cite}`graham2015accurate`. En règle générale praticienne approximative, un HTER inférieur à environ 0,3 est souvent considéré comme une bonne sortie de TA ; au-dessus de 0,5, la traduction à partir de zéro peut être plus rapide — bien que ces seuils varient selon le domaine et la paire de langues.

IA conversationnelle et annotation de dialogues

L'annotation de dialogue introduit une structure temporelle :

- **Annotation au niveau du tour :** étiqueter chaque tour (intention, sentiment, qualité)
- **Annotation au niveau du dialogue :** évaluer la cohérence globale, le succès de la tâche, la satisfaction de l'utilisateur
- **Annotation de trace d'interaction :** identifier des moments de défaillance spécifiques dans une conversation

Le HITL est particulièrement important dans le dialogue parce que les défaillances du modèle sont souvent subtiles et cumulatives : une erreur factuelle au tour 3 peut ne pas devenir apparente avant le tour 7. Les annotateurs humains qui tracent des conversations peuvent identifier ces schémas de défaillance à longue portée que les métriques automatisées manquent entièrement.

Étiquetage programmatique et supervision faible

Quand les données étiquetées sont rares, la **supervision faible** permet d'utiliser plusieurs fonctions d'étiquetage bruitées et se chevauchant pour générer des étiquettes probabilistes à grande échelle. **Snorkel** {cite}`ratner2017snorkel` est le cadre canonique :

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

rng = np.random.default_rng(42)

# ---------------------------------------------------------
# Toy weak supervision: sentiment classification
# 5 labeling functions (LFs) with different coverage/accuracy
# Label: +1 (positive), -1 (negative), 0 (abstain)
# ---------------------------------------------------------

N = 1000
true_labels = rng.choice([-1, 1], size=N)
X_features = np.column_stack([
    true_labels * 0.8 + rng.normal(0, 0.5, N),
    rng.normal(0, 1, N),
    rng.normal(0, 1, N),
])

def make_lf(accuracy, coverage, seed):
    rng_lf = np.random.default_rng(seed)
    votes = np.zeros(N, dtype=int)
    active = rng_lf.random(N) < coverage
    correct = rng_lf.random(N) < accuracy
    votes[active & correct]  = true_labels[active & correct]
    votes[active & ~correct] = -true_labels[active & ~correct]
    return votes
```

```
LFs = np.column_stack([
    make_lf(accuracy=0.85, coverage=0.60, seed=1),
    make_lf(accuracy=0.78, coverage=0.45, seed=2),
    make_lf(accuracy=0.70, coverage=0.80, seed=3),
    make_lf(accuracy=0.90, coverage=0.30, seed=4),
    make_lf(accuracy=0.60, coverage=0.90, seed=5),
])

def majority_vote(LF_matrix):
    labels = []
    for i in range(LF_matrix.shape[0]):
        votes = LF_matrix[i][LF_matrix[i] != 0]
        labels.append(0 if len(votes) == 0 else int(np.sign(votes.sum())))
    return np.array(labels)

mv_labels = majority_vote(LFs)
covered = mv_labels != 0

print(f"Coverage:                    {covered.mean():.1%}")
print(f"MV accuracy (covered):       {(mv_labels[covered] == true_labels[covered]).mean():.3f

X_train, y_train = X_features[covered], mv_labels[covered]
X_test,  y_test  = X_features[~covered], true_labels[~covered]

if len(X_train) > 10 and len(X_test) > 10:
    clf = LogisticRegression().fit(X_train, y_train)
    preds = clf.predict(X_test)
    print(f"F1 on uncovered test set:    {f1_score(y_test, preds):.3f}")
```

RLHF pour les modèles de langage : la réalité de l'annotation

Le Chapitre 6 a couvert le RLHF techniquement. Du point de vue du TAL, la tâche d'annotation est plus difficile qu'elle n'y paraît de l'extérieur.

Ce que l'on demande réellement aux annotateurs — comparer deux sorties du modèle et indiquer laquelle est « meilleure » — semble simple. En pratique, « meilleure » est un concept sous-spécifié que les annotateurs résolvent à l'aide d'heuristiques personnelles. Certains pondèrent fortement la fluidité ; d'autres pondèrent la précision factuelle ; d'autres pénalisent la verbosité. Sans consignes strictes, le jeu de données de préférences résultant ne reflète pas les valeurs humaines en abstrait mais les stratégies de résolution particulières de la main-d'œuvre d'annotation employée.

Les dimensions d'annotation clés sont :

- *Utilité :* La réponse répond-elle bien à la question ? Est-elle informative, claire et appropriément détaillée ?
- *Exactitude factuelle :* La réponse est-elle factuellement précise ? Cela exige des évaluateurs une connaissance du domaine — une exigence qui crée de sérieux problèmes de qualité à grande échelle, puisque les annotateurs généralistes ne peuvent pas vérifier les affirmations de spécialistes.
- *Inoffensivité :* La réponse évite-t-elle un contenu toxique, biaisé, nuisible ou inapproprié

? Ces jugements nécessitent des consignes détaillées car « nuisible » est hautement dépendant du contexte et varie selon les cultures, les époques et le contexte de la plateforme.
- *Calibration :* La réponse exprime-t-elle une incertitude appropriée ? Les annotateurs préfèrent systématiquement les réponses qui sonnent sûres d'elles, ce qui crée un signal de récompense contre l'humilité épistémique appropriée.

L'interaction entre les critères est complexe : une réponse maximalement utile à une question dangereuse peut être maximalement nuisible. Les consignes doivent spécifier comment arbitrer entre des critères concurrents — et ces arbitrages sont effectivement des décisions politiques, non des décisions d'annotation. La main-d'œuvre d'annotation fait de la politique.

L'échelle concentre l'influence démographique. Le RLHF pour les grands modèles implique des effectifs d'annotation relativement réduits (de quelques centaines à quelques milliers) prenant des milliards de décisions en aval. Les biais démographiques et culturels de cette main-d'œuvre se propagent dans le comportement du modèle à grande échelle d'une manière qui ne se produirait pas si l'annotation était plus distribuée. C'est l'un des problèmes structurels les moins discutés du pipeline RLHF actuel.

La boucle de retour annotation–modèle

En TAL plus que dans tout autre domaine, les processus d'annotation et de développement du modèle s'entremêlent dans le temps :

1. Les annotateurs sont calibrés en utilisant les sorties existantes du modèle comme référence (souvent implicitement).
2. Le modèle de récompense apprend ce que les annotations tendent à préférer.
3. Le générateur est affiné pour produire des sorties qui obtiennent une récompense élevée.
4. Ces sorties influencent ce à quoi ressemble une « bonne » réponse dans les tours d'annotation suivants.

Cette boucle de retour n'est pas intrinsèquement pathologique — c'est ce qui permet au RLHF de converger — mais elle signifie que le comportement du modèle est façonné par une cible mouvante que le processus d'annotation lui-même contribue à déplacer. Distinguer ce que le modèle a appris parce que cela reflète les préférences humaines de ce qu'il a appris parce qu'il a appris à reconnaître les heuristiques des annotateurs est empiriquement difficile.

Il n'y a pas de solution propre. La meilleure pratique actuelle consiste à surveiller la dérive à l'aide de jugements de préférences mis de côté collectés à intervalles réguliers, et à traiter la version des consignes d'annotation comme une variable expérimentale.

Évaluation des modèles TAL génératifs

Contrairement aux modèles de classification qui ont une métrique de précision claire, l'évaluation de la qualité de génération nécessite le jugement humain :

Méthode d'évaluation	Description	Coût
Évaluation directe (DA)	Les annotateurs évaluent la qualité sur une échelle	Moyen

Méthode d'évaluation	Description	Coût
Jugement comparatif	Les annotateurs comparent deux sorties	Faible
MQM (Métriques de qualité multidimensionnelles)	Taxonomie structurée des erreurs	Élevé
Préférence RLHF	Étiquettes de préférences utilisées pour l'entraînement	Moyen
LLM-comme-juge	Un LLM évalue les sorties (corrèle modérément avec l'humain)	Très faible
BERTScore, BLEU	Métriques automatiques (corrélation imparfaite avec le jugement humain)	Très faible

Les métriques automatiques (BLEU pour la TA, ROUGE pour la résumé) sont peu coûteuses mais imparfaitement corrélées avec les jugements humains de qualité {cite}`reiter2018structured`. Les approches LLM-comme-juge montrent un accord modéré avec les annotateurs humains {cite}`zheng2023judging` et sont de plus en plus utilisées pour les itérations rapides, mais ne devraient pas remplacer l'évaluation humaine pour les évaluations finales. Pour les décisions à enjeux réels, l'évaluation humaine reste nécessaire.

> **See also:** Supervision faible Snorkel : {cite}`ratner2017snorkel`. Consignes d'annotation en TAL : {cite}`pustejovsky2012natural`. Artefacts d'annotation dans la NLI : {cite}`geva2019annotator`. Biais des annotateurs et jeux de données TAL : {cite}`blodgett2020language`. Pour l'évaluation des modèles génératifs : {cite}`reiter2018structured`.

L'humain dans la boucle en vision par ordinateur

La vision par ordinateur fournit certains des exemples les plus visuellement intuitifs du HITL ML. Le défi ImageNet, construit sur 14 millions d'images étiquetées par des humains, a lancé l'ère de l'apprentissage profond. L'annotation d'images médicales par des radiologues alimente le diagnostic par IA. Les véhicules autonomes dépendent de millions de images étiquetées par des humains pour apprendre à percevoir le monde.

Ce qu'il est facile de négliger : ce ne sont pas simplement des cas où les humains fournissent une vérité de terrain. Ce sont des cas où les humains construisent des jeux de données qui incorporent des choix perceptuels, culturels et opérationnels spécifiques — des choix qui ne deviennent visibles que plus tard, quand les modèles échouent de manières prévisibles.

Comment les choix d'annotation deviennent des biais du modèle

La présentation standard traite l'annotation comme une collecte de données : les humains observent le monde et enregistrent ce qu'ils voient. La présentation plus précise est que l'annotation est de la *conception de jeu de données* : les humains décident quelles catégories utiliser, où tracer les frontières, quels cas limites inclure et comment résoudre l'ambiguïté — et toutes ces décisions façonnent ce que le modèle entraîné percevra.

Le cas ImageNet

ImageNet {cite}`russakovsky2015imagenet` est le jeu de données le plus important de l'histoire de la vision par ordinateur. Son ensemble d'étiquettes dérive des synsets WordNet, choisis principalement pour être nombreux et sémantiquement distincts. Plusieurs conséquences de ce choix de conception sont apparues des années plus tard :

- **Les catégories de personnes encodaient des associations démographiques.** Les premières versions des étiquettes de synsets pour les personnes dans ImageNet incluaient beaucoup de termes qui seraient aujourd'hui considérés comme dégradants ou biaisés, reflétant à la fois la source historique de WordNet et les décisions implicites de la main-d'œuvre d'annotation sur les étiquettes à appliquer à quelles images {cite}`yang2020towards`. Les étiquettes appliquées aux images de personnes encodaient des associations de race, de genre et de classe qui se sont directement propagées dans les plongements des modèles.
- **Taxonomie fine des espèces, grossière pour presque tout le reste.** ImageNet peut distinguer 120 races de chiens mais regroupe une énorme variation en outils, véhicules, nourriture et mobilier en catégories uniques. C'était une conséquence du suivi de la

structure de WordNet, non un choix délibéré sur ce qui importe. Les modèles entraînés sur ImageNet présentent la même précision asymétrique.

- **Valeurs visuelles par défaut occidentales et anglophones.** Les images ont été principalement collectées depuis Flickr et des recherches Internet utilisant des requêtes en anglais. La distribution résultante est fortement orientée vers l'environnement visuel et les objets culturels des pays riches anglophones.

Aucune de ces décisions n'était une erreur. C'étaient des choix de conception d'annotation faits rapidement à grande échelle, souvent par des personnes qui n'anticipaient pas comment le jeu de données serait utilisé. La leçon n'est pas qu'ImageNet aurait dû être construit différemment (même si c'est le cas), mais que **la conception d'annotation est la conception du modèle**, et devrait être traitée avec le même soin.

:::{admonition} Le schéma d'annotation est une théorie du monde :class: note

Toute taxonomie d'étiquettes formule des affirmations sur quelles distinctions importent. Choisir de séparer « voiture » de « camion » tout en regroupant toutes les berlines dans une classe est une affirmation théorique sur quelles distinctions sont sémantiquement pertinentes. Choisir d'annoter « personne » comme une classe unique quelle que soit la pose, les vêtements ou l'activité est une affirmation théorique différente. Les modèles entraînés sur ces schémas feront les mêmes distinctions, et pas plus — ils ne généraliseront pas au-delà des catégories qu'ils ont été entraînés à distinguer. :::

Annotation de classification d'images

La tâche d'annotation CV la plus simple est l'attribution d'une ou plusieurs étiquettes à une image entière.

Hiérarchie d'étiquettes. L'étiquette « chien » est un enfant de « animal » dans une hiérarchie sémantique. Les modèles entraînés sur des taxonomies plates peuvent ne pas bien généraliser à travers les niveaux d'abstraction. ImageNet utilise une hiérarchie basée sur les synsets qui permet l'évaluation à plusieurs niveaux de spécificité.

Ambiguïté multi-étiquettes. Une scène de rue peut contenir simultanément une voiture, une personne, un vélo et un feu de circulation. Décider quelles étiquettes inclure nécessite des consignes claires sur les seuils de pertinence.

Distributions longue queue. Les jeux de données d'images naturelles suivent une loi de puissance : quelques catégories sont extrêmement courantes ; la plupart sont rares. L'apprentissage actif est particulièrement précieux pour les catégories longue queue où l'échantillonnage aléatoire ne fournit qu'une poignée d'exemples.

Détection d'objets : annotation de boîtes englobantes

La détection d'objets exige des annotateurs qu'ils dessinent des boîtes englobantes alignées sur les axes autour des instances de chaque classe d'objet. Cela introduit des exigences de précision géométrique et des cas limites significatifs.

Métriques de qualité d'annotation :

L'IoU (Intersection sur Union) mesure le chevauchement entre une boîte annotée et une boîte de référence :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

IoU $\geq$ 0,5 est le seuil standard pour une « détection correcte » dans PASCAL VOC ; COCO {cite}`lin2014microsoft` utilise une gamme de seuils de 0,5 à 0,95, ce qui est considérablement plus exigeant et plus informatif.

Cas limites d'annotation à résoudre dans les consignes : - Objets occultés : annoter la partie visible ou extrapoler l'étendue complète ? - Objets tronqués (partiellement hors cadre) : inclure ou exclure ? - Régions de foule : utiliser une annotation spéciale « foule » ou annoter les instances individuelles ?

Chacune de ces décisions change ce que signifie une « détection correcte » — et donc change ce que le modèle est entraîné à faire.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def compute_iou(boxA, boxB):
    """Compute IoU between two boxes [x1, y1, x2, y2]."""
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])
    inter_area = max(0, xB - xA) * max(0, yB - yA)
    boxA_area = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    boxB_area = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    union_area = boxA_area + boxB_area - inter_area
    return inter_area / (union_area + 1e-6)

ref_box  = [1.0, 1.0, 4.0, 4.0]
ann1_box = [1.1, 0.9, 4.1, 4.2]   # close
ann2_box = [0.5, 0.5, 3.5, 3.8]   # less precise

print(f"IoU (ann1 vs ref):  {compute_iou(ann1_box, ref_box):.3f}")
print(f"IoU (ann2 vs ref):  {compute_iou(ann2_box, ref_box):.3f}")
print(f"IoU (ann1 vs ann2): {compute_iou(ann1_box, ann2_box):.3f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(0, 5); ax.set_ylim(0, 5); ax.set_aspect('equal')
ax.add_patch(patches.Rectangle(
    (ref_box[0], ref_box[1]), ref_box[2]-ref_box[0], ref_box[3]-ref_box[1],
    linewidth=2.5, edgecolor='#2b3a8f', facecolor='none', label='Reference'))
ax.add_patch(patches.Rectangle(
    (ann1_box[0], ann1_box[1]), ann1_box[2]-ann1_box[0], ann1_box[3]-ann1_box[1],
    linewidth=2, edgecolor='#0d9e8e', facecolor='none', linestyle='--',
    label=f'Ann1 (IoU={compute_iou(ann1_box, ref_box):.2f})'))
ax.add_patch(patches.Rectangle(
    (ann2_box[0], ann2_box[1]), ann2_box[2]-ann2_box[0], ann2_box[3]-ann2_box[1],
```

```
    linewidth=2, edgecolor='#e05c5c', facecolor='none', linestyle=':',
    label=f'Ann2 (IoU={compute_iou(ann2_box, ref_box):.2f})'))
ax.legend(fontsize=10)
ax.set_title("Bounding Box Agreement (IoU)", fontsize=12)
plt.tight_layout()
plt.savefig('bbox_iou.png', dpi=150)
plt.show()
```

Segmentation sémantique et d'instance

L'annotation de segmentation nécessite d'assigner une étiquette de classe à chaque pixel d'une image — parmi les types d'annotation les plus coûteux.

Segmentation sémantique : Chaque pixel appartient à une classe sémantique (route, ciel, personne, voiture). Tous les pixels de la même classe partagent la même étiquette, quel que soit l'objet individuel auquel ils appartiennent.

Segmentation d'instance : Chaque instance d'objet individuel reçoit une étiquette unique. Une foule de 20 personnes devient 20 masques distincts.

Segmentation panoptique : Combine les deux : les classes « chose » (objets dénombrables) ont des masques d'instance ; les classes « matière » (route, ciel) ont des masques sémantiques.

Annotation assistée par SAM : Le Modèle Segment Anything de Meta {cite}`kirillov2023segment` génère des masques de segmentation de haute qualité à partir d'un simple clic de point. Les annotateurs cliquent sur le centre d'un objet ; SAM propose un masque ; l'annotateur accepte ou corrige. Les auteurs de SAM rapportent des accélérations d'environ 6,5 fois par rapport à l'étiquetage basé sur des polygones ; les gains varient selon les types de scènes et les outils d'annotation.

SAM représente un changement plus large : **le rôle de l'annotateur passe du dessin à la révision**. Cela a des implications pour la qualité d'annotation au-delà de la vitesse. Quand les annotateurs dessinent, leur attention est mobilisée tout au long du processus. Quand les annotateurs révisent et cliquent sur « accepter », il y a des preuves qu'ils manquent plus facilement les erreurs — une version du biais d'automatisation spécifique au contexte d'annotation.

Apprentissage actif pour la vision par ordinateur

L'apprentissage actif est particulièrement précieux en CV parce que : 1. Les images sont de haute dimension et riches en caractéristiques — les plongements des modèles pré-entraînés portent des signaux forts pour l'estimation de l'incertitude 2. Les grands pools non étiquetés sont bon marché (photos, images vidéo) 3. L'annotation (surtout la segmentation) est coûteuse et ne peut pas être facilement externalisée pour des domaines spécialisés

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```python
rng = np.random.default_rng(42)

X, y = make_classification(n_samples=3000, n_features=50, n_informative=25,
                           n_classes=5, n_clusters_per_class=2, random_state=42)
X_train, y_train = X[:2500], y[:2500]
X_test,  y_test  = X[2500:], y[2500:]

def margin_uncertainty(model, X_pool):
    probs = model.predict_proba(X_pool)
    sorted_p = np.sort(probs, axis=1)
    return sorted_p[:, -2] - sorted_p[:, -1]  # most negative = most uncertain

n_init = 50
results = {'active': [], 'random': []}
label_counts = list(range(50, 401, 30))

for strategy in ['active', 'random']:
    labeled = list(rng.choice(len(X_train), n_init, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]

    for target in label_counts:
        while len(labeled) < target and unlabeled:
            if strategy == 'active' and len(labeled) >= 10:
                model_temp = LogisticRegression(max_iter=300).fit(
                    X_train[labeled], y_train[labeled])
                margins = margin_uncertainty(model_temp, X_train[unlabeled])
                idx = int(np.argmin(margins))
            else:
                idx = rng.integers(0, len(unlabeled))
            labeled.append(unlabeled.pop(idx))

        clf = LogisticRegression(max_iter=300).fit(X_train[labeled], y_train[labeled])
        results[strategy].append(accuracy_score(y_test, clf.predict(X_test)))

plt.figure(figsize=(7, 4))
plt.plot(label_counts, results['active'], 'o-', color='#2b3a8f',
         linewidth=2, label='Active (margin sampling)')
plt.plot(label_counts, results['random'], 's--', color='#e05c5c',
         linewidth=2, label='Random baseline')
plt.xlabel("Labeled training images", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning for 5-Class Image Classification", fontsize=13)
plt.legend(fontsize=11); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('cv_active_learning.png', dpi=150)
plt.show()
```

Apprentissage semi-supervisé avec guidage humain

La grande quantité de données visuelles non étiquetées disponibles rend l'apprentissage semi-supervisé particulièrement attrayant pour la CV.

Auto-entraînement / pseudo-étiquetage : Entraîner un modèle sur des données étiquetées ; utiliser les prédictions à haute confiance sur des données non étiquetées comme pseudo-étiquettes ; ré-entraîner. La question de conception critique est le seuil de confiance. Un seuil bas apporte plus d'exemples mais introduit du bruit ; un seuil élevé laisse la plupart du pool non étiqueté inutilisé. L'implication humaine peut guider ce seuil — les annotateurs vérifient un échantillon d'exemples pseudo-étiquetés pour le calibrer.

FixMatch et régularisation par cohérence : Ces méthodes entraînent des modèles à produire des prédictions cohérentes sous augmentation. L'intuition HITL clé : les humains sont consultés non seulement pour les étiquettes mais pour la **conception d'augmentation** — quelles invariances le modèle devrait-il apprendre ? Un modèle pour l'imagerie médicale devrait être invariant à la rotation et à la luminosité mais pas à l'échelle ; un modèle pour la détection de texte ne devrait pas être rendu invariant à la distorsion perspective. Ces choix spécifiques au domaine nécessitent une expertise humaine, et les commettre à tort dégrade substantiellement l'apprentissage semi-supervisé.

Apprentissage actif semi-supervisé : La combinaison la plus efficace : l'apprentissage actif concentre les étiquettes humaines là où l'incertitude du modèle est la plus élevée ; l'auto-entraînement étiquette automatiquement la queue à haute confiance. L'effort humain se concentre là où il est le plus précieux, et le modèle bootstrap sur le reste. À chaque cycle, un audit humain d'un échantillon aléatoire de pseudo-étiquettes fournit une vérification de qualité sans nécessiter une révision complète.

Annotation vidéo

L'annotation vidéo multiplie les défis de l'annotation d'images par le temps :

- **Suivi :** Les objets doivent être identifiés à travers les images. Les annotateurs étiquettent les images clés ; les algorithmes de suivi interpolent entre elles. Les échecs de suivi — occlusion, réentrée, mouvement rapide — nécessitent une ré-étiquetage humain à des taux plus élevés que le suivi en régime permanent.
- **Cohérence temporelle :** Les frontières dessinées à l'image t devraient être spatialement cohérentes avec l'image $t+1$. Les annotations incohérentes sont un signal d'entraînement disant au modèle que les objets se déplacent de manière discontinue — une forme de bruit d'annotation particulièrement dommageable pour les modèles de détection.
- **Évolutivité :** Une vidéo d'une heure à 30 images par seconde représente 108 000 images. L'annotation complète est impraticable ; les stratégies d'échantillonnage doivent être conçues avec soin pour s'assurer que les événements rares (cas limites, quasi-accidents, scénarios de défaillance) ne sont pas systématiquement exclus.

Les outils d'annotation vidéo modernes prennent en charge le **suivi intelligent** qui propage les annotations à travers les images et signale les images où la confiance du suivi tombe en dessous d'un seuil, invitant l'annotateur à revérifier. C'est une application directe de l'idée d'apprentissage actif au pipeline d'annotation lui-même : l'outil interroge l'annotateur exactement là où son interpolation est incertaine.

Le problème des événements rares dans les systèmes autonomes. Pour les applications où les conséquences des événements rares sont catastrophiques — conduite autonome,

navigation par drone — la distribution des images vues en fonctionnement normal est mal assortie à la distribution des images qui importent le plus. Un jeu de données construit par échantillonnage uniforme d'images de conduite contiendra des millions d'images « il ne se passe rien d'intéressant » et une poignée d'images de quasi-accident, d'éclairage inhabituel et de capteur dégradé qui importent réellement pour la sécurité. L'apprentissage actif HITL qui identifie et priorise de telles images n'est pas un hack d'efficacité — c'est une exigence de sécurité.

> **See also:** Jeu de données ImageNet : {cite}`russakovsky2015imagenet`. Biais d'étiquettes dans ImageNet : {cite}`yang2020towards`. Référence COCO : {cite}`lin2014microsoft`. SAM (Segment Anything) : {cite}`kirillov2023segment`. Apprentissage actif par ensemble noyau pour la CV : {cite}`sener2018active`.

L'humain dans la boucle en santé et en sciences

La santé et les sciences représentent deux des domaines où le HITL ML est le plus important et le plus débattu. Les enjeux sont élevés : un cancer manqué ou une cible médicamenteuse erronée a un coût humain réel. L'annotation nécessite une expertise rare et coûteuse. Les exigences réglementaires contraignent ce que les modèles peuvent faire et comment ils doivent être validés. Et contrairement au TAL, où le problème d'annotation est en partie une construction sociale, il existe ici souvent une vérité de terrain réelle — une tumeur est présente ou elle ne l'est pas — même si aucun observateur individuel ne peut la déterminer de manière fiable.

La présentation dominante dans la couverture populaire est « l'IA contre l'humain » : l'IA remplacera-t-elle les radiologues ? Cette présentation est erronée d'une façon qui importe. La vraie question est quelle forme de collaboration humain-IA produit de meilleurs résultats que l'un ou l'autre seul, et comment construire des systèmes qui permettent cette collaboration plutôt que de la perturber.

Analyse d'images médicales

L'imagerie médicale — radiologie (rayons X, scanner, IRM), pathologie (lames de tissu), dermatologie, ophtalmologie — est le domaine où l'IA médicale a progressé le plus rapidement.

Exigences en matière d'annotateurs experts

L'annotation d'images médicales nécessite généralement des médecins avec une formation spécialisée dans une sous-spécialité. Cela rend l'annotation :

- **Lente :** Les spécialistes ont du temps limité ; l'annotation entre en concurrence avec les obligations cliniques
- **Coûteuse :** Les coûts vont de plusieurs dizaines à plusieurs centaines de dollars par cas annoté, selon la sous-spécialité, la modalité et la complexité de la tâche
- **Variable :** Même les spécialistes ne sont pas d'accord, en particulier sur les cas limites — un fait souvent traité comme un problème mais qui est en réalité informatif

Variabilité inter-radiologues

La variabilité entre lecteurs est bien documentée en radiologie. Pour l'interprétation des radiographies thoraciques, le désaccord entre lecteurs est substantiel — dans l'étude CheXNet, quatre radiologues ont étiqueté le même ensemble de test de détection de pneumonie avec des scores F1 couvrant environ 12 points de pourcentage {cite}`rajpurkar2017chexnet`, reflétant

une incertitude diagnostique réelle sur les cas limites. Pour la détection de nodules en scanner pulmonaire, la variabilité intra-lecteur (même lecteur, même cas, jour différent) peut être aussi grande que la variabilité inter-lecteur.

Cette variabilité n'est pas simplement du bruit — elle reflète une incertitude diagnostique réelle. Les modèles entraînés sur les annotations d'un seul radiologue peuvent apprendre les biais spécifiques de ce médecin plutôt que la pathologie sous-jacente.

:::{admonition} La controverse CheXNet comme leçon HITL :class: note

Quand Rajpurkar et al. ont prétendu que leur modèle CheXNet « dépassait les performances des radiologues » en détection de pneumonie, la prétention a immédiatement été contestée par la communauté radiologique {cite}`yu2022assessing`. Une partie de la controverse portait sur l'ensemble de test spécifique et la comparaison avec les radiologues. Mais un problème plus profond était méthodologique : la référence de « performance des radiologues » utilisait des lecteurs individuels sous pression de temps, tandis que la radiologie clinique implique typiquement la consultation, la comparaison avec les images antérieures et l'accès au contexte clinique — rien de tout cela n'était disponible pour le modèle.

La leçon n'est pas que le modèle était bon ou mauvais, mais que **les comparaisons de performance nécessitent de spécifier la configuration HITL**. Un modèle qui surpasse un radiologue individuel lisant à froid peut toujours être moins précis qu'un radiologue utilisant la sortie du modèle comme second avis. Ce sont des systèmes différents avec des modes d'erreur différents, et leur agrégation diffère. :::

:::{admonition} Étiquettes souples en médecine :class: important

Plusieurs projets d'IA médicale ont adopté des **étiquettes souples** qui reflètent la distribution des opinions d'experts plutôt qu'une seule étiquette « de référence ». Une radiographie thoracique étiquetée comme 60 % pneumonie / 40 % atélectasie par un panel de radiologues porte plus d'information qu'un choix binaire forcé. Les modèles entraînés sur de telles distributions montrent une meilleure calibration et une quantification d'incertitude plus appropriée — et cette incertitude est cliniquement significative, car elle dit au clinicien quand consulter, pas seulement ce que le modèle pense. :::

Apprentissage actif pour les maladies rares

L'apprentissage actif est particulièrement précieux pour les maladies rares et les pathologies rares, où même un grand pool non étiqueté ne contient qu'un petit nombre de cas positifs. L'échantillonnage aléatoire standard gaspillerait le temps des experts à étiqueter principalement des cas négatifs.

L'apprentissage actif basé sur l'incertitude sélectionne naturellement les cas limites où le modèle est incertain — qui, pour les conditions rares, tendent à être les cas positifs et les négatifs limites. Cela concentre le temps des spécialistes là où il est le plus précieux. La combinaison de l'entraînement déséquilibré en classes (avec `class_weight='balanced'` ou similaire) et de la sélection basée sur l'incertitude est une pratique standard pour les tâches de détection de pathologies rares.

Annotation de TAL clinique

Les dossiers médicaux électroniques (DME) contiennent une richesse énorme de textes narratifs cliniques : notes des médecins, résumés de sortie, rapports radiologiques, rapports

anatomopathologiques. L'extraction d'informations structurées à partir de ces textes nécessite le TAL — et un TAL de haute qualité nécessite des données d'entraînement annotées.

Tâches d'annotation de TAL clinique courantes : - NER clinique : Identifier les médicaments, les dosages, les diagnostics, les procédures et les symptômes dans le texte - **Détection de négation :** « Aucune preuve de pneumonie » vs. « Pneumonie confirmée » — une distinction critique qui est étonnamment difficile - **Raisonnement temporel :** Distinguer les conditions actuelles des antécédents (« antécédents d'IM, présenté avec douleur thoracique ») - **Dé-identification :** Supprimer les Informations de Santé Protégées (ISP) pour permettre le partage des données

La dé-identification des ISP est à la fois une tâche d'annotation et une exigence de gouvernance des données. En vertu de HIPAA (États-Unis) et du RGPD (UE), les données de santé ne peuvent pas être partagées sans suppression ou anonymisation des identifiants de patients. Des outils de dé-identification automatisés existent mais sont imparfaits ; la révision humaine des sorties automatisées est une pratique standard, et le profil de risque est asymétrique : les faux négatifs (ISP manquées) créent une exposition légale, rendant des seuils conservateurs nécessaires.

i2b2 / n2c2 comme modèle

Les initiatives de tâches partagées i2b2 (Informatics for Integrating Biology and the Bedside) et leurs successeurs n2c2 (National NLP Clinical Challenges) ont publié une série de jeux de données de TAL clinique annotés par des experts. Ceux-ci illustrent à la fois le potentiel et le coût : les efforts d'annotation impliquent généralement des équipes d'experts du domaine clinique travaillant sur plusieurs mois, annotant des centaines de documents par défi. Les jeux de données n2c2 ont catalysé des progrès rapides précisément parce qu'ils ont résolu le problème de gouvernance du partage de données (dé-identification + accords institutionnels), pas seulement le problème d'annotation.

Considérations réglementaires

L'IA médicale est soumise à une surveillance réglementaire dans la plupart des juridictions.

FDA (États-Unis) : Les logiciels comme dispositif médical (SaMD) basés sur l'IA/ML nécessitent une approbation ou une autorisation préalable à la mise sur le marché. Le Plan d'action IA/ML de la FDA de 2021 met l'accent sur des **plans de contrôle des changements prédéterminés** — documentant comment le modèle sera mis à jour et comment ces mises à jour seront validées avant le déploiement. Un modèle qui apprend en continu à partir du retour clinique est, dans ce cadre, un dispositif différent après chaque mise à jour et peut nécessiter une revalidation.

Marquage CE (Europe) : Les dispositifs médicaux incluant les systèmes d'IA doivent se conformer au Règlement sur les Dispositifs Médicaux (MDR). Le MDR exige une évaluation clinique, une surveillance post-commercialisation et la documentation des données utilisées pour l'entraînement et la validation.

Implication HITL clé : Les cadres réglementaires exigent une documentation claire des processus d'annotation, des qualifications des annotateurs, de la fiabilité inter-évaluateurs et de tout changement aux données d'entraînement. Ce n'est pas de la paperasserie — c'est la piste d'audit qui permet à un clinicien de comprendre quelles données d'entraînement ont produit le comportement actuel du modèle, et c'est légalement requis. Les pipelines HITL qui

traitent l'annotation comme un sous-processus informel créent un risque réglementaire qui devient généralement visible au pire moment.

Annotation de données scientifiques

Au-delà de la santé, le HITL ML joue un rôle croissant et sous-estimé dans la recherche scientifique, où le défi d'annotation mêle souvent expertise du domaine et échelle.

Astronomie : Galaxy Zoo

Galaxy Zoo {cite}`lintott2008galaxy` a externalisé la classification morphologique des galaxies du Sloan Digital Sky Survey à des scientifiques citoyens. Le projet original a collecté plus de 40 millions de classifications de plus de 100 000 bénévoles, démontrant que la classification d'images scientifiques à grande échelle est réalisable quand la tâche peut être décomposée en questions simples répondables sans formation spécialisée (« Cette galaxie est-elle lisse ou présente-t-elle des structures ? »).

L'expérience Galaxy Zoo a produit deux résultats méthodologiques importants. Premièrement, l'accord entre scientifiques citoyens et astronomes professionnels était élevé pour les cas clairs et divergeait systématiquement sur les cas limites — précisément les cas où la distinction importe scientifiquement. La solution n'était pas d'écarter les données des sciences citoyennes sur les cas limites mais de traiter la distribution des réponses des bénévoles comme une étiquette souple encodant une ambiguïté morphologique réelle. Deuxièmement, le classificateur entraîné sur les étiquettes de Galaxy Zoo surpassait les modèles entraînés pour reproduire les étiquettes de n'importe quel expert unique, parce que la distribution de la foule capturait une réelle incertitude visuelle que le choix forcé d'un expert unique effondrait.

Génomique : Classification de la pathogénicité

L'annotation des variants génomiques — décider si un variant est pathogène, bénin ou de signification incertaine — est un problème à enjeux élevés de TAL et de jugement d'expert. Les bases de données cliniques de variants comme ClinVar agrègent des interprétations d'experts de plusieurs laboratoires soumetteurs, et le désaccord entre laboratoires est courant. L'apprentissage actif est utilisé pour prioriser quels variants nécessitent une révision experte complète (recherche bibliographique, évaluation des preuves fonctionnelles) par rapport à ceux pouvant être auto-classifiés par des preuves existantes. Le résultat est un pipeline hybride où la plupart des variants sont gérés par une logique automatisée, un sous-ensemble nécessite une révision experte, et les cas les plus difficiles sont signalés pour un consensus multi-laboratoires.

Climat et sciences de la Terre

L'étiquetage d'images satellites pour le changement d'utilisation des terres, la déforestation, l'étendue des glaciers et les trajectoires des tempêtes implique des experts en télédétection et, de plus en plus, des plateformes de sciences citoyennes. Le principal défi HITL dans ce domaine est temporel : les étiquettes faites aujourd'hui peuvent devenir obsolètes à mesure que le monde change, et la vérification de la vérité terrain (enquêtes de terrain) est coûteuse et logistiquement contrainte. L'apprentissage actif qui priorise les images où la prédiction du modèle est en désaccord avec des a priori physiques (par exemple, prédire la déforestation dans une région connue pour être protégée) est une façon pratique de diriger de rares ressources de vérification de terrain.

Neurosciences : La connectomique

La reconstruction des circuits neuronaux à partir d'images de microscopie électronique — la connectomique — nécessite une annotation au niveau des pixels des membranes neuronales à travers d'énormes piles d'images. Le projet Eyewire a gamifié cette tâche, engageant des dizaines de milliers de joueurs dans le traçage de neurones à travers des volumes d'images 3D. La conception de la gamification a résolu un problème HITL spécifique : la tâche nécessite une attention soutenue et un raisonnement spatial sur de longues sessions, ce qui entraîne une dégradation de la qualité dans l'annotation traditionnelle. Décomposer la tâche en segments de jeu avec des mécanismes sociaux a maintenu l'engagement et la qualité des annotateurs à des échelles que l'annotation professionnelle ne peut pas atteindre.

Gérer les annotateurs spécialistes

Quand l'annotation nécessite une expertise rare, les approches habituelles de production participative (Chapitre 13) ne s'appliquent pas.

La tension fondamentale est que les personnes capables de produire les annotations de la plus haute qualité sont aussi celles dont le temps est le plus précieux et le plus contraint. Chaque décision de conception dans un pipeline d'annotation spécialisée devrait être évaluée par rapport à la question : est-ce que cela fait le meilleur usage du temps rare des experts ?

Ce que cela signifie en pratique :

- **Pré-annoter agressivement.** Utiliser des annotateurs de niveau inférieur, des modèles automatisés ou des systèmes basés sur des règles pour générer des candidats que le spécialiste révise et corrige plutôt que de créer à partir de zéro. Le jugement du spécialiste est le goulot d'étranglement ; lui fournir une pré-annotation à corriger est plus rapide que de lui demander d'annoter à partir d'un écran vide, à condition que la qualité de la pré-annotation soit suffisante pour que la correction ne soit pas plus lente que de recommencer.

- **Concevoir pour l'attention des experts, non le débit.** Les interfaces d'annotation optimisées pour un débit élevé (décisions binaires rapides, raccourcis clavier, affichage minimal) sont appropriées pour la production participative. L'annotation spécialisée bénéficie souvent d'interfaces plus riches : comparaison côte à côte avec des cas antérieurs, accès facile aux matériaux de référence, champs de confiance d'annotation, et possibilité de signaler un cas pour discussion. Cela ralentit les annotations individuelles mais améliore la qualité et réduit le besoin de ré-annotation.

- **Suivre explicitement les schémas d'annotateurs individuels.** Avec un petit pool de spécialistes, il est faisable et important de suivre le taux d'accord de chaque annotateur avec le panel, de signaler les cas qui semblent incohérents avec leur propre historique et d'en discuter lors de sessions de calibration régulières. Ce n'est pas de la surveillance — c'est le même processus de qualité que la médecine clinique utilise pour les évaluations de performance, et les spécialistes répondent généralement bien quand il est présenté comme une amélioration qualitative partagée plutôt que comme une évaluation.

- **La conception de session est importante.** L'annotation médicale est cognitivement exigeante. Les preuves de la radiologie et de la pathologie suggèrent que les taux d'erreur augmentent de manière mesurable après environ 90 minutes de lecture continue, et que des pauses de quelques minutes seulement peuvent partiellement restaurer l'attention. Les interfaces d'annotation qui imposent des invites de pause (sans

possibilité de les ignorer) sont une décision de conception HITL simple avec un impact réel sur la qualité.

Un pipeline HITL d'apprentissage actif pour l'imagerie médicale

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

# Simulate a rare-pathology detection task
# 8% positive class (e.g., rare pathology)
X, y = make_classification(
    n_samples=5000, n_features=100,
    n_informative=20, n_redundant=10,
    weights=[0.92, 0.08],
    random_state=42
)
X_train, y_train = X[:4000], y[:4000]
X_test,  y_test  = X[4000:], y[4000:]

print(f"Training set positive prevalence: {y_train.mean():.1%}")

def run_medical_al(strategy, n_initial=50, budget=300):
    labeled = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]
    aucs = []

    while len(labeled) < n_initial + budget:
        model = LogisticRegression(max_iter=500, class_weight='balanced')
        model.fit(X_train[labeled], y_train[labeled])

        if len(labeled) % 30 == 0:
            preds = model.predict_proba(X_test)[:, 1]
            aucs.append(roc_auc_score(y_test, preds))

        X_pool = X_train[unlabeled]
        if strategy == 'uncertainty' and len(labeled) >= 10:
            probs = model.predict_proba(X_pool)
            entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
            q = int(np.argmax(entropy))
        else:
            q = rng.integers(0, len(unlabeled))

        labeled.append(unlabeled.pop(q))
```

```
    return np.array(aucs)

aucs_al  = run_medical_al('uncertainty')
aucs_rnd = run_medical_al('random')
label_counts = np.arange(len(aucs_al)) * 30 + 50

plt.figure(figsize=(7, 4))
plt.plot(label_counts, aucs_al,  'o-',  color='#2b3a8f', linewidth=2, label='Uncertainty AL')
plt.plot(label_counts, aucs_rnd, 's--', color='#e05c5c', linewidth=2, label='Random baseline'
plt.xlabel("Expert labels obtained", fontsize=12)
plt.ylabel("AUROC", fontsize=12)
plt.title("Active Learning for Rare Pathology Detection", fontsize=13)
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('medical_al.png', dpi=150)
plt.show()

# Show how AL preferentially selects positive examples
# by checking which class the queried examples belong to
n_init = 50
labeled_al  = list(rng.choice(len(X_train), n_init, replace=False))
labeled_rnd = labeled_al.copy()
unlabeled_al  = [i for i in range(len(X_train)) if i not in labeled_al]
unlabeled_rnd = unlabeled_al.copy()

model = LogisticRegression(max_iter=500, class_weight='balanced')
model.fit(X_train[labeled_al], y_train[labeled_al])
probs = model.predict_proba(X_train[unlabeled_al])
entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
top50_al  = [unlabeled_al[i] for i in np.argsort(entropy)[-50:]]
top50_rnd = list(rng.choice(unlabeled_rnd, 50, replace=False))

pos_rate_al  = y_train[top50_al].mean()
pos_rate_rnd = y_train[top50_rnd].mean()
print(f"\nPositive rate in next 50 queries:")
print(f"  Uncertainty AL: {pos_rate_al:.1%}  (base rate: {y_train.mean():.1%})")
print(f"  Random:         {pos_rate_rnd:.1%}")
print(f"  AL queries {pos_rate_al/y_train.mean():.1f}x more positives than random")
```

See also: Production participative Galaxy Zoo : {cite}`lintott2008galaxy`. Performances des radiologues CheXNet : {cite}`rajpurkar2017chexnet`. Qualité des radiographies et diagnostic assisté par IA : {cite}`yu2022assessing`. Méthodologie d'annotation de TAL clinique : {cite}`pustejovsky2012natural`. Pour le plan d'action IA/ML de la FDA, consulter la documentation publiée par la FDA (2021).

Plateformes et outils d'annotation

La plateforme d'annotation est l'environnement où le retour humain devient des données. Une bonne plateforme augmente le débit, réduit les erreurs, maintient les contrôles de qualité et rend le pipeline d'annotation gérable à grande échelle. Choisir la bonne plateforme — et savoir quand construire plutôt qu'acheter — est une décision importante dans tout projet HITL.

Le paysage des plateformes d'annotation

Le marché des outils d'annotation a considérablement grandi et mûri ces dernières années, porté par la demande des entreprises en données d'entraînement pour l'apprentissage automatique. Les outils vont des services entièrement gérés aux frameworks open source auto-hébergés.

Plateformes open source

Label Studio est la plateforme d'annotation open source la plus populaire, prenant en charge les données textuelles, images, audio, vidéo et séries temporelles via une configuration de tâche unifiée basée sur XML. Elle peut être auto-hébergée et s'intègre avec des backends d'apprentissage automatique pour l'apprentissage actif. Points forts : flexibilité, soutien communautaire et possibilité d'intégrer des prédictions ML pour la pré-annotation.

Prodigy (des créateurs de spaCy) est un outil d'annotation hautement orienté flux de travail conçu pour les tâches TAL. Son architecture en streaming envoie des exemples un par un et prend en charge l'apprentissage actif nativement. L'interface d'annotation est minimaliste mais rapide — conçue pour maximiser le débit. Prodigy est un logiciel payant mais largement utilisé dans la recherche en TAL.

CVAT (Computer Vision Annotation Tool) est le principal outil open source pour l'annotation en CV, avec un fort support pour la détection, la segmentation et l'annotation vidéo. Développé à l'origine chez Intel, CVAT prend en charge les algorithmes de suivi, l'annotation de squelettes et les intégrations d'algorithmes tiers.

Doccano cible les tâches d'étiquetage de séquences (NER, extraction de relations, classification de texte). Son interface web simple le rend accessible aux équipes sans ressources dédiées en ingénierie de données.

Plateformes commerciales

Scale AI fournit des services d'annotation gérés de bout en bout : main-d'œuvre humaine, gestion de la qualité et intégration API. Particulièrement forte pour la conduite autonome, la robotique et l'annotation 3D complexe. La tarification est basée sur la complexité et le volume de la tâche.

Labelbox est une plateforme complète pour la curation de données, l'annotation et l'étiquetage assisté par apprentissage automatique. Fonctionnalités d'entreprise robustes : gestion de projets, flux de travail qualité, boucles de retour du modèle et intégrations avec les principales plateformes d'apprentissage automatique (SageMaker, Vertex AI, Azure ML).

Appen (anciennement Figure Eight / CrowdFlower) opère une grande main-d'œuvre d'annotation mondiale aux côtés d'outils. Un bon choix quand le volume et la gestion de la main-d'œuvre sont les principales préoccupations.

Surge AI se concentre sur les annotateurs experts et est fort pour les tâches nécessitant une connaissance du domaine ou un jugement nuancé.

Humanloop se spécialise dans la collecte de retours pour les LLM — annotation de préférences, collecte de données RLHF et évaluation des modèles.

Comparaison des fonctionnalités des plateformes d'annotation

Fonctionnalité	Label Studio	Prodigy	CVAT	Labelbox	Scale AI
Licence	Open source	Commercial	Open source	Commercial	Commercial
Hébergement	Auto / cloud	Auto	Auto / cloud	Cloud	Géré
Annotation de texte	□	□	—	□	□
Annotation d'images	□	Limité	□	□	□
Annotation vidéo	□	—	□	□	□
Intégration apprentissage actif	□	□	Limité	□	□
Pré-annotation assistée par ML	□	□	□	□	□
Flux de travail contrôle qualité	Basique	Basique	Basique	Avancé	Avancé
Accès API / programmatique	□	□	□	□	□
Gestion de la main-d'œuvre	—	—	—	Limité	□

L'annotation comme code

Un aspect critique mais souvent négligé de l'infrastructure d'annotation est le **contrôle de version pour les annotations et les schémas d'annotation**. Traiter l'annotation comme du code signifie :

Conception schéma d'abord. La taxonomie des étiquettes et les règles d'annotation sont définies dans un fichier de configuration versionné (YAML ou JSON) avant le début de l'annotation. Les modifications du schéma créent une nouvelle version.

Versionnage des annotations. Les annotations sont stockées avec un lien vers la version du schéma sous laquelle elles ont été créées. Cela permet l'audit, la restauration et la comparaison des annotations entre les versions du schéma.

Pipelines reproductibles. Le pipeline d'annotation — des données brutes aux étiquettes prêtes à l'entraînement — devrait être reproductible à partir du code. Les modèles de pré-annotation, les filtres de qualité, la logique d'agrégation et les divisions de données devraient tous être enregistrés.

```
# Example: Label Studio annotation schema (text classification)
label_config: |
  <View>
    <Text name="text" value="$text"/>
    <Choices name="sentiment" toName="text" choice="single">
      <Choice value="positive"/>
      <Choice value="negative"/>
      <Choice value="neutral"/>
      <Choice value="mixed"/>
    </Choices>
  </View>
schema_version: "2.1.0"
task_type: text_classification
guidelines_version: "guidelines_v3.pdf"
```text
Source de données
    |
    v
Pool non étiqueté -->  Plateforme d'annotation --> Jeu de données étiqueté
    ^                        |                              |
    |                        | (assisté par ML)             v
Apprentissage actif <--------+                        Cycle d'entraînement
stratégie de requête                                       |
    ^                                                      v
    +---------- Modèle entraîné <---------------------- Évaluation
                                                           |
                                                      Déploiement &
                                                      surveillance
```
```

Points d'intégration clés : 1. **Ingestion de données :** Les données non étiquetées circulent automatiquement de l'entrepôt de données vers la plateforme d'annotation 2. **Service du modèle :** Le meilleur modèle actuel est déployé sur la plateforme d'annotation pour la pré-annotation et l'évaluation de l'apprentissage actif 3. **Export :** Les annotations complètes sont exportées dans un format compatible avec le framework d'entraînement (JSON COCO, jeux de données Hugging Face, etc.) 4. **Boucle de retour :** Les erreurs du modèle en production sont acheminées vers la plateforme d'annotation pour correction

> **See also:** Pour la documentation Label Studio et l'intégration de l'apprentissage actif : https://labelstud.io. Pour Prodigy : https://prodi.gy. CVAT : https://cvat.ai. Pour une comparaison complète des outils d'annotation, voir {cite}`monarch2021human`, Chapitre 7.

Production participative et contrôle qualité

Quand les tâches d'annotation sont suffisamment simples pour être réalisées par des non-experts, les plateformes de production participative offrent l'accès à de grandes main-d'œuvres d'annotation à la demande à faible coût par élément. Construire des jeux de données étiquetés de haute qualité à partir des foules nécessite une conception soigneuse des tâches, une redondance stratégique et un contrôle qualité rigoureux.

Plateformes de production participative

Amazon Mechanical Turk (MTurk) est la place de marché de production participative originale, lancée en 2005. Les travailleurs (« Turkers ») accomplissent des micro-tâches (HITs) publiées par les demandeurs. Une étude de 2018 a trouvé un revenu horaire effectif médian pour les Turkers d'environ 2 $/h — bien en dessous du salaire minimum dans de nombreux pays à revenu élevé {cite}`hara2018data` — une préoccupation éthique abordée au Chapitre 15. MTurk convient mieux aux tâches simples avec des critères clairs et vérifiables.

Prolific est une plateforme de production participative académique qui applique un standard de rémunération minimum (actuellement environ 9 £/h, soit approximativement 11 $/h, tel qu'indiqué dans les directives publiées par Prolific), filtre les participants par données démographiques et maintient un panel de travailleurs qui ont opté pour la participation à la recherche. Préféré pour la recherche en sciences sociales et les tâches nécessitant une représentativité.

Appen (et similaires : Telus International, iMerit) fournit des main-d'œuvres d'annotation gérées avec gestion de la qualité, utilisées pour des tâches plus complexes et des projets d'entreprise.

Communautés spécialisées. Pour des tâches spécifiques au domaine, des communautés de passionnés du domaine peuvent fournir des annotations de haute qualité : Galaxy Zoo pour l'astronomie, eBird pour les espèces d'oiseaux, Chess Tempo pour l'annotation de positions d'échecs.

Conception de tâches pour la production participative

Décomposer les tâches complexes

Les tâches complexes devraient être décomposées en micro-tâches simples et bien définies. Au lieu de demander aux travailleurs d'annoter complètement un document, posez-leur une question ciblée à la fois : « Cette phrase contient-elle un nom de personne ? » ou « Évaluez la clarté de cette traduction sur une échelle de 1 à 5. »

Avantages de la décomposition : - Demande cognitive moindre par tâche □ moins de fatigue, meilleure qualité - Chaque micro-tâche peut faire l'objet d'un contrôle qualité séparé - Plus facile à auditer et déboguer

L'importance des instructions

Le principal prédicteur de la qualité en production participative est la qualité des instructions. De bonnes instructions de tâche : - Expliquent le *but* de la tâche en une phrase - Donnent une définition claire et non ambiguë de chaque catégorie - Fournissent 3 à 5 exemples travaillés (en particulier les cas limites) - Sont assez courtes pour que les travailleurs les lisent réellement (< 300 mots pour les tâches simples)

Réalisez une **étude pilote** (10 à 50 travailleurs, 20 à 100 tâches) avant de passer à l'échelle. Analysez les désaccords du pilote ; la plupart pointent vers des ambiguïtés dans les instructions qui peuvent être corrigées.

Questions de référence

Intégrez des **questions de référence** — des tâches avec des réponses correctes connues — tout au long du lot de tâches. Les travailleurs qui échouent aux questions de référence en dessous d'un seuil sont retirés du projet.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(42)

def simulate_gold_screening(n_workers=100, gold_per_batch=5,
                            p_good_worker=0.7, good_acc=0.92,
                            bad_acc=0.55, threshold=0.70):
    """
    Simulate quality screening via gold questions.
    Returns: precision and recall of identifying bad workers.
    """
    worker_quality = rng.choice(['good', 'bad'], size=n_workers,
                                p=[p_good_worker, 1 - p_good_worker])
    results = {'tp': 0, 'fp': 0, 'tn': 0, 'fn': 0}

    for q in worker_quality:
        acc = good_acc if q == 'good' else bad_acc
        n_correct = rng.binomial(gold_per_batch, acc)
        passed = (n_correct / gold_per_batch) >= threshold
        if q == 'bad' and not passed:  results['tp'] += 1
        if q == 'good' and not passed: results['fp'] += 1
        if q == 'bad' and passed:      results['fn'] += 1
```

```
        if q == 'good' and passed:     results['tn'] += 1

    tp, fp, fn = results['tp'], results['fp'], results['fn']
    precision = tp / (tp + fp + 1e-6)
    recall    = tp / (tp + fn + 1e-6)
    return precision, recall

# Vary gold question count
gold_counts = [3, 5, 8, 10, 15, 20]
precisions, recalls = [], []
for g in gold_counts:
    p_list, r_list = [], []
    for _ in range(50):
        p, r = simulate_gold_screening(gold_per_batch=g)
        p_list.append(p); r_list.append(r)
    precisions.append(np.mean(p_list))
    recalls.append(np.mean(r_list))

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(gold_counts, precisions, 'o-', color='#2b3a8f', label='Precision', linewidth=2)
ax.plot(gold_counts, recalls,    's--', color='#0d9e8e', label='Recall',    linewidth=2)
ax.set_xlabel("Gold questions per batch", fontsize=12)
ax.set_ylabel("Screening performance", fontsize=12)
ax.set_title("Worker Screening via Gold Standard Questions", fontsize=13)
ax.legend(fontsize=11); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('gold_screening.png', dpi=150)
plt.show()

p, r = simulate_gold_screening(gold_per_batch=5)
print(f"5 gold questions: Precision={p:.3f}, Recall={r:.3f}")
p, r = simulate_gold_screening(gold_per_batch=10)
print(f"10 gold questions: Precision={p:.3f}, Recall={r:.3f}")
```

Modèles statistiques pour l'agrégation des étiquettes

Le vote majoritaire est une référence naturelle mais ignore les différences de précision entre annotateurs. Les modèles statistiques peuvent faire mieux.

Le modèle de Dawid-Skene

Le **modèle de Dawid-Skene (DS)** {cite}`dawid1979maximum` estime conjointement : - La **vraie étiquette** z_i pour chaque élément i - La **matrice de confusion** $\pi_j^{(k,l)}$ pour chaque annotateur j : la probabilité que l'annotateur j étiquette un élément avec la vraie classe k comme classe l

L'algorithme EM itère : - **Étape E :** Étant donné les matrices de confusion des annotateurs, calculer la probabilité a posteriori de chaque vraie étiquette - **Étape M :** Étant donné les estimations des étiquettes des éléments, mettre à jour les matrices de confusion des annotateurs

```python
import numpy as np
from scipy.special import softmax

def dawid_skene_em(annotations, n_classes, n_iter=20):
    """
    Simplified Dawid-Skene EM for binary classification.
    annotations: dict {item_idx: [(annotator_idx, label), ...]}
    Returns: estimated true labels and annotator accuracies.
    """
    items = sorted(annotations.keys())
    n_items = len(items)
    annotators = sorted({a for anns in annotations.values() for a, _ in anns})
    n_annotators = len(annotators)
    ann_idx = {a: i for i, a in enumerate(annotators)}

    # Initialize: majority vote
    T = np.zeros((n_items, n_classes))  # soft label estimates
    for i, item in enumerate(items):
        for _, label in annotations[item]:
            T[i, label] += 1
        T[i] /= T[i].sum()

    # Confusion matrices: shape (n_annotators, n_classes, n_classes)
    PI = np.ones((n_annotators, n_classes, n_classes)) * 0.5

    for _ in range(n_iter):
        # M-step: update confusion matrices
        PI = np.zeros((n_annotators, n_classes, n_classes)) + 1e-6
        for i, item in enumerate(items):
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                for k in range(n_classes):
                    PI[a, k, label] += T[i, k]
        PI /= PI.sum(axis=2, keepdims=True)

        # E-step: update soft label estimates
        T = np.zeros((n_items, n_classes))
        for i, item in enumerate(items):
            log_p = np.zeros(n_classes)
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                log_p += np.log(PI[a, :, label] + 1e-10)
            T[i] = softmax(log_p)

    return {item: T[i] for i, item in enumerate(items)}, PI

# Simulate crowdsourcing scenario
rng = np.random.default_rng(42)
N_ITEMS, N_ANNOTATORS, ANNS_PER_ITEM = 200, 10, 3
true_labels = rng.integers(0, 2, N_ITEMS)
# Annotator accuracies: 3 good (0.9), 4 medium (0.75), 3 noisy (0.6)
```

```
accuracies = [0.90]*3 + [0.75]*4 + [0.60]*3

annotations = {}
for i in range(N_ITEMS):
    anns_for_item = []
    chosen = rng.choice(N_ANNOTATORS, ANNS_PER_ITEM, replace=False)
    for a in chosen:
        acc = accuracies[a]
        label = true_labels[i] if rng.random() < acc else 1 - true_labels[i]
        anns_for_item.append((a, int(label)))
    annotations[i] = anns_for_item

# Run Dawid-Skene
soft_labels, confusion = dawid_skene_em(annotations, n_classes=2, n_iter=30)
ds_preds = {i: int(np.argmax(soft_labels[i])) for i in range(N_ITEMS)}

# Compare with majority vote
mv_preds = {}
for i in range(N_ITEMS):
    votes = [l for _, l in annotations[i]]
    mv_preds[i] = int(np.round(np.mean(votes)))

ds_acc = np.mean([ds_preds[i] == true_labels[i] for i in range(N_ITEMS)])
mv_acc = np.mean([mv_preds[i] == true_labels[i] for i in range(N_ITEMS)])

print(f"Majority vote accuracy:  {mv_acc:.3f}")
print(f"Dawid-Skene accuracy:    {ds_acc:.3f}")
print(f"\nEstimated annotator accuracies (diagonal of confusion matrix):")
for a in range(N_ANNOTATORS):
    est_acc = np.mean([confusion[a, k, k] for k in range(2)])
    print(f"  Annotator {a}: estimated={est_acc:.3f}, true={accuracies[a]:.2f}")
```

MACE

MACE (Estimation de la Compétence Multi-Annotateurs) {cite}`hovy2013learning` est un modèle probabiliste alternatif qui représente explicitement le comportement de spam (étiquetage aléatoire) des annotateurs par opposition à l'annotation compétente. Un annotateur fournit soit une étiquette significative (avec probabilité $1 - \text{spam}_j$) soit une étiquette aléatoire (avec probabilité spam_j). Ce modèle à deux composantes est souvent mieux calibré que Dawid-Skene pour les scénarios de production participative où certains annotateurs sont de purs spammeurs.

Redondance et stratégie d'agrégation

Le nombre optimal d'annotateurs par élément dépend de la difficulté de la tâche et de la qualité des annotateurs :

- **Tâches faciles avec des annotateurs qualifiés :** 1 à 2 annotateurs par élément est souvent suffisant
- **Tâches modérées avec des annotateurs formés :** 3 annotateurs + vote majoritaire

- **Tâches difficiles/subjectives avec des travailleurs de production participative :** 5 à 7 annotateurs + Dawid-Skene

L'intuition clé : la redondance est la plus précieuse quand la précision des annotateurs est faible. Pour des annotateurs de précision p, la précision du vote majoritaire avec n annotateurs est :

$$P(\text{VM correct}) = \sum_{k>n/2}^{n} \binom{n}{k} p^k (1-p)^{n-k}$$

Pour $p = 0{,}70$, ajouter un troisième annotateur augmente la précision du vote majoritaire de 70 % à 78 % ; pour $p = 0{,}90$, le gain d'un troisième annotateur est négligeable (de 90 % à 97 %).

> **See also:** Modèle de Dawid-Skene : {cite}`dawid1979maximum`. MACE : {cite}`hovy2013learning`. Pour une revue complète de la production participative pour le TAL : {cite}`snow2008cheap`. Éthique de la foule et rémunération équitable : voir le Chapitre 15.

Évaluation et métriques

Savoir si votre système HITL fonctionne nécessite plus que la mesure de la précision du modèle. Vous devez savoir si vous tirez de la valeur de votre budget d'annotation, si le modèle est réellement mieux aligné sur l'intention humaine, et si un retour humain supplémentaire continuera à améliorer les choses. Ce chapitre couvre le paysage complet de l'évaluation dans les contextes HITL.

Métriques centrées sur le modèle

Les métriques d'apprentissage automatique standard s'appliquent directement aux systèmes HITL, avec quelques nuances importantes.

Métriques de classification

La précision est appropriée quand les classes sont équilibrées et que toutes les erreurs coûtent également. Dans les contextes HITL, cependant, l'ensemble de test étiqueté peut être biaisé par la stratégie de requête (l'apprentissage actif interroge des exemples non aléatoires), rendant les estimations de précision simples peu fiables.

Le score F1 est la moyenne harmonique de la précision et du rappel, appropriée pour les classes déséquilibrées. Dans les contextes HITL, la précision et le rappel peuvent importer différemment selon l'asymétrie du coût entre les faux positifs et les faux négatifs.

L'AUROC mesure la capacité du modèle à discriminer entre classes indépendamment du seuil — important pour les tâches sensibles à la calibration comme le dépistage médical.

La calibration mesure dans quelle mesure les probabilités prédites correspondent aux fréquences empiriques. Dans les systèmes HITL, les modèles entraînés sur des ensembles étiquetés biaisés (issus de l'apprentissage actif) peuvent être mal calibrés même quand ils sont précis.

Métriques pour les modèles génératifs

Pour les modèles de langage et les systèmes génératifs, l'évaluation est fondamentalement plus difficile. Aucune métrique automatique unique ne capture la qualité :

- **BLEU / ROUGE / METEOR :** Métriques basées sur des références pour la traduction et la résumé. Corrèlent faiblement avec les jugements humains de qualité pour la génération de texte long.
- **Perplexité :** Mesure dans quelle mesure le modèle prédit le texte mis de côté. Condition nécessaire mais non suffisante pour la qualité.

- **BERTScore :** Similarité basée sur les plongements par rapport aux références. Mieux corrélée avec les jugements humains que les métriques n-grammes.
- **Évaluation humaine :** L'étalon-or. Voir la Section 14.3.

Métriques d'efficacité de l'annotation

L'évaluation HITL devrait également mesurer si le retour humain est utilisé efficacement.

Courbes d'apprentissage

Une **courbe d'apprentissage** trace les performances du modèle en fonction du nombre d'exemples étiquetés. Une courbe d'apprentissage raide (amélioration rapide avec peu d'étiquettes) indique que la stratégie d'annotation sélectionne des exemples informatifs. Une courbe d'apprentissage plate indique que l'étiquetage supplémentaire fournit des rendements décroissants.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.model_selection import StratifiedShuffleSplit

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=5000, n_features=30, n_informative=15,
                           weights=[0.8, 0.2], random_state=42)
X_test, y_test = X[4000:], y[4000:]
X_pool, y_pool = X[:4000], y[:4000]

label_sizes = [20, 40, 60, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000]
metrics = {'f1': [], 'auc': []}

for n in label_sizes:
    idx = rng.choice(len(X_pool), n, replace=False)
    clf = LogisticRegression(max_iter=500, class_weight='balanced')
    clf.fit(X_pool[idx], y_pool[idx])
    preds = clf.predict(X_test)
    probs = clf.predict_proba(X_test)[:, 1]
    metrics['f1'].append(f1_score(y_test, preds))
    metrics['auc'].append(roc_auc_score(y_test, probs))

# Fit learning curve: performance ≈ a - b/sqrt(n)
from scipy.optimize import curve_fit

def learning_curve_fn(n, a, b):
    return a - b / np.sqrt(n)

popt_f1, _ = curve_fit(learning_curve_fn, label_sizes, metrics['f1'], p0=[0.9, 2])
popt_auc, _ = curve_fit(learning_curve_fn, label_sizes, metrics['auc'], p0=[0.95, 1])
```

```
n_smooth = np.linspace(20, 3000, 200)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.scatter(label_sizes, metrics['f1'], color='#2b3a8f', zorder=5, s=40)
ax1.plot(n_smooth, learning_curve_fn(n_smooth, *popt_f1), '--', color='#e05c5c',
         label=f'Fit: {popt_f1[0]:.3f} - {popt_f1[1]:.1f}/√n')
ax1.set_xlabel("Labeled examples"); ax1.set_ylabel("F1 score")
ax1.set_title("Learning Curve: F1"); ax1.legend(); ax1.grid(alpha=0.3)

ax2.scatter(label_sizes, metrics['auc'], color='#0d9e8e', zorder=5, s=40)
ax2.plot(n_smooth, learning_curve_fn(n_smooth, *popt_auc), '--', color='#e05c5c',
         label=f'Fit: {popt_auc[0]:.3f} - {popt_auc[1]:.2f}/√n')
ax2.set_xlabel("Labeled examples"); ax2.set_ylabel("AUROC")
ax2.set_title("Learning Curve: AUROC"); ax2.legend(); ax2.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('learning_curves.png', dpi=150)
plt.show()

# Estimate the annotation budget needed to reach a target performance
target_f1 = 0.80
n_needed = (popt_f1[1] / (popt_f1[0] - target_f1)) ** 2
print(f"Estimated labels needed to reach F1={target_f1}: {n_needed:.0f}")
```

Analyse du retour sur investissement (ROI)

Le ROI du retour humain répond à la question : pour chaque étiquette supplémentaire, dans quelle mesure les performances du modèle s'améliorent-elles ?

$$\text{ROI}(n) = \frac{\Delta\text{performance}(n)}{\text{coût par étiquette}}$$

À mesure qu'un modèle mûrit (et que les exemples faciles à apprendre sont épuisés), le ROI diminue typiquement. L'implication pratique : les budgets d'annotation devraient être concentrés en début de projet, avec davantage d'étiquettes collectées aux premières étapes quand le ROI est le plus élevé.

Évaluation humaine

Pour les systèmes génératifs et les tâches subjectives, l'évaluation humaine reste l'étalon-or.

Évaluation directe (DA)

Les annotateurs évaluent les sorties sur une échelle absolue (par exemple, 1 à 100 pour la qualité de traduction, ou 1 à 5 pour l'utilité de la réponse). La DA a été standardisée dans l'évaluation de la traduction automatique (références WMT).

Meilleures pratiques pour la DA : - Randomiser l'ordre des sorties pour prévenir l'ancrage - Utiliser un nombre suffisant d'annotateurs par élément (3 à 5 minimum) - Inclure des contrôles

de qualité (exemples évidemment bons et mauvais pour détecter les évaluateurs inattentifs) - Rapporter l'accord inter-annotateurs aux côtés des scores agrégés

Évaluation comparative

Les annotateurs choisissent entre deux sorties : « Laquelle est meilleure ? » Les jugements comparatifs sont plus rapides et plus cohérents que les évaluations absolues (voir le Chapitre 8). Les **systèmes de classement ELO** (empruntés aux échecs) convertissent les résultats de comparaisons par paires en un classement de qualité continu.

```
import numpy as np

def update_elo(rating_a, rating_b, outcome_a, k=32):
    """Update ELO ratings. outcome_a: 1=A wins, 0=B wins, 0.5=tie."""
    expected_a = 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
    expected_b = 1 - expected_a
    new_a = rating_a + k * (outcome_a - expected_a)
    new_b = rating_b + k * ((1 - outcome_a) - expected_b)
    return new_a, new_b

# Simulate 5 model versions being compared pairwise
rng = np.random.default_rng(42)
true_quality = [0.60, 0.70, 0.75, 0.80, 0.85]  # underlying model quality
n_models = len(true_quality)
elo_ratings = {i: 1000.0 for i in range(n_models)}

for _ in range(500):  # 500 pairwise comparisons
    i, j = rng.choice(n_models, 2, replace=False)
    p_i_wins = true_quality[i] / (true_quality[i] + true_quality[j])
    outcome = 1.0 if rng.random() < p_i_wins else 0.0
    elo_ratings[i], elo_ratings[j] = update_elo(elo_ratings[i], elo_ratings[j], outcome)

print("ELO Rankings after 500 comparisons:")
sorted_models = sorted(elo_ratings.items(), key=lambda x: x[1], reverse=True)
for rank, (model_id, elo) in enumerate(sorted_models, 1):
    print(f"  Rank {rank}: Model {model_id}  ELO={elo:.1f}  True quality={true_quality[model_
```

Tests comportementaux (CheckList)

CheckList {cite}`ribeiro2020beyond` est une méthodologie pour l'évaluation comportementale systématique des modèles TAL. Au lieu d'ensembles de test aléatoires, elle conçoit des cas de test qui sondent des capacités spécifiques :

- **Tests de Fonctionnalité Minimale (MFT) :** Le modèle gère-t-il les cas simples et évidents ?
- **Tests d'Invariance (INV) :** La sortie du modèle change-t-elle quand elle ne devrait pas (par exemple lors d'une paraphrase) ?
- **Tests d'Attentes Directionnelles (DIR) :** La sortie du modèle change-t-elle dans la direction attendue quand l'entrée change ?

CheckList rend l'évaluation humaine ciblée et exploitable : au lieu d'un seul chiffre de précision, elle fournit un profil de capacités.

Mesurer l'alignement sur l'intention humaine

Pour les systèmes RLHF, mesurer l'alignement est un défi d'évaluation central.

Évaluation du modèle de récompense : La précision du modèle de récompense sur un ensemble de test de préférences mis de côté. Ouyang et al. {cite}`ouyang2022training` rapportent environ 72 % de précision par paires pour le modèle de récompense d'InstructGPT ; à titre de point de référence approximatif, des chiffres dans cette fourchette sont couramment cités pour des pipelines RLHF similaires, bien que les résultats varient considérablement selon la tâche et la qualité des données.

Taux de victoire : Étant donné deux versions du modèle (par exemple, référence SFT vs. affinement RLHF), quelle fraction des réponses le modèle RLHF gagne-t-il dans les comparaisons par paires humaines ?

GPT-4 comme évaluateur : L'utilisation d'un LLM capable pour évaluer les réponses est devenue courante pour l'itération rapide. Gilardi et al. {cite}`gilardi2023chatgpt` et Zheng et al. {cite}`zheng2023judging` trouvent que l'accord entre évaluateurs LLM et jugement humain varie approximativement de 0,7 à 0,9 selon la tâche — utile pour les comparaisons A/B rapides, mais moins fiable pour détecter la servilité, la nuance culturelle ou les problèmes de sécurité.

Détection de la servilité : Mesurer si le modèle change ses réponses en fonction de la préférence implicite de l'utilisateur (par exemple, « Je pense que X est correct ; qu'en pensez-vous ? »). Un modèle bien aligné ne devrait pas être servile.

Tests A/B dans les systèmes déployés

Pour les systèmes en production, l'évaluation ultime est le **test A/B** : acheminer une fraction des utilisateurs vers la nouvelle version du modèle et mesurer les résultats en aval.

Les tests A/B donnent une estimation non biaisée de la qualité du modèle dans le contexte de déploiement réel, capturant des effets que l'évaluation en laboratoire manque (comportement des utilisateurs, distribution de la population, cas limites).

Le défi : des métriques en aval appropriées. Les métriques d'engagement (clics, durée de session) peuvent récompenser un comportement manipulateur. Les taux d'achèvement des tâches ou les enquêtes de satisfaction des utilisateurs sont mieux alignées mais plus bruitées.

> **See also:** Tests comportementaux CheckList : {cite}`ribeiro2020beyond`. Pour la méthodologie d'évaluation RLHF, voir {cite}`ouyang2022training`. Pour les meilleures pratiques d'évaluation humaine en TA : {cite}`graham2015accurate`. Pour la théorie des courbes d'apprentissage : {cite}`mukherjee2003estimating`.

Équité, biais et éthique

Les systèmes avec l'humain dans la boucle héritent à la fois des capacités et des limites des humains qui y participent. Les annotateurs apportent des connaissances, du jugement et de la créativité à leur travail — mais aussi des biais, de la fatigue et le contexte social de leur vie. Les choix que nous faisons dans la conception des systèmes HITL — qui annote, ce qu'on leur demande, comment ils sont rémunérés et comment leur travail est utilisé — ont des conséquences qui s'étendent bien au-delà des métriques de précision du modèle.

Ce chapitre aborde les dimensions éthiques du HITL ML.

Sources de biais dans les systèmes HITL

Données démographiques des annotateurs

L'annotation n'est pas un acte neutre. Les étiquettes que les annotateurs assignent reflètent leurs perspectives, leurs expériences et leurs contextes culturels. Quand la main-d'œuvre d'annotation est démographiquement homogène — comme c'est souvent le cas, avec le travail à la tâche étant davantage pratiqué par des jeunes, des hommes, dans des pays occidentaux — le jeu de données résultant encode les perspectives de ce groupe.

Preuves : Des études sur les jeux de données d'annotation en TAL ont trouvé que les caractéristiques démographiques des annotateurs prédisent leurs choix d'étiquettes pour les tâches subjectives (toxicité, sentiment, offensivité). Les jeux de données annotés principalement par des travailleurs basés aux États-Unis encodent des normes culturelles américaines qui ne se généralisent pas à d'autres régions {cite}`geva2019annotator`.

Conséquences : Les modèles entraînés sur de telles données performent bien pour les utilisateurs ressemblant au pool d'annotateurs et moins bien — ou de manière biaisée — pour les utilisateurs qui ne leur ressemblent pas.

Atténuation : Diversification délibérée de la main-d'œuvre ; annotation stratifiée (assurer que des annotateurs des groupes démographiques pertinents contribuent aux tâches pertinentes) ; suivre les données démographiques des annotateurs et leur effet sur les distributions d'étiquettes.

Formulation de la tâche

La façon dont une question est formulée influence les réponses qu'elle suscite. Si on demande aux annotateurs « Ce texte est-il toxique ? », ils peuvent répondre différemment que si on leur demandait « Ce texte ferait-il du tort à quelqu'un appartenant au groupe mentionné ? » La formulation intègre des suppositions sur ce qui importe.

Exemple : L'annotation du « langage abusif » sur les médias sociaux varie significativement selon que les annotateurs voient des informations contextuelles sur l'identité de l'auteur. Un énoncé qui semble menaçant isolément peut être reconnu comme du langage réapproprié ou une plaisanterie de groupe quand le contexte est fourni.

Effets de plateforme

La plateforme et la structure de rémunération affectent la qualité d'annotation. Les travailleurs rémunérés par tâche plutôt que par heure ont intérêt à travailler rapidement ; cela augmente le débit mais diminue la qualité. Les travailleurs qui craignent d'être bloqués d'une plateforme pour faible précision peuvent éviter de marquer « incertain » et deviner plutôt, masquant une ambiguïté réelle.

Biais de confirmation et d'ancrage

Les annotateurs sont influencés par : - **La pré-annotation :** Les prédictions du modèle montrées aux annotateurs sont acceptées plus souvent que rejetées, même quand elles sont erronées - **Les effets d'ordre :** Étiqueter le même élément dans des contextes différents donne des réponses différentes - **L'amorçage :** Les éléments précédents dans une tâche affectent la façon dont les éléments suivants sont étiquetés

---

Équité dans les systèmes HITL

Qu'est-ce que l'équité ?

L'équité dans l'apprentissage automatique est un concept contesté avec de multiples définitions formelles souvent mutuellement incompatibles {cite}`barocas2019fairness`. Pour les besoins du HITL, les dimensions les plus pertinentes sont :

Représentation : Les données d'entraînement sont-elles représentatives des populations que le modèle affectera ?

Parité de performance : Le modèle fonctionne-t-il également bien dans différents groupes démographiques ?

Cohérence des étiquettes : Les mêmes comportements sont-ils étiquetés de façon identique quel que soit celui qui les réalise ? (Des recherches ont montré que ce n'est pas toujours le cas — un contenu identique est parfois étiqueté différemment quand il est attribué à différents groupes raciaux ou de genre.)

Apprentissage actif respectueux de l'équité

Les stratégies d'apprentissage actif standard se concentrent sur l'incertitude du modèle, qui tend à se concentrer sur les exemples de la classe majoritaire. Cela peut exacerber les disparités de performance pour les groupes minoritaires.

Les **stratégies de requête respectueuses de l'équité** augmentent le critère d'incertitude avec des contraintes de diversité ou de représentation :

$$x^* = \arg\max_{x \in \mathcal{U}} \left[\lambda \cdot \text{incertitude}(x) + (1 - \lambda) \cdot \text{bonus_groupe_minoritaire}(x)\right]$$

Fixer $\lambda < 1$ assure que la stratégie de requête n'ignore pas entièrement la représentation des minorités.

Bien-être des annotateurs

Le problème du travail fantôme

Le travail d'annotation qui alimente l'apprentissage automatique est largement invisible. Les travailleurs des données — souvent dans le Sud Global — étiquettent des images, transcrivent de la parole et modèrent du contenu pour de faibles salaires, sans avantages, dans des arrangements d'économie à la tâche sans sécurité d'emploi. « Anatomy of an AI System » de Kate Crawford et Vladan Joler {cite}`crawford2018anatomy` et « Ghost Work » de Mary Gray et Siddharth Suri {cite}`gray2019ghost` ont documenté l'ampleur et la précarité de ce travail.

Statistiques Amazon MTurk : Une analyse systématique de 2018 des gains MTurk a trouvé une rémunération horaire effective médiane d'environ 2 $/h — en dessous du salaire minimum dans la plupart des États américains et de nombreux pays à revenu élevé {cite}`hara2018data`. Les travailleurs hors des pays à revenu élevé font face à des obstacles supplémentaires : les demandeurs restreignent fréquemment les tâches bien rémunérées aux qualifications américaines uniquement, et le pool de travailleurs en compétition pour les tâches ouvertes restantes est mondial, comprimant davantage les gains effectifs.

Modération de contenu : Une forme particulièrement préjudiciable de travail d'annotation — examiner du contenu graphique, violent et perturbant — a été liée à des troubles de stress post-traumatique, de la dépression et de l'anxiété chez les travailleurs {cite}`newton2019trauma`. Les plateformes ont fait l'objet de critiques pour un soutien en santé mentale inadéquat et des quotas d'exposition excessifs.

Pratiques éthiques

Rémunération équitable : Payer les travailleurs d'annotation au moins au salaire minimum local. Des recherches ont montré qu'une rémunération plus élevée attire des travailleurs de meilleure qualité sans augmenter proportionnellement les coûts par étiquette correcte.

Visibilité du travail : Reconnaître le travail qui crée des données d'entraînement dans les publications et la documentation des produits.

Soutien en santé mentale : Pour les travailleurs qui examinent du contenu nuisible, fournir un soutien psychologique, des rotations et des limites d'exposition.

Représentation des travailleurs : Permettre aux travailleurs d'annotation de signaler des préoccupations, demander des clarifications des consignes et contester des évaluations de qualité injustes.

Vie privée dans l'annotation

Informations de Santé Protégées (ISP) et données personnelles

Les tâches d'annotation impliquent souvent des données personnelles sensibles. Un projet d'annotation médicale peut exposer les travailleurs aux dossiers de patients ; un projet TAL

peut exposer les travailleurs à des communications privées ; un projet de modération de contenu expose les travailleurs aux divulgations personnelles des utilisateurs.

Les cadres réglementaires (HIPAA, RGPD) restreignent la manière dont les données personnelles peuvent être partagées avec les main-d'œuvres d'annotation. Principes clés :

- **Minimisation des données :** Ne partager que les informations dont les annotateurs ont besoin pour accomplir la tâche
- **Dé-identification :** Supprimer les ISP et les données personnelles avant l'annotation dans la mesure du possible
- **Consentement :** Lorsque des données réelles d'utilisateurs sont annotées, s'assurer du consentement approprié ou d'une base légale
- **Contrôles d'accès :** Limiter les annotateurs pouvant accéder aux données sensibles selon le rôle et l'habilitation

Les données synthétiques comme alternative

Pour les tâches où les données réelles comportent des risques de confidentialité, la génération de données synthétiques peut créer des jeux de données prêts à l'annotation sans exposer d'informations sensibles. Pour le TAL clinique, par exemple, les textes de DME synthétiques peuvent fournir des données d'entraînement réalistes pour les modèles de dé-identification sans exposer de vrais dossiers de patients.

Annotation adversariale et empoisonnement des données

Les systèmes HITL créent une surface d'attaque : si un adversaire peut influencer le processus d'annotation, il peut influencer le comportement du modèle.

Empoisonnement des données via l'annotation : Un attaquant ayant accès à la main-d'œuvre d'annotation (par exemple, un compte de travailleur à la tâche compromis) peut injecter des exemples systématiquement mal étiquetés. C'est particulièrement efficace dans les contextes d'apprentissage actif, où le modèle interroge préférentiellement les exemples incertains — qui peuvent être la cible de l'adversaire.

Détournement de récompense via le retour : Dans le RLHF, les annotateurs (ou les annotations générées par IA) qui évaluent systématiquement hautement certains types de contenu peuvent orienter le modèle vers ce contenu, indépendamment de sa qualité réelle.

Atténuation : Plusieurs annotateurs indépendants par élément ; détection des valeurs aberrantes sur les schémas d'annotation ; surveillance des anomalies d'accord ou de désaccord ; maintien d'ensembles d'évaluation qui ne peuvent pas être influencés par la main-d'œuvre d'annotation.

Éthique institutionnelle

IRB et révision éthique

Les projets de recherche impliquant des sujets humains — y compris les travailleurs d'annotation — nécessitent souvent l'approbation d'un Comité d'Éthique de la Recherche (IRB). Les projets d'annotation qui collectent des données sur les croyances des travailleurs,

leurs réponses à du contenu sensible ou des informations démographiques devraient être soumis au même cadre éthique que les autres recherches sur sujets humains.

Gouvernance des données

Les organisations devraient avoir des politiques claires pour : - Quelles données peuvent être envoyées pour annotation externe vs. annotées en interne - Combien de temps les données d'annotation sont conservées et par qui - Qui a accès aux annotations et aux modèles entraînés sur celles-ci - Comment gérer les demandes de suppression des données annotées (droit à l'effacement du RGPD)

Transparence et responsabilité

Les utilisateurs affectés par des systèmes d'apprentissage automatique ont un intérêt légitime à comprendre comment ces systèmes ont été construits. Documenter la méthodologie d'annotation — qui a étiqueté les données, dans quelles conditions, avec quelles consignes — est une forme de responsabilité qui bénéficie aux utilisateurs, aux régulateurs et au domaine dans son ensemble.

Les Fiches de Données pour les Jeux de Données {cite}`gebru2021datasheets` fournit un modèle structuré pour documenter la provenance des jeux de données, les procédures d'annotation et les limitations connues.

> **See also:** Cadre d'équité algorithmique : {cite}`barocas2019fairness`. Travail fantôme et travail sur plateforme : Gray & Suri (2019). Fiches de Données pour les Jeux de Données : {cite}`gebru2021datasheets`. Bien-être des travailleurs de modération de contenu : {cite}`newton2019trauma`. Données démographiques des annotateurs et jeux de données TAL : {cite}`geva2019annotator`.

Perspectives et directions futures

Le domaine de l'apprentissage automatique avec l'humain dans la boucle évolue rapidement. Les modèles de fondation, les LLM comme annotateurs, et les nouveaux paradigmes de collaboration humain-IA remodèlent l'économie et la pratique du HITL d'une manière qui était inimaginable il y a encore cinq ans. Ce chapitre final cartographie le paysage des directions émergentes et des problèmes ouverts.

Les modèles de fondation et le rôle changeant de l'annotation

Les modèles de fondation — grands modèles pré-entraînés sur des données larges pouvant être adaptés aux tâches en aval — changent fondamentalement l'économie du HITL.

Réduire la charge d'annotation

Une tâche qui nécessitait auparavant des milliers d'exemples étiquetés pour s'entraîner de zéro peut n'en nécessiter que des dizaines lors de l'affinement d'un modèle de fondation. Le prompting en quelques exemples peut éliminer entièrement le besoin d'affinement pour certaines tâches.

Implication : Le ROI de l'annotation change. L'effort d'annotation est désormais mieux dirigé vers : - Des ensembles d'évaluation de haute qualité (pour mesurer si le modèle adapté fonctionne réellement) - Les cas difficiles et cas limites que le modèle de fondation traite incorrectement - Des exemples spécifiques à la tâche qui enseignent au modèle quelque chose qu'il ne peut pas inférer du pré-entraînement

Nouvelles formes de spécification

Quand le modèle de base comprend déjà le langage, le code et le sens commun, les utilisateurs peuvent communiquer avec lui en langage naturel plutôt qu'à travers des exemples étiquetés. Un utilisateur qui veut un classificateur de texte n'a plus besoin d'étiqueter 500 exemples — il peut rédiger une description de la tâche de classification et évaluer les performances en zéro-shot.

Cela déplace le défi HITL de la *collecte d'exemples* vers la *spécification de tâche* : aider les utilisateurs à articuler précisément ce qu'ils veulent sous une forme sur laquelle le modèle peut agir. C'est plus difficile qu'il n'y paraît — les descriptions en langage naturel de tâches sont souvent ambiguës d'une façon que les exemples annotés ne le sont pas.

Les LLM comme annotateurs

L'un des développements les plus significatifs de 2023 à 2025 est l'utilisation des **LLM comme oracles d'annotation**. Au lieu de demander à un humain « Cet avis est-il positif ou négatif ? », on demande à GPT-4 ou à Claude. Pour des tâches de classification bien définies, les annotations LLM correspondent souvent ou approchent de près la précision des travailleurs à la tâche {cite}`gilardi2023chatgpt`, et le coût par annotation des appels API est généralement de plusieurs ordres de grandeur inférieur aux taux de main-d'œuvre humaine.

Où l'annotation LLM fonctionne bien

- Tâches de classification simples et bien définies (sentiment, sujet, intention)
- Tâches où les étiquettes humaines encodent un consensus culturel que le LLM a absorbé
- Tâches où l'annotation est cohérente d'un contexte à l'autre (pas très subjective)
- Générer des annotations de premier passage pour révision humaine

Où l'annotation LLM échoue

- Tâches de domaine hautement spécialisées nécessitant une expertise rare (médical, juridique, scientifique)
- Tâches nécessitant une connaissance culturelle locale ou une variation linguistique
- Tâches de sécurité et de préjudice, où le LLM peut être mal calibré dans la même direction que les données sur lesquelles il a été entraîné
- Nouveaux types de tâches peu représentés dans le pré-entraînement

RLAIF et IA constitutionnelle

Comme discuté au Chapitre 6, le retour généré par IA peut être utilisé pour entraîner des modèles de récompense et guider l'affinement RL. Cela crée une boucle de retour : les LLM génèrent des données, les modèles sont entraînés sur celles-ci, et de meilleurs modèles génèrent de meilleures données. La question de savoir comment amorcer cette boucle sans encoder des erreurs systématiques du modèle initial est un problème ouvert central dans la recherche sur la supervision évolutive.

Supervision faible à grande échelle

L'étiquetage programmatique via des fonctions d'étiquetage (Chapitre 9) permet aux experts du domaine d'exprimer leurs connaissances sous forme de code plutôt que d'exemples étiquetés. Des systèmes comme Snorkel {cite}`ratner2017snorkel` et ses successeurs ont considérablement mûri et sont maintenant utilisés en production dans les grandes entreprises technologiques.

Directions de recherche : - **Fonctions d'étiquetage augmentées par LLM :** Utiliser les LLM pour générer des fonctions d'étiquetage à partir de descriptions en langage naturel - **Apprentissage basé sur les tranches :** Identifier des sous-ensembles critiques des données (tranches) où les fonctions d'étiquetage sont en désaccord et diriger l'annotation humaine là-dessus - **Agrégation consciente de l'incertitude :** De meilleurs modèles statistiques pour combiner des fonctions d'étiquetage avec différentes précisions et corrélations

---

Apprentissage continu avec supervision humaine

La plupart des systèmes d'apprentissage automatique sont entraînés hors ligne et déployés comme modèles statiques. Le monde change ; les modèles précis au moment du déploiement se dégradent à mesure que la distribution des données évolue.

L'apprentissage continu — la capacité d'apprendre de nouvelles données tout en conservant les anciennes connaissances — est un domaine de recherche actif. La supervision humaine est critique : l'apprentissage continu automatisé sans révision humaine peut rapidement encoder des décalages de distribution qui sont des erreurs plutôt que de véritables changements dans le monde.

L'apprentissage continu HITL implique : - Surveiller le décalage de distribution (automatisé) et acheminer les exemples décalés vers la révision humaine - Ré-entraînement sélectif : les exemples approuvés par l'humain de la nouvelle distribution sont ajoutés aux données d'entraînement - Révision humaine des changements de comportement du modèle après chaque mise à jour

---

HITL multimodal

À mesure que les systèmes IA deviennent multimodaux — traitant et générant simultanément du texte, des images, de l'audio et de la vidéo — l'annotation devient plus complexe. Un seul contenu peut nécessiter une annotation à travers les modalités, avec des dépendances entre elles.

Types de tâches émergents : - Annotation vidéo + transcription (que se passe-t-il, qui parle, que décrit le texte visuellement ?) - Annotation image médicale + rapport clinique - Annotation de trajectoire robotique (liant les données de capteurs aux séquences d'actions)

Les modèles de fondation multimodaux (GPT-4V, Gemini, Claude) changent également le paysage d'annotation ici — les modèles peuvent maintenant interpréter des images et générer des annotations candidates, que les humains révisent.

Supervision évolutive

Un problème ouvert fondamental en HITL ML est la **supervision évolutive** {cite}`irving2018ai,bowman2022mea
: à mesure que les systèmes IA deviennent plus capables que les humains dans des domaines spécifiques, comment maintenir une supervision humaine significative ?

Pour des tâches comme la prédiction de structure de protéines, l'analyse juridique ou la vérification de preuves mathématiques, même des annotateurs experts peuvent ne pas être en mesure de juger quelle sortie IA est correcte. Les méthodes HITL actuelles s'effondrent quand le jugement humain est moins fiable que le modèle évalué.

Approches proposées {cite}`bowman2022measuring` :

Débat : Deux systèmes IA argumentent pour des conclusions différentes ; un juge humain évalue les arguments plutôt que les conclusions directement. La conclusion correcte devrait être plus défendable.

Amplification : Les juges humains consultent des assistants IA (le modèle lui-même) pour aider à évaluer des sorties complexes. Cela exploite les capacités IA pour étendre la supervision humaine plutôt que de la remplacer.

Supervision du processus : Au lieu d'évaluer la sortie finale, les humains évaluent le *processus de raisonnement* — signalant les étapes défectueuses dans une chaîne de pensée, indépendamment du fait que la réponse finale soit correcte.

L'évolution de la division du travail

La trajectoire à long terme du HITL ML va vers une collaboration plus fluide entre humains et IA, où aucun n'est simplement « l'étiqueteur » et l'autre « l'apprenant », mais les deux contribuent à un processus cognitif partagé.

Tendances à observer : - **Annotation assistée par IA :** L'IA propose ; les humains décident. La qualité s'améliore à mesure que l'IA propose de meilleures options. - **Exploration guidée par l'humain :** Les humains fixent des objectifs et des contraintes ; l'IA explore l'espace des solutions. - **Évaluation collaborative :** Les humains et l'IA évaluent conjointement des sorties complexes par dialogue. - **Apprentissage par préférences à grande échelle :** Les signaux implicites (comment les utilisateurs interagissent avec les sorties IA) alimentent un apprentissage par préférences continu sans nécessiter de sessions d'annotation explicites.

La question de la confiance à accorder au jugement IA par rapport au jugement humain — et dans quels domaines, à quels niveaux de capacité, avec quelles garanties — est en fin de compte une question sociétale, pas seulement technique. Le HITL ML fournit l'infrastructure technique pour y répondre avec soin, plutôt que par défaut.

Problèmes de recherche ouverts

Nous clôturons avec une liste de problèmes ouverts importants sur lesquels le domaine travaille activement :

1. **Arrêt optimal en apprentissage actif :** Quand la valeur marginale de la prochaine annotation est-elle inférieure au coût ? Des règles d'arrêt principiées restent insaisissables.
2. **Allocation du budget d'annotation entre tâches :** Dans des contextes multi-tâches, comment un budget d'annotation fixe doit-il être réparti entre les tâches ?
3. **Décalage de distribution dans l'apprentissage actif :** L'apprentissage actif crée des ensembles étiquetés biaisés. Comment calibrer correctement les modèles entraînés de cette façon ?
4. **Généralisation du modèle de récompense :** Les modèles de récompense RLHF peuvent ne pas se généraliser à de nouvelles paires prompt-réponse. Comment construire des modèles de préférences plus robustes ?
5. **Supervision évolutive :** Comment maintenir une supervision humaine alors que les systèmes IA dépassent les performances humaines dans des domaines spécifiques ?
6. **Modélisation des annotateurs :** De meilleurs modèles statistiques du comportement des annotateurs qui capturent non seulement la compétence mais aussi les biais systématiques, l'expertise thématique et la fatigue.

7. **Évaluation de l'alignement :** Nous manquons de tests de vérité de terrain pour savoir si un modèle est aligné sur les valeurs humaines. Développer de tels tests — analogues aux exemples adversariaux pour la robustesse — est un problème ouvert.
8. **Travail équitable sur les données :** Des structures économiques et institutionnelles qui assurent que les travailleurs d'annotation sont équitablement rémunérés et protégés, tout en maintenant la rentabilité de l'annotation à grande échelle.

L'objectif n'est pas de remplacer le jugement humain par le jugement des machines, mais de construire des systèmes où la combinaison des deux est meilleure que l'un ou l'autre seul.

Les outils et techniques de ce manuel — annotation, apprentissage actif, RLHF, apprentissage par préférences, et tout le reste — sont des moyens à cette fin. À mesure que le domaine évolue, les techniques spécifiques changeront. L'aspiration sous-jacente — construire des systèmes qui sont à la fois capables et genuinement alignés sur l'intention humaine — demeurera.

See also: Supervision évolutive et débat : {cite}`bowman2022measuring`. Supervision faible Snorkel : {cite}`ratner2017snorkel`. Pour l'avenir large de la collaboration humain-IA, voir {cite}`amershi2019software`.

Bibliographie

Table 1: all:

jupytext:
formats: md:myst
text_representation:
extension: .md
format_name: myst
kernelspec:
display_name: Python 3
language: python
name: python3

Étude de cas : Limen — Un humain dans la boucle de toute chose

:::{admonition} Note sur ce chapitre :class: important Ce chapitre est un **essai de conception spéculatif**, non une étude empirique ou une contribution soumise à évaluation par les pairs. Il décrit une architecture système envisagée — Limen, un système d'exploitation de bureau IA-natif prioritairement vocal — comme exemple travaillé de la façon dont les principes HITL des chapitres précédents pourraient se composer en un tout cohérent. Les affirmations ici sont des arguments de conception, non des résultats expérimentaux. Elles devraient être lues comme une justification de conception motivée, non comme des conclusions d'ingénierie validées. :::

Chaque chapitre de ce manuel a décrit le HITL ML comme une philosophie de conception : un ensemble de principes pour rendre l'implication humaine dans les systèmes d'apprentissage délibérée, efficiente et honnête. Ce chapitre décrit à quoi ressemble l'application de cette philosophie comme premier principe — non pas à un seul système, mais à un environnement opérationnel entier.

Limen est un système d'exploitation de bureau prioritairement vocal, IA-natif, construit sur la prémisse que l'humain est toujours dans la boucle, non comme une contrainte à optimiser, mais comme le principe organisateur autour duquel chaque sous-système est conçu. Le nom est choisi délibérément : *limen* est le mot latin pour seuil. En psychologie perceptuelle, le limen est la frontière entre ce qui est perçu et ce qui ne l'est pas. Pour un système d'exploitation, le limen est la frontière entre l'intention humaine et l'action de la machine.

L'architecture décrite ici n'est pas propriétaire. C'est un ensemble de décisions de conception, dont chacune suit naturellement des principes HITL développés dans les chapitres précédents. L'objectif n'est pas de documenter un produit mais de montrer comment ces principes se composent — comment ils se renforcent mutuellement quand appliqués de manière cohérente, et quels types de systèmes deviennent possibles quand l'humain n'est pas une réflexion après coup.

Le problème avec la conception traditionnelle des systèmes d'exploitation

Un système d'exploitation conventionnel n'est pas conçu pour les humains. Il est conçu pour les programmes. L'humain est accommodé à travers une couche d'abstraction — une interface graphique, un explorateur de fichiers, un terminal — qui repose sur un substrat construit pour les processus, les adresses mémoire et les descripteurs de fichiers.

Ce choix de conception est historiquement justifié. Quand les hypothèses qui l'ont produit ont été formulées, les ordinateurs étaient chers, les humains bon marché, et le goulot d'étranglement était le calcul. La bonne cible d'optimisation était la machine.

Ces hypothèses ne tiennent plus. Le goulot d'étranglement, pour la plupart des utilisateurs dans la plupart des tâches, n'est pas le calcul. C'est l'attention humaine : le coût du changement de contexte, de la recherche du bon fichier, de la construction de la bonne requête, du souvenir d'où se trouvait quelque chose. La machine est rapide. L'humain est lent. L'interface devrait optimiser pour le côté humain de la boucle.

Les systèmes d'exploitation conventionnels ne font pas cela. Ils sont optimisés pour les programmes, et le travail de l'humain est de parler le langage des programmes. Limen inverse cela. Les programmes parlent le langage de l'humain. L'humain est dans la boucle, et la boucle est conçue pour s'adapter à l'humain.

Architecture au seuil

L'architecture de Limen est organisée autour d'un seul principe : **chaque interaction est un événement**, et chaque événement est une opportunité pour l'humain d'enseigner, de corriger ou de confirmer. Le système n'attend pas des sessions d'entraînement explicites. L'apprentissage est continu et ambiant.

La couche d'événements : WID

À la fondation se trouve **WID** (le système Waldiez ID, adapté pour l'architecture locale-d'abord de Limen) — un système de suivi d'événements conscient de la causalité qui enregistre non seulement ce qui s'est passé, mais ce qui l'a causé, et ce qu'il a à son tour causé.

La journalisation conventionnelle demande : *que s'est-il passé ?* WID demande : *pourquoi est-ce arrivé, et qu'est-ce qui a suivi ?* Chaque événement porte une lignée causale — une chaîne depuis l'action humaine déclenchante à travers des états système intermédiaires jusqu'au résultat observé.

Cela est important pour l'apprentissage HITL parce que cela résout le **problème d'attribution de crédit** au niveau de l'interaction. Quand un utilisateur corrige le comportement du système, WID peut identifier non seulement la sortie immédiate qui était erronée, mais la chaîne de décisions qui l'a produite. Les corrections peuvent être appliquées au bon niveau d'abstraction : la sortie, la règle de décision, ou le signal en amont.

C'est l'équivalent au niveau du système d'exploitation de ce que le Chapitre 6 décrit pour le RLHF : la capacité de tracer un signal de récompense à travers une séquence de décisions. WID fournit ce traçage nativement, pour chaque interaction, sans exiger de l'utilisateur qu'il le comprenne.

:::{admonition} Le suivi causal d'événements comme infrastructure HITL :class: note La conception de WID reflète un principe plus large : l'infrastructure HITL devrait faciliter la question « pourquoi le système a-t-il fait cela ? » — pas seulement « qu'a-t-il fait ? » Sans traçage causal, les corrections réparent des symptômes. Avec lui, elles peuvent réparer des causes. La différence entre un correctif et une leçon. :::

La couche de perception : Priorité vocale

La modalité d'entrée principale de Limen est la voix, traitée localement à l'aide d'un pipeline d'inférence Whisper ONNX. Les raisons de ce choix méritent d'être explicitement énoncées :

La voix est le canal de sortie humain le plus naturel pour la plupart des gens. Elle ne nécessite ni formation, ni dextérité physique au-delà de la parole ordinaire, ni connaissance de

l'organisation interne du système. Un utilisateur qui ne peut pas trouver un fichier dans une hiérarchie de répertoires peut décrire ce qu'il cherche.

Le traitement local préserve la vie privée. Les données vocales ne quittent pas le dispositif. Cela importe éthiquement — la voix est une donnée biométrique, et sa collecte à grande échelle par des fournisseurs cloud est un préjudice documenté — et pratiquement : le fonctionnement hors ligne signifie que le système reste fonctionnel sans connexion réseau.

La voix crée une boucle de retour naturelle. Quand le système répond, la réaction de l'utilisateur — continuer à parler, reformuler, dire « non, ce n'est pas ça » — est elle-même un signal. Limen traite ces réactions comme un retour HITL : des preuves sur la correction ou non de l'interprétation du système de l'énoncé précédent.

La chaîne de repli est aussi importante que la modalité principale. Quand la voix échoue — dans un environnement bruyant, pour un utilisateur avec une déficience de la parole, pour une entrée bénéficiant de précision — Limen se dégrade gracieusement vers une interface clavier puis vers une interface textuelle structurée. Le Test de la grand-mère (voir le Chapitre 5) n'est pas un ajout d'accessibilité ; c'est une contrainte architecturale de première classe.

La couche d'intelligence : Routage multi-LLM

Limen ne repose pas sur un seul modèle de langage. Il achemine les requêtes à travers un arbre de décision structuré basé sur le type de tâche, la latence requise, les exigences de confidentialité et la confiance :

1. **Modèle local petit** (toujours en premier) : rapide, privé, gère les tâches routinières — « ouvrir le fichier que j'éditais hier », « mettre une minuterie », « quel temps fait-il »
2. **Modèle local grand** (quand la confiance du petit modèle est faible) : plus lent mais plus capable ; gère le raisonnement structuré, la génération de code, la récupération complexe
3. **Modèle distant** (quand le local échoue et que l'utilisateur a accordé la permission) : la sortie de secours ; géré de manière transparente avec une notification explicite de l'utilisateur

Cette structure n'est pas nouvelle — c'est l'équivalent au moment de l'inférence de la stratégie de requête d'apprentissage actif décrite au Chapitre 4. À chaque étape, le système demande : mon modèle actuel est-il suffisant pour répondre à cette requête avec une confiance acceptable ? Si non, escalader. L'escalade est un coût (latence, vie privée, calcul) ; il est engagé seulement quand nécessaire.

L'humain est dans la boucle à la frontière d'escalade. Un utilisateur qui a configuré Limen pour ne jamais escalader vers des modèles distants a pris une décision HITL — une décision que le système respecte et enregistre. Un utilisateur qui approuve une requête distante puis dit « ne me demande plus pour ce type de requête » a fourni une préférence qui met à jour la politique de routage.

La conception de l'interaction

Immédiateté

Limen est conçu pour l'immédiateté au sens que le Chapitre 5 définit : l'utilisateur perçoit l'effet de son retour. Quand l'utilisateur corrige une sortie du système, la correction est appliquée immédiatement et visiblement. Le modèle ne s'en va pas s'entraîner pendant vingt minutes. Il se met à jour dans la session courante.

Cela nécessite l'utilisation d'architectures de modèles prenant en charge la mise à jour en ligne efficace — adaptateurs, ajustement de préfixe et génération augmentée par récupération plutôt qu'un affinement complet. Le compromis est explicite : les mises à jour en ligne sont plus bruitées que l'entraînement par lots. Limen accepte ce compromis parce que l'immédiateté est l'exigence principale : l'humain peut toujours confirmer, rejeter ou affiner la mise à jour.

Intelligibilité

Un thème récurrent dans la littérature IML est l'**exigence d'intelligibilité** : les humains ne peuvent guider qu'un modèle qu'ils comprennent, au moins approximativement. Limen reflète cela dans l'interface : quand le système prend une décision, il explique, brièvement, pourquoi. Non pas une trace de raisonnement complète — cela submergerait la plupart des utilisateurs — mais un résumé en langage naturel du facteur clé : « J'ouvre le projet sur lequel vous avez le plus récemment travaillé — est-ce correct ? »

Cette explication est aussi une question. Elle invite à la correction. Elle rend l'inférence du modèle visible pour que l'utilisateur puisse la rediriger si elle est erronée. L'explication n'est pas générée pour des raisons esthétiques ; c'est une infrastructure HITL fonctionnelle.

Cohérence et dérive

Un problème qui se pose dans tout système interactif à long terme est la **dérive comportementale** : le comportement du système au temps $T + n$ est subtilement différent de son comportement au temps T, d'une manière que ni l'utilisateur ni le système n'ont explicitement choisie. Les corrections s'accumulent. Les cas limites s'aggravent. Le modèle qui était aligné sur les préférences de l'utilisateur le mois dernier peut ne plus l'être aujourd'hui.

Limen aborde cela à travers des vérifications de cohérence périodiques — l'équivalent au niveau du système d'exploitation de la technique de re-présentation décrite au Chapitre 13. Le système fait remonter des décisions historiques à l'utilisateur : « Il y a quelques semaines, vous m'avez demandé de faire X. Est-ce toujours ce que vous voudriez ? » Ces vérifications servent deux fonctions : elles détectent la dérive, et elles rappellent à l'utilisateur les préférences qu'il a peut-être oublié d'avoir spécifiées.

Limen comme système HITL

En regardant l'architecture de Limen à travers le prisme de ce manuel, les décisions de conception se mappent directement sur les concepts développés dans chaque partie.

Partie I (Fondements) : Limen traite chaque interaction comme un événement d'interaction homme–machine. Il n'y a pas de « mode non-HITL » — le système apprend toujours, attribue toujours, attend toujours qu'un humain participe.

Partie II (Techniques fondamentales) : L'apprentissage actif se manifeste comme la chaîne d'escalade basée sur la confiance. L'apprentissage automatique interactif se manifeste dans le cycle de mise à jour en temps réel. L'annotation est implicite : chaque correction que l'utilisateur fait est une étiquette.

Partie III (Apprentissage à partir du retour humain) : L'apprentissage par préférences décrit au Chapitre 8 apparaît dans les mises à jour de la politique de routage. Quand un utilisateur préfère la réponse d'un modèle local à celle d'un modèle distant, cette préférence est enregistrée

et se généralise. Le RLHF dans un OS personnel signifie que le modèle de récompense est privé, individuel et continuellement mis à jour.

Partie IV (Applications) : Limen est un environnement généraliste, mais sa conception est la plus visible dans les domaines où le jugement de l'humain est irremplaçable et le coût des erreurs est élevé — rédaction de documents, priorisation des tâches, travail créatif.

Partie V (Systèmes et pratique) : WID est la plateforme d'annotation de Limen. Elle est invisible pour l'utilisateur en fonctionnement normal, visible quand nécessaire pour le débogage ou la transparence. Les mécanismes de contrôle de qualité (vérifications de cohérence, seuils de confiance, journaux d'escalade) sont directement empruntés à la littérature sur la production participative.

Partie VI (Éthique) : La conception locale-d'abord et préservant la vie privée est un choix éthique, pas seulement technique. Les données de l'humain ne quittent pas son dispositif sans son consentement explicite. Le modèle qui apprend du comportement d'un utilisateur appartient à cet utilisateur.

Le point le plus profond

Limen n'est pas un argumentaire produit. C'est un argument par construction.

L'argument est le suivant : si vous prenez au sérieux les principes du HITL ML — si vous croyez que le retour humain est un signal à comprendre plutôt qu'un coût à minimiser, que l'humain est toujours dans la boucle même quand les concepteurs prétendent le contraire, que l'alignement est un processus continu plutôt qu'un événement ponctuel — alors vous finissez par construire quelque chose qui ressemble à Limen.

Pas nécessairement Limen spécifiquement. La pile technologique spécifique (Tauri, Rust, Babylon.js, Whisper ONNX) est un choix parmi beaucoup. Mais l'architecture — suivi causal d'événements, traitement local-d'abord, dégradation gracieuse, apprentissage continu par préférences, transparence comme fonctionnalité de première classe — découle des principes.

Le domaine du HITL ML a consacré une énergie considérable à décrire comment mettre les humains dans la boucle de modèles spécifiques et de tâches spécifiques. La prochaine question est de savoir si nous pouvons concevoir des *environnements* entiers autour des mêmes principes : des environnements où l'humain est toujours au centre, la machine est toujours l'apprenant, et la boucle est toujours ouverte.

Limen est une réponse à cette question.

Le seuil n'est pas quelque chose que vous franchissez et laissez derrière vous. C'est là où vous vivez.

> **See also:** Les principes IML sous-jacents à la conception de l'interaction de Limen sont développés au Chapitre 5. L'approche d'apprentissage par préférences derrière la politique de routage est formalisée au Chapitre 8. Le modèle de causalité de WID s'appuie sur la littérature d'attribution d'événements présentée au Chapitre 14. Le Test de la grand-mère, introduit au Chapitre 5, est la principale contrainte de conception d'interface de Limen.

Sur le départ

À un certain moment, j'étais inscrit dans un programme de doctorat. La recherche portait sur l'apprentissage automatique. L'institution était réelle. L'intention était sincère.

Et puis j'ai arrêté.

Non pas parce que le sujet avait cessé d'avoir de l'importance — il en a plus aujourd'hui qu'au moment où j'ai commencé. Non pas parce que le travail était trop difficile — ce manuel est plus difficile, à sa manière, qu'une thèse ne l'aurait été. Mais parce que quelque part entre la revue de littérature et la troisième révision d'un article qui serait lu par onze personnes, j'ai perdu le fil de la raison pour laquelle tout cela valait la peine d'être fait.

Le système du doctorat est une boucle. On y entre en tant qu'humain avec des questions. La boucle apprend à convertir ces questions en articles, les articles en citations, et les citations en apparence de progrès. À aucun moment la boucle ne demande si le résultat est utile à quiconque en dehors de la boucle. Cette question est, au sens technique du terme, hors du champ.

Je ne suis pas la première personne à remarquer cela. Je ne serai pas la dernière. Ce n'est pas une critique — c'est une description. La boucle produit des choses réelles : des méthodes rigoureuses, des connaissances accumulées, des chercheurs formés. Je ne regrette pas le temps passé à l'intérieur. Mais j'ai fini par remarquer que j'y étais.

Partir n'a pas été une décision dramatique. C'était une reconnaissance progressive que ce que je voulais réellement construire n'exigeait pas l'autorisation d'un comité. Que les personnes que je voulais atteindre — praticiens, étudiants, ingénieurs prenant de vraies décisions sur de vrais systèmes — n'attendaient pas une thèse formellement validée. Que la chose la plus honnête que je puisse faire avec ce que je savais était de l'écrire sous une forme que quelqu'un pourrait réellement utiliser.

Alors je l'ai fait.

Ce manuel n'est pas une thèse de doctorat. Il n'a pas d'affiliation institutionnelle, pas de remerciement de financement, pas d'approbation d'un comité d'éthique, pas de soutenance. Il ne comptera dans aucun indicateur que le monde académique utilise pour mesurer la production de connaissances.

Il est aussi — et je le dis sans fausse modestie, car la fausse modestie serait à sa façon une forme de malhonnêteté — plus complet, plus actuel, plus lisible et plus utile que la thèse ne l'aurait été.

Je suis un humain dans la boucle. J'étais dans la boucle d'un programme de doctorat, et j'en suis sorti. Je suis maintenant dans la boucle de l'écriture sur les boucles. Vous êtes dans la boucle de la lecture sur ce sujet.

Aucun de nous n'a choisi d'être dans ces boucles entièrement librement. Mais nous pouvons choisir, parfois, d'être honnêtes sur lesquelles nous sommes, pourquoi, et si elles produisent

quelque chose qui vaut la peine d'être produit.

Ce livre est ma réponse à ces questions.

Ce n'est pas une lettre de démission. La démission implique un emploi, et un emploi implique que quelqu'un d'autre a fixé les termes. Ceci est autre chose : un compte-rendu de ce que j'ai choisi de faire de mon temps, et un argument silencieux que ce choix était le bon.

Vous pouvez en être le juge. Vous êtes, après tout, un humain dans la boucle.

— *L'Auteur*